

Rediseño de las clases de potenciales
Análisis de problemas y opciones de diseño
org.openmarkov.core — paquete model.network.potential

Manuel Arias Calleja

Marzo de 2026

Índice

1. Catálogo de problemas de diseño	5
1.1. Interdependencias entre problemas	7
2. Fundamentos teóricos	8
2.1. Principios de diseño orientado a objetos	8
2.2. Antipatrones encontrados	9
2.3. Patrones de diseño aplicados	10
3. Situación anterior: diagnóstico detallado	10
3.1. Jerarquía de clases	10
3.2. Problema 1: <code>TablePotential</code> con tres responsabilidades	11
3.3. Problema 2: métodos abstractos que las subclases no pueden implementar	12
3.4. Problema 3: <code>GTablePotential<E></code> viola el contrato de su padre	13
3.5. Problema 4: <i>downcasts</i> en inferencia	13
3.6. Problema 5: dependencia cíclica entre <code>TablePotential</code> y <code>StrategyTree</code>	13
3.7. Problema 6: <code>PotentialRole</code> como sustituto de tipos	14
4. Opciones de rediseño	14
4.1. Opción 1 — Extracción mínima de responsabilidades	14
4.2. Opción 2 — Jerarquía basada en roles	16
4.3. Opción 3 — Composición mediante interfaces	18
4.4. Opción 4 — Patrón Visitor para separar operaciones de datos	20
4.5. Opción 5 — Potenciales inmutables con <i>builder</i>	23
5. Comparativa de opciones	26
6. Valoración y recomendación	28
6.1. Si el objetivo es la estabilidad a corto plazo	28
6.2. Si hay apetito para un rediseño sustancial	28
6.3. Lo que se desaconseja	28
6.4. Enfoque incremental recomendado	28
I Rediseño 1: Implementación de la Opción 1	29
7. Resumen de la implementación	29

8. Fundamentos teóricos del rediseño	29
8.1. SRP aplicado: responsabilidades de <code>TablePotential</code>	29
8.2. LSP aplicado: violación en <code>GTablePotential</code>	30
8.3. Despacho virtual (<i>virtual dispatch</i>) y método plantilla	30
8.4. Constructor de copia y <code>deepCopy()</code> por reflexión	31
9. Jerarquía antes del rediseño	32
10. Jerarquía después del rediseño	33
11. Detalle de las fases completadas	35
11.1. Fase 0: Verificación de la línea base	35
11.2. Fase 1: Creación de <code>AbstractIndexedPotential</code>	35
11.3. Fase 2: Extracción de <code>StrategicTablePotential</code>	36
11.4. Fase 3: Extracción de <code>UncertainTablePotential</code>	36
11.5. Fase 4: Actualización de las clases de operaciones	37
12. Fase 5: Cambiar la superclase <code>GTablePotential</code> (realizada)	38
12.1. Ventajas del cambio de superclase	39
12.2. Inconvenientes y motivos del aplazamiento	39
12.3. Condiciones para retomar la fase 5	40
12.4. Preparación de tests para la fase 5 (realizada)	40
12.5. Implementación de la fase 5	42
13. Verificación y resultados	45
13.1. Tests automáticos	45
13.2. Impacto pendiente en otros módulos	46
14. Resumen de ficheros afectados	46
15. Próximos pasos: Rediseño 3	46
 II Rediseño 3: Análisis de la Opción 3 (Interfaces de Capacidad)	 48
16. Introducción y objetivos	48
17. Estado de partida (tras el Rediseño 1)	48
18. Diseño propuesto: interfaces de capacidad	50
18.1. Tabla de implementación de interfaces por clase	52
19. Análisis de impacto en otros módulos	54
19.1. <code>org.openmarkov.inference</code> (impacto alto)	54
19.2. <code>org.openmarkov.gui</code> (impacto alto)	54
19.3. <code>org.openmarkov.io</code> (impacto medio)	55
19.4. <code>org.openmarkov.learning.*</code> (impacto bajo)	55
19.5. <code>org.openmarkov.costEffectiveness</code> (impacto bajo-medio)	56
20. Problemas identificados	56
20.0.1. Problema P1: Ruptura de la API pública de <code>Potential</code>	56
20.0.2. Problema P2: Retorno de <code>reorder()</code> no tipado	56
20.0.3. Problema P3: <code>GTablePotential</code> como caso especial	56
20.0.4. Problema P4: Dependencia cíclica no resuelta	57

20.0.5. Problema P5: Volumen de cambios y riesgo de regresión	57
21. Valoración y recomendación	57
21.1. Argumentos a favor	57
21.2. Argumentos en contra	57
21.3. Recomendación final	58
III Implementación del Rediseño 3	58
22. Ejecución del plan: cuatro pasos incrementales	58
22.1. Paso 1: Modo <i>opt-in</i> (interfaces sin eliminar métodos abstractos)	58
22.2. Paso 2: Migración del módulo <code>inference</code>	59
22.3. Paso 3: Migración de los módulos <code>gui</code> e <code>io</code>	60
22.4. Paso 4: Hacer concretos los métodos abstractos de <code>Potential</code>	60
23. Divergencias respecto al diseño original	61
23.1. Nombre del método en <code>StrategyCarrier</code>	61
23.2. Cambio de superclase de <code>GTablePotential</code> : aplazada	62
24. Resultados de los tests	62
25. Cambio de superclase de <code>GTablePotential</code>: análisis y decisión	62
25.1. Motivación: por qué cambiar la superclase	62
25.2. Bloqueadores del cambio de superclase	63
25.2.1. Bloqueador B1: Tipo de retorno de <code>DANInference.getUtility()</code>	63
25.2.2. Bloqueador B2: Cast en <code>DANCEAnalysis</code>	63
25.2.3. Bloqueador B3: Listas <code>List<TablePotential></code> en <code>TablePotentialMerge</code>	63
25.3. Opción A: generalizar los tipos a <code>Potential</code>	64
25.4. Opción B: nueva interfaz <code>CEUtilityPotential</code>	65
25.5. Comparación entre opciones	67
25.6. Decisión: Opción B	67
26. Encapsulación del campo <code>values</code> en <code>TablePotential</code>	68
26.1. Situación anterior	68
26.2. Problemas del diseño anterior	68
26.3. La encapsulación: <code>private + getValues() + setValues()</code>	68
26.4. Por qué <code>setValues()</code> no valida el tamaño	69
26.5. Migración y efectos secundarios	70
26.6. Resultado	70
IV Diseño ideal desde cero	72
27. Motivación	72
28. Requisitos del sistema de potenciales	72
28.1. Requisitos funcionales	72
28.2. Requisitos no funcionales	73
29. Arquitectura propuesta: tres capas	73
29.1. Capa 1: Representación	74
29.2. Capa 2: Proyección	75
29.3. Capa 3: Computación	76

30. Rol como comportamiento: PotentialSemantics	77
31. Metadatos por composición	78
32. Eliminación de los <i>downcasts</i>	79
33. Resolución de la dependencia cíclica	80
34. GenericTable<E> : posición limpia en la jerarquía	80
35. Evaluación contra los problemas originales	81
36. Evaluación contra los requisitos	81
37. Impacto en rendimiento	82
38. Distancia al diseño actual tras los rediseños 1 y 3	82
38.1. Hoja de ruta sugerida	82

Resumen

Las clases del paquete `potential` de `org.openmarkov.core` presentan problemas de diseño estructurales acumulados a lo largo de varios años de desarrollo. El más grave es que `TablePotential` —la clase de la que depende prácticamente todo el motor de inferencia— mezcla tres abstracciones distintas en una sola clase mediante campos opcionales que pueden ser `null`. A esto se añaden métodos abstractos que la mayor parte de las subclases no pueden implementar, dependencias cíclicas, y un uso extensivo de *downcasts* en el módulo de inferencia que rompe el polimorfismo y produce fallos en tiempo de ejecución.

Este documento describía los problemas con detalle, propone cinco opciones de rediseño de distinto alcance e impacto, y evalúa cada una en términos de coste de implementación, riesgo, ventajas de diseño e impacto en el rendimiento (tiempo de cómputo y memoria).

1. Catálogo de problemas de diseño

Esta sección proporciona una visión panorámica de los seis problemas de diseño identificados en la jerarquía de `Potential`. Cada problema se describe de forma sintética aquí; las secciones 3.2–3.7 desarrollan el diagnóstico en profundidad.

La figura 1 muestra la jerarquía actual con los seis problemas localizados sobre las clases afectadas.

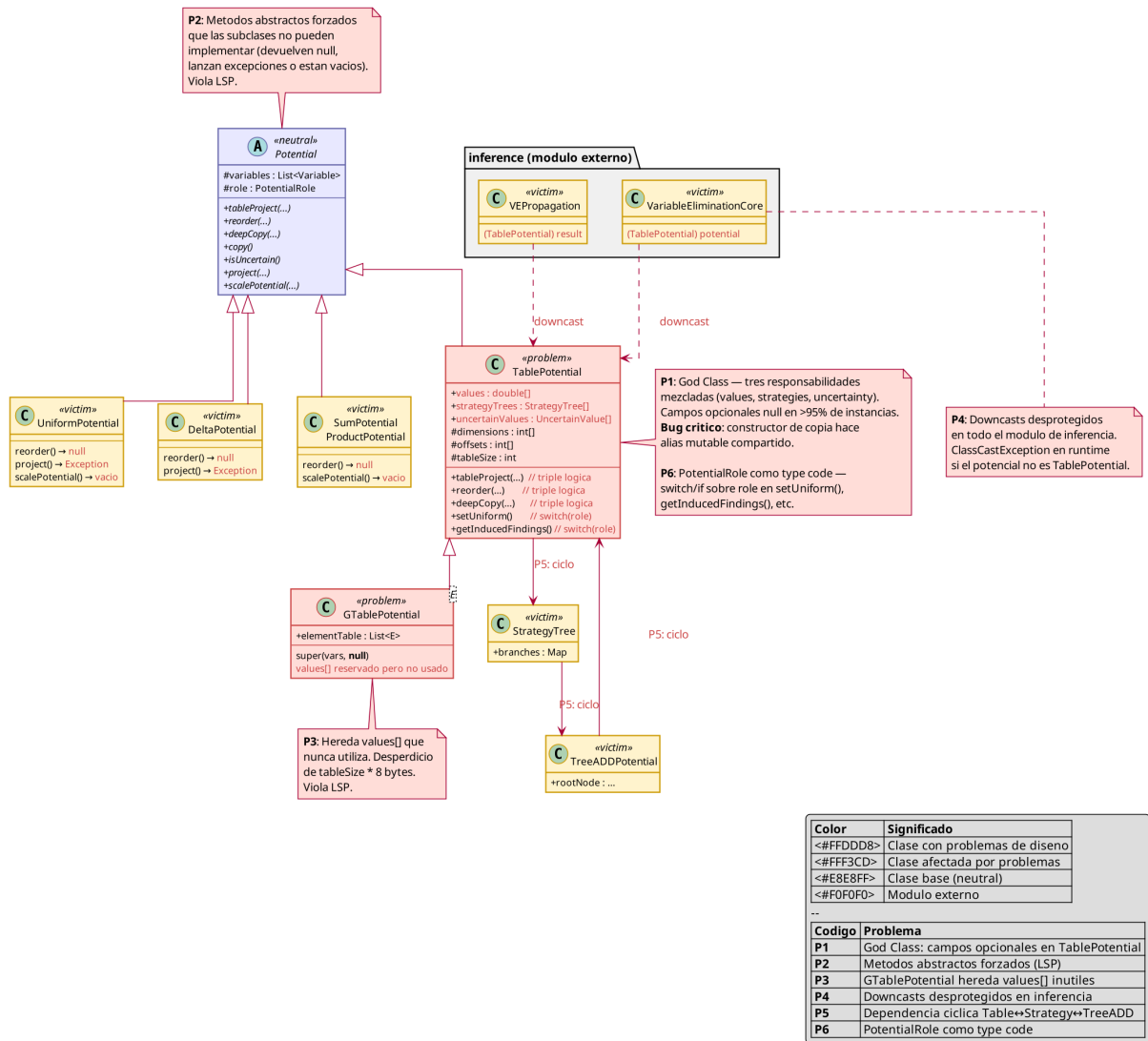


Figura 1: Vista panorámica de los seis problemas de diseño en la jerarquía de **Potential**. Los colores indican la gravedad: rojo para clases con problemas directos, amarillo para clases afectadas.

Id	Problema	Descripción	Clase(s) afectada(s)	Principio
P1	<i>God Class</i> con campos opcionales	<code>TablePotential</code> mezcla valores, estrategias de intervención e incertidumbre en una sola clase mediante campos que son <code>null</code> en >95 % de las instancias. El constructor de copia produce <i>alias mutables</i> compartidos.	<code>TablePotential</code> (1.413 líneas)	SRP
P2	Métodos abstractos forzados	<code>Potential</code> declara métodos abstractos (<code>reorder</code> , <code>project</code> , <code>scalePotential</code>) que muchas subclases implementan devolviendo <code>null</code> , lanzando excepciones o dejando el cuerpo vacío.	<code>UniformPotential</code> , <code>DeltaPotential</code> , <code>SumPotential</code> , <code>ProductPotential</code> , etc.	LSP, ISP
P3	<code>GTablePotential</code> hereda estado inútil	Extiende <code>TablePotential</code> pero nunca usa el array <code>values[]</code> heredado (desperdicio de <code>tableSize * 8</code> bytes). Pasa <code>null</code> como <code>PotentialRole</code> al constructor del padre.	<code>GTablePotential<E></code>	LSP
P4	<i>Downcasts</i> desprotegidos en inferencia	El módulo de inferencia trabaja directamente con <code>TablePotential</code> , no con <code>Potential</code> polimórficamente. Los <i>downcasts</i> desprotegidos (<code>(TablePotential) potential</code>) producen <code>ClassCastException</code> en tiempo de ejecución.	<code>VariableEliminationCore</code> , <code>VEPropagation</code> , etc.	LSP, OCP
P5	Dependencia cíclica	<code>TablePotential</code> $\rightarrow$ <code>StrategyTree</code> $\rightarrow$ <code>TreeADDPotential</code> $\rightarrow$ <code>TablePotential</code> . Hace imposible razonar sobre cualquiera de las tres clases de forma aislada y bloquea la refactorización incremental.	<code>TablePotential</code> , <code>StrategyTree</code> , <code>TreeADDPotential</code>	Acoplamiento
P6	<code>PotentialRole</code> como <i>type code</i>	El enumerado <code>PotentialRole</code> controla el comportamiento de <code>TablePotential</code> mediante sentencias <code>switch</code> : <code>setUniform()</code> , <code>normalize()</code> , <code>getInducedFindings()</code> . Añadir un nuevo rol exige rastrear y actualizar todos los <code>switch</code> .	<code>TablePotential</code> , <code>DiscretePotentialOperations</code>	OCP

Tabla 1: Catálogo de los seis problemas de diseño identificados. Los principios violados se refieren a los principios SOLID descritos en la sección 2.

1.1. Interdependencias entre problemas

Los problemas no son independientes. Resolverlos de forma aislada puede ser insuficiente o incluso contraproducente:

- P1 (God Class) y P6 (type code) están entrelazados: los `switch` sobre `PotentialRole` son parte de la lógica condicional que `TablePotential` acumula. Resolver P1 sin abordar P6 simplifica los campos pero mantiene la dispersión de comportamiento.
- P2 (métodos forzados) es consecuencia directa de P4 (downcasts): el módulo de inferencia necesita que todos los potenciales tengan los mismos métodos porque no trabaja polimórficamente. Resolver P2 sin abordar P4 es superficial: las subclases dejarían de tener métodos

vacíos, pero la inferencia seguiría haciendo *downcasts*.

- P3 (GTablePotential) es un efecto colateral de P1: si TablePotential no hubiera acumulado tantas responsabilidades, GTablePotential nunca habría necesitado heredar de ella para acceder a la lógica de indexación.
- P5 (dependencia cíclica) está ligado a P1: el ciclo existe porque strategyTrees[] vive dentro de TablePotential. Extraer las estrategias a una subclase o a un contenedor externo rompe el ciclo.

Estas interdependencias explican por qué el documento propone opciones de rediseño de distinto alcance (sección 4.1 a 4.5) y por qué se recomienda un enfoque incremental que aborde primero los problemas más aislados (P1, P3, P5) antes de los más transversales (P2, P4, P6).

2. Fundamentos teóricos

Antes de analizar los problemas concretos del código es conveniente definir los conceptos de ingeniería del software que se emplean a lo largo del documento. Esta sección introduce los *principios de diseño* que guían las decisiones de refactorización, los *antipatrones* que describen los problemas encontrados, y los *patrones de diseño* aplicados en las soluciones.

2.1. Principios de diseño orientado a objetos

Los principios de diseño son reglas generales que, cuando se aplican con criterio, producen código más fácil de entender, modificar y probar. No son recetas infalibles, sino guías que ayudan a tomar decisiones de diseño razonadas. A continuación se describen los cuatro principios centrales de este documento.

Principio de Responsabilidad Única (SRP) El *Single Responsibility Principle* (Principio de Responsabilidad Única) establece que una clase debe tener una sola razón para cambiar [1]. Intuitivamente: si un módulo hace demasiadas cosas a la vez, cualquier cambio en una de ellas obliga a revisar y probar todas las demás. Imagina un empleado encargado a la vez de contabilidad, recursos humanos y soporte técnico: una auditoría contable, un cambio en las políticas de vacaciones y un fallo en el servidor afectan al mismo empleado, aunque no tengan nada en común. Cuando una clase viola el SRP, sus métodos se entrelazan y las modificaciones en una parte producen efectos secundarios inesperados en otra.

Principio de Sustitución de Liskov (LSP) El *Liskov Substitution Principle* (Principio de Sustitución de Liskov) establece que si una clase *B* extiende una clase *A*, entonces cualquier fragmento de código que funcione correctamente con objetos de tipo *A* debe seguir funcionando correctamente si se le pasa un objeto de tipo *B* [2]. En términos cotidianos: si en una receta de cocina se dice “usa un pájaro”, un pato debería servir. Pero si la receta dice “haz volar el pájaro”, un pingüino —que es un pájaro— no sirve, aunque la herencia biológica lo incluya. Cuando una subclase no puede cumplir el contrato del padre (porque devuelve `null` o lanza una excepción donde se espera un resultado válido), el código que usa esa subclase se vuelve frágil e impredecible.

Principio de Segregación de Interfaces (ISP) El *Interface Segregation Principle* (Principio de Segregación de Interfaces) establece que una clase no debe ser obligada a depender de métodos que no utiliza. Cuando se definen interfaces demasiado genéricas, las clases que las implementan acaban llenando de métodos vacíos o que lanzan excepciones. Es preferible tener muchas interfaces pequeñas y específicas que una sola interfaz grande: así cada clase implementa exactamente las capacidades que tiene, sin fingir que soporta operaciones que no puede realizar. Una biblioteca de

una ciudad debería ofrecer servicios de préstamo, consulta y devolución; no debería verse obligada a ofrecer también reparación de libros o encuadernación solo porque esos servicios forman parte de un hipotético “contrato de biblioteca completa”.

Principio Abierto/Cerrado (OCP) El *Open/Closed Principle* (Principio Abierto/Cerrado) establece que una clase debe estar *abierta* para extensión —se pueden añadir nuevos comportamientos— pero *cerrada* para modificación —no es necesario cambiar el código existente para añadir esos comportamientos—. Esto se logra normalmente mediante herencia o interfaces: en lugar de añadir un nuevo `case` a un `switch` existente cada vez que aparece un nuevo tipo, se crea una nueva subclase o implementación de interfaz. El código original no necesita tocarse; el nuevo comportamiento se encapsula en el nuevo tipo. La ventaja es que el código que ya funciona y ha sido probado permanece intacto, reduciendo el riesgo de introducir errores al añadir funcionalidad.

2.2. Antipatrones encontrados

Un *antipatrón* es un patrón recurrente de diseño que en apariencia parece razonable pero que en la práctica genera problemas de mantenimiento, fragilidad o dificultad de comprensión. Reconocer los antipatrones en el código propio es el primer paso para corregirlos.

God Class (Clase Dios) **Descripción:** Una clase que acumula demasiadas responsabilidades: lo sabe todo y lo hace todo. En lugar de delegar en otras clases, concentra lógica de distintos dominios en una sola unidad.

Por qué es dañina: Viola directamente el SRP. Cualquier cambio en cualquiera de sus responsabilidades requiere entender la clase entera, porque los distintos dominios están entrelazados en los mismos métodos y los mismos campos. Las pruebas unitarias son difíciles de escribir porque hay demasiadas combinaciones posibles de estado. La God Class se convierte en el cuello de botella del desarrollo: todo pasa por ella.

Solución habitual: Identificar las distintas responsabilidades que la clase acumula y extraer cada una a su propia clase. Esto es exactamente lo que la refactorización “Extraer Clase” (*Extract Class*) persigue.

Type Code (Código de Tipo) **Descripción:** Usar un campo numérico o enumerado para hacer que una clase se comporte de modo diferente según su valor, en lugar de usar subclases con comportamiento propio. El resultado es una clase llena de sentencias `switch` o bloques `if-else` que inspeccionan el campo para decidir qué hacer.

Por qué es dañino: Añadir un nuevo “tipo” exige rastrear y modificar *todos* los `switch/if` que miran ese campo, dispersos por la clase o incluso por el proyecto. El compilador no avisa de los casos olvidados. Cada vez que se añade un valor al enumerado, hay que analizar manualmente qué bloques de código necesitan actualizarse.

Solución habitual: Reemplazar el campo de tipo por una jerarquía de subclases (cada subclase representa uno de los tipos) cuyo comportamiento diferenciado se expresa mediante métodos sobrescritos. Los patrones de diseño State y Strategy son formalizaciones de esta solución.

Refused Bequest (Legado Rechazado) **Descripción:** Una subclase hereda métodos del padre que no puede o no quiere implementar, y los deja vacíos o lanzando excepciones. La subclase “rechaza la herencia” que le corresponde.

Por qué es dañino: Viola el LSP: el código que invoca un método sobre la clase padre puede recibir `null` o una excepción donde espera un resultado válido. Esto obliga a insertar compro-

baciones defensivas en todo el código que llama a esos métodos, ensuciando el código cliente y desplazando la responsabilidad de gestionar casos incorrectos fuera de donde debería estar.

Solución habitual: Si una subclase no puede cumplir el contrato del padre, la relación de herencia es incorrecta. La solución es retirar la herencia y usar composición o interfaces que describan solo las capacidades que la clase verdaderamente posee.

Aliasing Bug (Error de Alias) Descripción: Dos variables apuntan al mismo objeto mutable en memoria sin que el programador sea consciente de ello. Modificar el objeto a través de una variable afecta silenciosamente a la otra, produciendo efectos secundarios no intencionados.

Por qué es dañino: Produce corrupción de datos difícil de reproducir y depurar, especialmente en algoritmos que copian y modifican estructuras de datos. El fallo se manifiesta lejos del punto donde se introduce el alias, lo que hace muy difícil encontrar la causa raíz con un depurador.

Solución habitual: Usar *copias defensivas* (*defensive copy*) al crear o retornar objetos mutables: en lugar de asignar la referencia al mismo objeto, se crea una nueva instancia con el mismo contenido. De este modo, modificar la copia no afecta al original.

2.3. Patrones de diseño aplicados

Un *patrón de diseño* es una solución reutilizable a un problema frecuente de diseño orientado a objetos [3]. A diferencia de los antipatrones, los patrones de diseño son soluciones contrastadas. Este documento aplica dos patrones.

Método Plantilla (Template Method) Descripción: La clase base define el esqueleto de un algoritmo llamando a métodos que las subclases pueden sobrescribir. La clase base controla la estructura y el orden de los pasos; las subclases aportan los detalles específicos de cada paso. Así, el algoritmo general se escribe una sola vez en el padre, y solo las partes que varían se delegan a los hijos.

Aplicación en este documento: El método `isUncertain()` en la clase base `TablePotential` funciona correctamente para toda la jerarquía porque llama a `getUncertainValues()`, que es un método virtual. Las subclases sobrescriben `getUncertainValues()` para devolver su array propio; la clase base no necesita saber qué subclase concreta está en uso.

Interfaz Marcadora (Marker Interface) Descripción: Una interfaz sin métodos que actúa como etiqueta de tipo. Su presencia en la declaración de una clase comunica al compilador —y al lector del código— que esa clase pertenece a una categoría específica. El compilador puede entonces verificar que solo los tipos correctos se usan en ciertos contextos.

Aplicación en este documento: La interfaz `CEUtilityPotential` marca los potenciales válidos como utilidad en el análisis coste-efectividad. Solo `TablePotential` y `GTablePotential` implementan esta interfaz, garantizando en tiempo de compilación que no lleguen tipos inadecuados a los algoritmos de fusión del módulo de coste-efectividad.

3. Situación anterior: diagnóstico detallado

3.1. Jerarquía de clases

La jerarquía de `Potential` contenía más de treinta subclases que cubren desde tablas de probabilidad discretas hasta distribuciones gaussianas, potenciales de función simbólica, árboles de

decisión y potenciales de incertidumbre de modelo. La clase base abstracta declara ocho métodos abstractos que todos sus descendientes debían implementar, independientemente de si la operación tiene sentido para ese tipo de potencial.

Las clases más relevantes para la inferencia son:

- **TablePotential**: tabla de probabilidades o utilidades sobre variables discretas. Es la clase de trabajo de casi todos los algoritmos.
- **GTablePotential<E>**: tabla genérica que almacena objetos en lugar de `double`; usada en el análisis coste-efectividad para almacenar objetos **Choice**.
- **UniformPotential**, **DeltaPotential**: representaciones compactas de distribuciones uniformes y deterministas.
- **SumPotential**, **ProductPotential**: nodos supervalor para diagramas de influencia con múltiples funciones de utilidad.
- **TreeADDPotential**, **StrategyTree**: árboles de decisión algebraicos y árboles de estrategia de intervención.

3.2. Problema 1: TablePotential con tres responsabilidades

TablePotential tiene 1.413 líneas de código. Su responsabilidad declarada es almacenar una tabla de valores `double` indexada por las configuraciones de sus variables (antipatrón *God Class*, véase la sección 2.2). Sin embargo, contiene además dos responsabilidades adicionales mediante campos opcionales:

```

1 public volatile double[]      values;          // probabilidades/utilidades
2 public volatile StrategyTree[] strategyTrees;  // OPCIONAL: intervenciones
3 public volatile UncertainValue[] uncertainValues; // OPCIONAL: incertidumbre

```

Listado 1: Campos principales de **TablePotential**

El campo `strategyTrees` almacena, para cada configuración de la tabla, un árbol de decisión que codifica las acciones óptimas a ejecutar en las variables de decisión del problema. Está presente únicamente en potenciales de utilidad dentro de la resolución de diagramas de influencia. El campo `uncertainValues` almacena distribuciones de incertidumbre sobre cada valor de la tabla para el análisis de sensibilidad paramétrica. Ambos campos son `null` en la inmensa mayoría de los potenciales del sistema.

La consecuencia directa es que todos los métodos relevantes de la clase tienen que manejar tres casos a la vez:

```

1 boolean hasUncertainTable = (uncertainValues != null);
2 if (hasUncertainTable) {
3     projectedPotential.setUncertainValues(new UncertainValue[length]);
4 }
5 // ...
6 projectedPotential.copyValuesInterventionsAndUncertainValues(
7     projectedPosition, this, firstPosition, hasUncertainTable);
8 // ...
9 if (strategyTrees != null) {
10     resultPotential.strategyTrees = new StrategyTree[resultPotential.values.length
11         ↪ ];
12     for (int i = 0; i < resultPotential.strategyTrees.length; i++) {
13         resultPotential.strategyTrees[i] = ...;
14     }
15 }

```

Listado 2: Fragmento de `tableProject()` con condicionales por campos opcionales

Este patrón se repite en `reorder()`, `deepCopy()`, `copy()` y en los métodos de fusión de `TablePotentialMerge`. La complejidad efectiva de cada operación sobre potenciales es el triple de la que debería ser.

Además, el constructor de copia no realiza una copia profunda de los arrays opcionales:

```
1 // Constructor de copia de TablePotential -- los arrays opcionales NO se clonan
2 uncertainValues = potential.uncertainValues; // alias mutable compartido
3 strategyTrees   = potential.strategyTrees;   // alias mutable compartido
```

Listado 3: Error de copia superficial en el constructor de copia

Modificar `uncertainValues` o `strategyTrees` en la copia modifica también el original, lo que puede causar corrupción silenciosa de datos durante la inferencia (antipatrón *Aliasing Bug*, sección 2.2).

3.3. Problema 2: métodos abstractos que las subclases no pueden implementar

La clase abstracta `Potential` declara métodos que muchas subclases son incapaces de implementar significativamente. El resultado es una acumulación de implementaciones déficit que violan el Principio de Sustitución de Liskov (LSP, sección 2.1; antipatrón *Refused Bequest*, sección 2.2):

```
1 // UniformPotential -- reorder no implementado
2 @Override
3 public Potential reorder(List<Variable> newOrderOfVariables) {
4     return null; // TODO Auto-generated method stub
5 }
6
7 // UniformPotential -- project no soportado
8 @Override
9 public Potential project(EvidenceCase evidenceCase) {
10    throw new UnsupportedOperationException();
11 }
12
13 // UniformPotential -- scalePotential irrelevante (no tiene valores)
14 @Override
15 public void scalePotential(double scale) {
16    // vacío: no hay nada que escalar
17 }
```

Listado 4: Implementaciones forzadas que no tienen sentido para la subclase

El mismo patrón aparece en `DeltaPotential`, `SumPotential`, `ProductPotential` y otras subclases. Desde el punto de vista del llamador, invocar `reorder()` sobre cualquier `Potential` puede retornar `null`, lo que obliga a insertar comprobaciones defensivas en todo el código que llama a este método.

La tabla 2 resume la situación por método y subclase.

Método	TablePotential	Uniform	Delta	Sum/Product	ExactDistr
tableProject()	Complejo	Crea tabla	Crea tabla	Delega	Delega
project()	Delega	Excepción	Excepción	Excepción	Delega
reorder(List)	50 líneas	null	null	null	Delega
scalePotential()	Multiplica	Vacío	Multiplica	Vacío	Delega
isUncertain()	Consulta campo	false	false	false	Delega
copy()	Copia profunda	Copia	Copia	Copia	Copia

Tabla 2: Implementación de métodos abstractos por subclase. **Rojo**: implementación incorrecta o ausente. **Naranja**: trivial/forzada.

3.4. Problema 3: GTablePotential<E> viola el contrato de su padre

GTablePotential<E> extiende TablePotential para almacenar objetos genéricos (objetos Choice en el análisis coste-efectividad) en lugar de valores double. Sin embargo, esta relación de herencia es incorrecta:

```

1 public GTablePotential(List<Variable> variables, PotentialRole role) {
2     super(variables, null); // pasa null como role -- invalida setUniform()
3     // ...
4     elementTable = new ArrayList<E>(sizeTable);
5 }

```

Listado 5: Constructor de GTablePotential que viola el contrato del padre

Al pasar null como role, el constructor del padre no puede inicializar values correctamente. El array values heredado se reserva y se rellena con ceros pero nunca se usa: GTablePotential almacena sus datos en List<E>elementTable. Esto supone un desperdicio de memoria (un array de double completamente inútil) y hace que todos los métodos heredados que acceden a values devuelvan resultados sin sentido.

3.5. Problema 4: downcasts en inferencia

El módulo org.openmarkov.inference trabaja en la práctica con TablePotential directamente, no con Potential polimórficamente. El resultado es un código sembrado de downcasts:

```

1 // VariableEliminationCore.java
2 for (Potential potential : probPotentials) {
3     probTP.add((TablePotential) potential); // sin instanceof
4 }
5
6 // VEPropagation.java
7 posteriorValue = (TablePotential) posteriorValue.reorder(orderedVariables);
8
9 // VariableElimination.java
10 TablePotential utility = new TablePotential(null, PotentialRole.UNSPECIFIED);

```

Listado 6: Downcasts desprotegidos en el motor de inferencia

Si en cualquier momento un potencial no es un TablePotential (porque se usa una red con UniformPotential, por ejemplo, y tableProject() no se llamó antes), el sistema lanza ClassCastException en tiempo de ejecución, lejos del origen del problema y difícil de depurar.

3.6. Problema 5: dependencia cíclica entre TablePotential y StrategyTree

El diseño actual introduce una dependencia cíclica:

TablePotential $\rightarrow$ StrategyTree[] StrategyTree $\rightarrow$ TreeADDPotential
 TreeADDPotential $\rightarrow$ TablePotential

Este ciclo hace imposible razonar sobre cualquiera de las tres clases de forma aislada, dificulta las pruebas unitarias y bloquea cualquier intento de refactorización incremental de la jerarquía.

3.7. Problema 6: PotentialRole como sustituto de tipos

El enumerado PotentialRole (valores: CONDITIONAL_PROBABILITY, JOINT_PROBABILITY, POLICY, UTILITY, LINK_RESTRICTION, UNSPECIFIED) controla el comportamiento de TablePotential en varios métodos mediante sentencias switch y condicionales:

```

1  switch (role) {
2      case CONDITIONAL_PROBABILITY:
3      case POLICY:
4          value = 1.0 / variables.get(0).getNumStates();
5          break;
6      case JOINT_PROBABILITY:
7          value = 1.0 / computeTableSize(variables);
8          break;
9      case LINK_RESTRICTION:
10         value = 1.0;
11         break;
12     default: // UTILITY, UNSPECIFIED
13         value = 0.0;
14 }

```

Listado 7: PotentialRole controlando comportamiento en setUniform()

Esto es el antipatrón *type code* (sección 2.2): en lugar de subclases con comportamiento propio, una sola clase se comporta de modo diferente según un campo enum. Añadir un nuevo rol exige rastrear y actualizar todos los switch existentes en la clase.

4. Opciones de rediseño

4.1. Opción 1 — Extracción mínima de responsabilidades

Descripción

Esta opción no modifica la jerarquía de Potential ni las firmas de los métodos abstractos. Su único objetivo es eliminar los dos campos opcionales de TablePotential descritos en la sección 3.2.

Se crean dos subclases especializadas:

```

1  // Clase base sin campos opcionales
2  class TablePotential extends Potential {
3      public volatile double[] values;    // unico campo de datos
4      // sin strategyTrees
5      // sin uncertainValues
6  }
7
8  // Subclase para potenciales con intervenciones
9  final class StrategicTablePotential extends TablePotential {
10     public StrategyTree[] strategyTrees;
11
12     @Override public boolean hasInterventions() { return true; }
13     @Override public Potential deepCopy(ProbNet net) { ... }
14     @Override public TablePotential tableProject(...) { ... }
15 }

```

```

16
17 // Subclase para potenciales con incertidumbre de modelo
18 final class UncertainTablePotential extends TablePotential {
19     public UncertainValue[] uncertainValues;
20
21     @Override public boolean isUncertain() { return true; }
22     @Override public Potential deepCopy(ProbNet net) { ... }
23     @Override public TablePotential tableProject(...) { ... }
24 }

```

Listado 8: Jerarquía resultante de la Opción 1

GTablePotential<E> deja de extender TablePotential y pasa a extender directamente Potential, eliminando el array values inútil y el null pasado como role.

Los métodos de TablePotentialMerge, TablePotentialMaximization y TablePotentialElimination que actualmente comprueban if (strategyTrees != null) o if (uncertainValues != null) se simplifican mediante instanceof o sobrecarga.

Lo que no cambia

- Los 28 llamadores de DiscretePotentialOperations no se tocan.
- Los métodos abstractos de Potential permanecen iguales.
- Los *downcasts* en inferencia siguen igual (la inferencia trabaja con TablePotential, que sigue siendo la clase base).
- El problema del PotentialRole como type code no se aborda.

Ventajas

- TablePotential pierde ≈ 300 líneas de manejo condicional de campos opcionales.
- Los métodos reorder(), deepCopy() y tableProject() de TablePotential se simplifican notablemente.
- Se elimina el error de copia superficial del constructor de copia: al no haber campos opcionales en TablePotential, no hay alias mutables.
- El bug de GTablePotential (array values inútil) queda resuelto.
- Compatible con los tests actuales sin modificación.

Desventajas

- No resuelve los métodos abstractos forzados (sección 3.3).
- No elimina los *downcasts* en inferencia (sección 3.5).
- No rompe la dependencia cíclica (sección 3.6).
- No aborda el antipatrón PotentialRole (sección 3.7).

Impacto en rendimiento

Tiempo: Ligera mejora en todos los métodos de TablePotential al eliminar las comprobaciones null en cada iteración del bucle de offsets acumulados. El impacto es pequeño pero consistente: los bucles críticos de tableProject() y reorder() eliminan 2–3 ramas condicionales por iteración.

Memoria: Reducción moderada. Los potenciales ordinarios (`TablePotential`) dejan de reservar espacio para dos punteros a arrays opcionales que en la gran mayoría de los casos son `null`. En el caso patológico de una red con miles de potenciales, esto puede significar varios cientos de kilobytes menos en el *heap*. `GTablePotential` deja de reservar el array `values` inútil: ahorro de $\text{tableSize} * 8$ bytes por instancia.

Esfuerzo estimado

2-3 semanas. La mayor parte del trabajo está en extraer `StrategicTablePotential` y `UncertainTablePotential` y adaptar `TablePotentialMerge` y `TablePotentialMaximization`.

4.2. Opción 2 — Jerarquía basada en roles

Descripción

Esta opción convierte el antipatrón `PotentialRole` en una jerarquía de tipos real. Cada rol que implica comportamiento distinto se convierte en una subclase de `DiscretePotential`, que a su vez extiende `Potential`. La figura 2 muestra la estructura resultante.

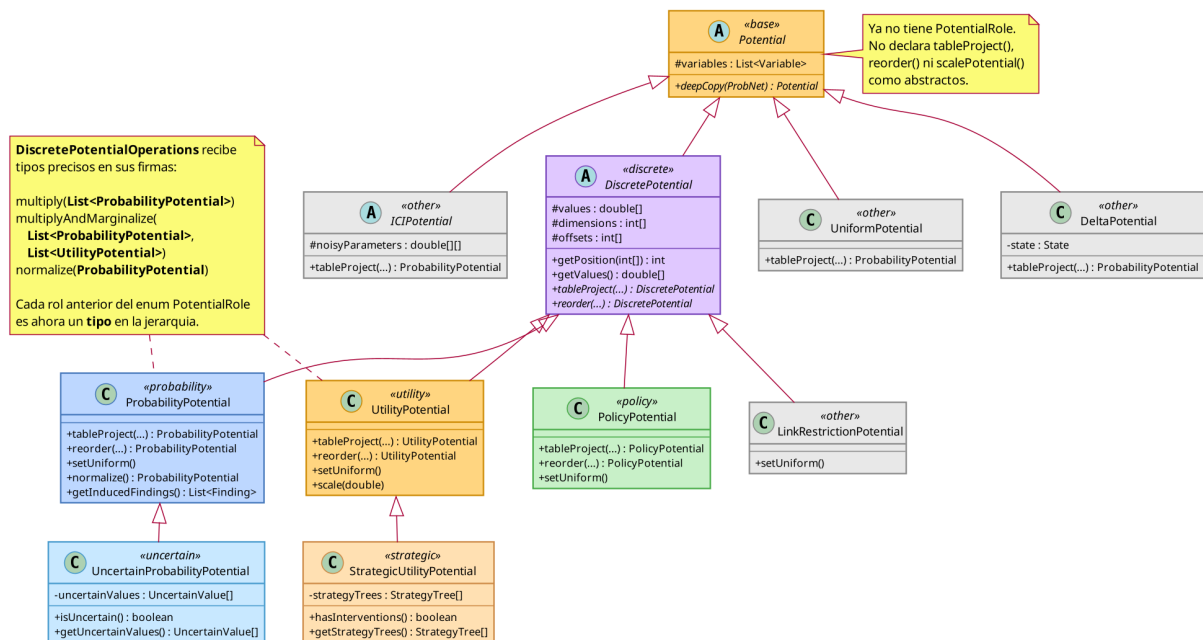


Figura 2: Opción 2: Jerarquía basada en roles. Cada valor del antiguo enum `PotentialRole` se convierte en una subclase de `DiscretePotential` con su propio comportamiento (`setUniform()`, `normalize()`, etc.). Las operaciones de `DiscretePotentialOperations` reciben tipos precisos.

La jerarquía resultante es:

```

1  abstract class Potential { ... }
2
3  abstract class DiscretePotential extends Potential {
4      protected final double[] values;
5      protected final int[] dimensions;
6      protected final int[] offsets;
7
8      // Metodos comunes a todos los potenciales discretos
9      public int getPosition(int[] coords) { ... }
10     public double[] getValues() { ... }
11 }

```



```

12
13 final class ProbabilityPotential extends DiscretePotential {
14     // role = CONDITIONAL_PROBABILITY o JOINT_PROBABILITY
15     // Sin estrategias, sin incertidumbre
16     @Override public ProbabilityPotential tableProject(...) { ... }
17     @Override public ProbabilityPotential reorder(...) { ... }
18 }
19
20 final class UtilityPotential extends DiscretePotential {
21     // role = UTILITY
22     // Puede tener StrategyTree[]
23     private final StrategyTree[] strategyTrees; // no opcional: si existe es
24         ↪ StrategicUtility
25     @Override public UtilityPotential tableProject(...) { ... }
26 }
27 final class PolicyPotential extends DiscretePotential {
28     // role = POLICY
29     @Override public PolicyPotential tableProject(...) { ... }
30 }
31
32 final class LinkRestrictionPotential extends DiscretePotential {
33     // role = LINK_RESTRICTION
34 }

```

Listado 9: Jerarquía resultante de la Opción 2

Los métodos de operación en `DiscretePotentialOperations` y sus clases delegadas reciben firmas más precisas. Por ejemplo, `multiplyAndMarginalize` acepta `List<ProbabilityPotential>` en lugar de `Collection<TablePotential>`.

El análisis de sensibilidad (`UncertainValue`) se modela como un decorador:

```

1 final class UncertainProbabilityPotential extends ProbabilityPotential {
2     private final UncertainValue[] uncertainValues;
3
4     @Override public boolean isUncertain() { return true; }
5     // deepCopy() correcto: copia uncertainValues
6 }

```

Listado 10: Decorador para incertidumbre de modelo

Ventajas

- Elimina completamente el antipatrón `PotentialRole` como controlador de comportamiento: cada clase tiene su propio `setUniform()`, `getInducedFindings()`, etc.
- Añadir un nuevo tipo de potencial discreto (p. ej. `RewardPotential` para MDPs) es añadir una subclase sin tocar las existentes.
- Las firmas de las operaciones de inferencia son más precisas y los *downcasts* desaparecen en la mayoría de los puntos de llamada.
- La dependencia cíclica `TablePotential` ↔ `StrategyTree` queda resuelta: `StrategyTree` se asocia solo con `UtilityPotential`.

Desventajas

- La frontera entre `ProbabilityPotential` y `UtilityPotential` es difusa en ciertos algoritmos: `multiplyAndMarginalize` multiplica una probabilidad por una utilidad y produce un resultado híbrido cuyo tipo no encaja limpiamente en ninguna de las dos subclases.

- Requiere actualizar las firmas de métodos en los llamadores del módulo `inference`, lo que puede propagar cambios a todos los algoritmos.
- Los métodos abstractos forzados (sección 3.3) no se resuelven en las subclases de `Potential` que no son discretas (`UniformPotential`, `DeltaPotential`, etc.).

Impacto en rendimiento

Tiempo: Neutro o ligera mejora. El despacho de métodos virtuales no cambia en profundidad respecto a la situación actual. La eliminación de los `switch` sobre `PotentialRole` en métodos llamados frecuentemente (`setUniform()`, `getInducedFindings()`) supone un ahorro de unos pocos nanosegundos por llamada, irrelevante en la mayoría de los casos.

Memoria: Reducción moderada. La eliminación de campos opcionales tiene el mismo efecto que en la Opción 1. Las subclases adicionales en la jerarquía no implican coste de memoria por instancia: solo el coste fijo de la clase en el *metaspace* de la JVM, que es despreciable.

Esfuerzo estimado

2–3 meses. El reto mayor es la propagación de cambios al módulo `inference` y a los formatos de E/S.

4.3. Opción 3 — Composición mediante interfaces

Descripción

En lugar de usar herencia para modelar capacidades, esta opción define *interfaces de capacidad*¹ que las clases implementan según lo que realmente soportan:

```

1 /** Cualquier potencial que puede proyectarse a una tabla discreta. */
2 interface Projectable {
3     TablePotential tableProject(EvidenceCase ev, InferenceOptions opts)
4         throws NonProjectablePotentialException;
5 }
6
7 /** Potencial cuyos valores pueden reordenarse segun un nuevo orden de variables.
8     ↪ */
9 interface Reorderable {
10     Potential reorder(List<Variable> newOrderOfVariables);
11 }
12
13 /** Potencial que puede escalarse por un factor escalar. */
14 interface Scalable {
15     void scale(double factor);
16 }
17
18 /** Potencial con distribucion de incertidumbre sobre sus valores. */
19 interface UncertaintyCarrier {
20     UncertainValue[] getUncertainValues();
21     boolean isUncertain();
22 }
23
24 /** Potencial que lleva arboles de estrategia de intervencion. */
25 interface StrategyCarrier {
26     StrategyTree[] getStrategies();
27     boolean hasInterventions();
28 }

```

¹Una *interfaz de capacidad* es una interfaz Java cuya implementación indica que la clase soporta una operación específica. A diferencia de la herencia, una clase puede declarar exactamente las capacidades que posee sin heredar métodos que no puede implementar. Este enfoque aplica el Principio de Segregación de Interfaces (ISP, sección 2.1).

Listado 11: Interfaces de capacidad

Las clases implementan solo las interfaces que tienen sentido para ellas:

```
1 // Clase base simple -- solo capacidad de proyeccion
2 class TablePotential extends Potential
3     implements Projectable, Reorderable, Scalable { ... }
4
5 // Con incertidumbre -- añade UncertaintyCarrier
6 class UncertainTablePotential extends TablePotential
7     implements UncertaintyCarrier { ... }
8
9 // Con estrategias -- añade StrategyCarrier
10 class StrategicTablePotential extends TablePotential
11     implements StrategyCarrier { ... }
12
13 // Potencial uniforme -- solo proyectable, no reordenable
14 class UniformPotential extends Potential implements Projectable { ... }
15
16 // Potencial delta -- proyectable y escalable
17 class DeltaPotential extends Potential implements Projectable, Scalable { ... }
```

Listado 12: Implementación de interfaces por clase

El código de inferencia declara los tipos que necesita en función de las interfaces:

```
1 // En lugar de: List<TablePotential> probPotentials = ...
2 List<Projectable> projectable = network.getPotentials().stream()
3     .filter(p -> p instanceof Projectable)
4     .map(p -> (Projectable) p)
5     .collect(toList());
6
7 // En lugar de: TablePotential tp = (TablePotential) potential;
8 if (potential instanceof Reorderable reorderable) {
9     Potential reordered = reorderable.reorder(newOrder); // Java 16+
10 }
```

Listado 13: Uso de interfaces en el motor de inferencia

Esta opción también permite combinar capacidades mediante composición:

```
1 class FullCapabilityPotential extends TablePotential
2     implements UncertaintyCarrier, StrategyCarrier {
3     private final UncertainValue[] uncertainValues;
4     private final StrategyTree[] strategyTrees;
5     // ...
6 }
```

Listado 14: Clase que combina múltiples capacidades

Ventajas

- Elimina completamente los métodos abstractos forzados: `UniformPotential` no necesita declarar `reorder()` ni `scalePotential()` porque no implementa `Reorderable` ni `Scalable`.
- Los *downcasts* en inferencia desaparecen: el código pide la capacidad que necesita mediante la interfaz apropiada.
- Es la opción más extensible: añadir un nuevo tipo de potencial equivale a decidir qué interfaces implementa, sin modificar nada más.

- Compatible con Java 21 (*pattern matching* para `instanceof` simplifica el código de consumo).

Desventajas

- Requiere revisar *todo* el módulo `inference` para sustituir los usos directos de `TablePotential` por interfaces.
- El impacto se extiende a los módulos `gui`, `io` y `learning`, que también crean y manipulan potenciales.
- La abstracción de interfaces añade una capa de indirección que algunos desarrolladores pueden encontrar menos legible que la herencia directa.
- Las operaciones de aritmética (`DiscretePotentialOperations`) deben ser revisadas para aceptar interfaces en lugar de clases concretas, lo que puede requerir sobrecargas adicionales.

Impacto en rendimiento

Tiempo: Impacto negativo pequeño. El despacho de métodos de interfaz (*invokeinterface*) es marginalmente más costoso que el despacho de métodos virtuales (*invokevirtual*) en la JVM: típicamente 1–3 ns por llamada en JVMs modernas con JIT. Los bucles críticos de inferencia realizan millones de llamadas a `tableProject()` y a operaciones de tabla; en entornos de producción, este coste puede acumularse hasta un 2–5 % de ralentización en benchmarks de alta precisión, aunque en la práctica la JVM puede eliminarlo mediante devirtualización agresiva.

Memoria: Similar a la Opción 1. La eliminación de campos opcionales en `TablePotential` tiene el mismo efecto positivo. Las interfaces en sí no añaden coste de memoria por instancia.

Esfuerzo estimado

3–4 meses con riesgo de compilación en cascada en módulos dependientes.

4.4. Opción 4 — Patrón Visitor para separar operaciones de datos

Descripción

Esta opción es la más radical: las clases `Potential` se convierten en contenedores de datos puros, y todas las operaciones (proyección, reordenamiento, escalado, copia profunda) se mueven a clases *Visitor* externas. La figura 3 muestra la separación entre el paquete de datos y el paquete de visitantes.

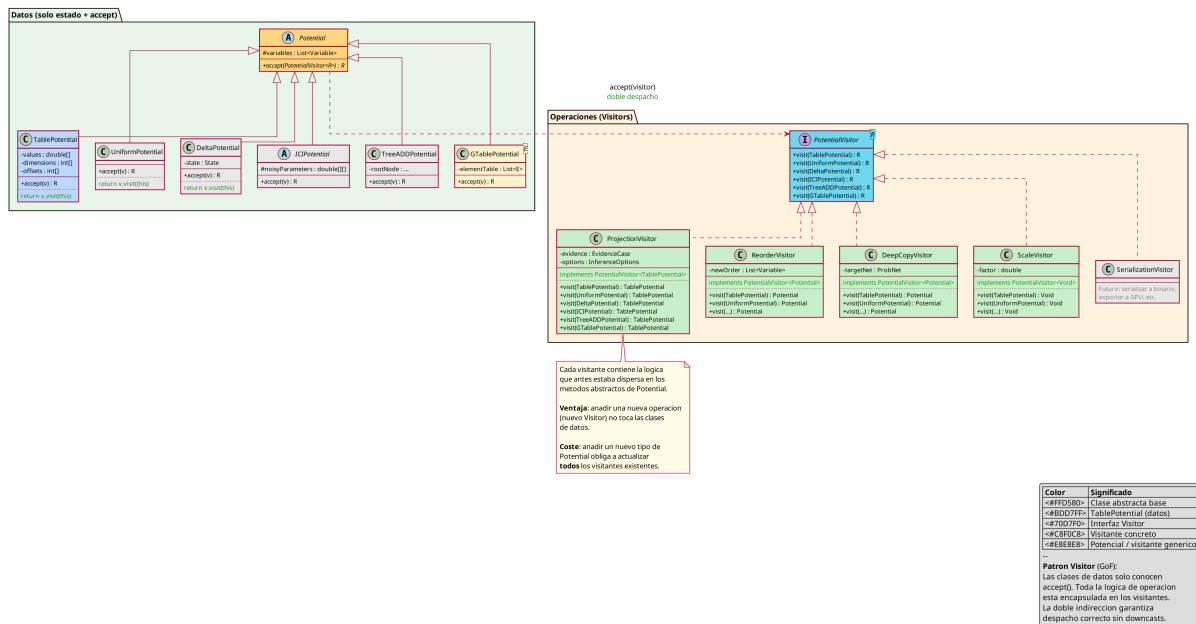


Figura 3: Opción 4: Patrón Visitor. Las clases de datos (izquierda) solo implementan `accept()`. Toda la lógica de operación reside en los visitantes concretos (derecha). La doble indirección `accept`→`visit` garantiza el despacho correcto sin *downcasts*, pero obliga a actualizar todos los visitantes cada vez que se añade un nuevo tipo de potencial.

```

1 // Datos puros -- sin logica de operacion
2 abstract class Potential {
3     protected final List<Variable> variables;
4     abstract <R> R accept(PotentialVisitor<R> visitor);
5 }
6
7 final class TablePotential extends Potential {
8     final double[] values;
9     @Override public <R> R accept(PotentialVisitor<R> v) { return v.visit(this); }
10 }
11
12 final class UniformPotential extends Potential {
13     @Override public <R> R accept(PotentialVisitor<R> v) { return v.visit(this); }
14 }
15
16 // Operaciones -- separadas de los datos
17 interface PotentialVisitor<R> {
18     R visit(TablePotential tp);
19     R visit(UniformPotential up);
20     R visit(DeltaPotential dp);
21     R visit(FunctionPotential fp);
22     // un metodo por tipo concreto de Potential
23 }
24
25 // Ejemplo de visitante concreto
26 class ProjectionVisitor implements PotentialVisitor<TablePotential> {
27     private final EvidenceCase evidence;
28     private final InferenceOptions opts;
29
30     @Override public TablePotential visit(TablePotential tp) {
31         // logica de proyeccion sobre tabla
32     }
33     @Override public TablePotential visit(UniformPotential up) {
34         // proyecta uniforme en una tabla discreta

```

```

35     }
36     @Override public TablePotential visit(DeltaPotential dp) {
37         // proyecta delta en una tabla discreta
38     }
39     // ...
40 }
41
42 // Otros visitantes
43 class ReorderVisitor implements PotentialVisitor<Potential> { ... }
44 class ScaleVisitor implements PotentialVisitor<Void> { ... }
45 class DeepCopyVisitor implements PotentialVisitor<Potential> { ... }

```

Listado 15: Jerarquía de datos y visitantes

El código de llamada pasa de:

```

1 TablePotential projected = potential.tableProject(evidenceCase, options);

```

a:

```

1 TablePotential projected = potential.accept(new ProjectionVisitor(evidenceCase,
    ↪ options));

```

Ventajas

- Separación perfecta entre datos y operaciones: las clases `Potential` no conocen nada de la lógica de operación.
- Añadir una nueva operación (p. ej. serializar a formato binario o evaluar en GPU) equivale a implementar un nuevo `Visitor` sin tocar ninguna clase de datos existente.
- El compilador garantiza que cada `Visitor` maneja todos los tipos de potencial: si se añade un nuevo tipo concreto, todos los visitantes existentes dejan de compilar hasta que se añade el método correspondiente.
- Elimina completamente los métodos abstractos forzados y los *downcasts*.
- La dependencia cíclica desaparece: los visitantes de `StrategyTree` y los de `TablePotential` son independientes.

Desventajas

- Añadir un nuevo tipo de `Potential` exige actualizar *todas* los visitantes existentes. Con una jerarquía de 30+ tipos y varios visitantes, esto puede ser una carga de mantenimiento elevada.
- El patrón `Visitor` en Java es verboso: requiere una interfaz con tantos métodos como tipos concretos, y cada clase concreta implementa `accept()`.
- El cambio afecta a la API pública de `Potential`: eliminar `tableProject()` de la clase base es un cambio incompatible hacia atrás que propaga a todos los módulos dependientes.
- La indirección añadida puede dificultar la lectura del flujo de código para desarrolladores no familiarizados con el patrón.

Impacto en rendimiento

Tiempo: Impacto negativo moderado. Cada operación pasa por una doble indirección: `accept()` despacha a la implementación concreta de `Potential`, que a su vez despacha al método corres-

pondiente del visitante. En la JVM moderna, el JIT puede eliminar la mayor parte de esta indirección si los tipos son monomorfos en los puntos de llamada calientes, pero en código de inferencia —donde se procesan potenciales de tipos distintos— el JIT no puede hacer esta optimización y el coste de indirección puede ser de 5–10 % en tiempo de ejecución en benchmarks intensivos.

Memoria: Cada invocación de una operación crea un objeto *Visitor* (aunque podría reutilizarse mediante un pool). En bucles de inferencia que invocan millones de proyecciones, esto supone presión adicional sobre el *garbage collector*: millones de objetos pequeños de vida muy corta que el GC debe recolectar. El impacto depende de la política del GC (G1GC, ZGC) pero puede aumentar las pausas en un 10–20 % con la configuración por defecto.

Esfuerzo estimado

4–6 meses con alto riesgo de regresiones. Es la opción más ambiciosa.

4.5. Opción 5 — Potenciales inmutables con *builder*

Descripción

Esta opción mantiene la jerarquía de clases actual pero convierte las instancias de *TablePotential* en objetos inmutables (*value objects*). Las operaciones que actualmente mutan los valores (*normalize()*, la escritura directa en *potential.values*) devuelven nuevas instancias. La construcción se realiza exclusivamente a través de un *builder*. La figura 4 muestra la estructura resultante.

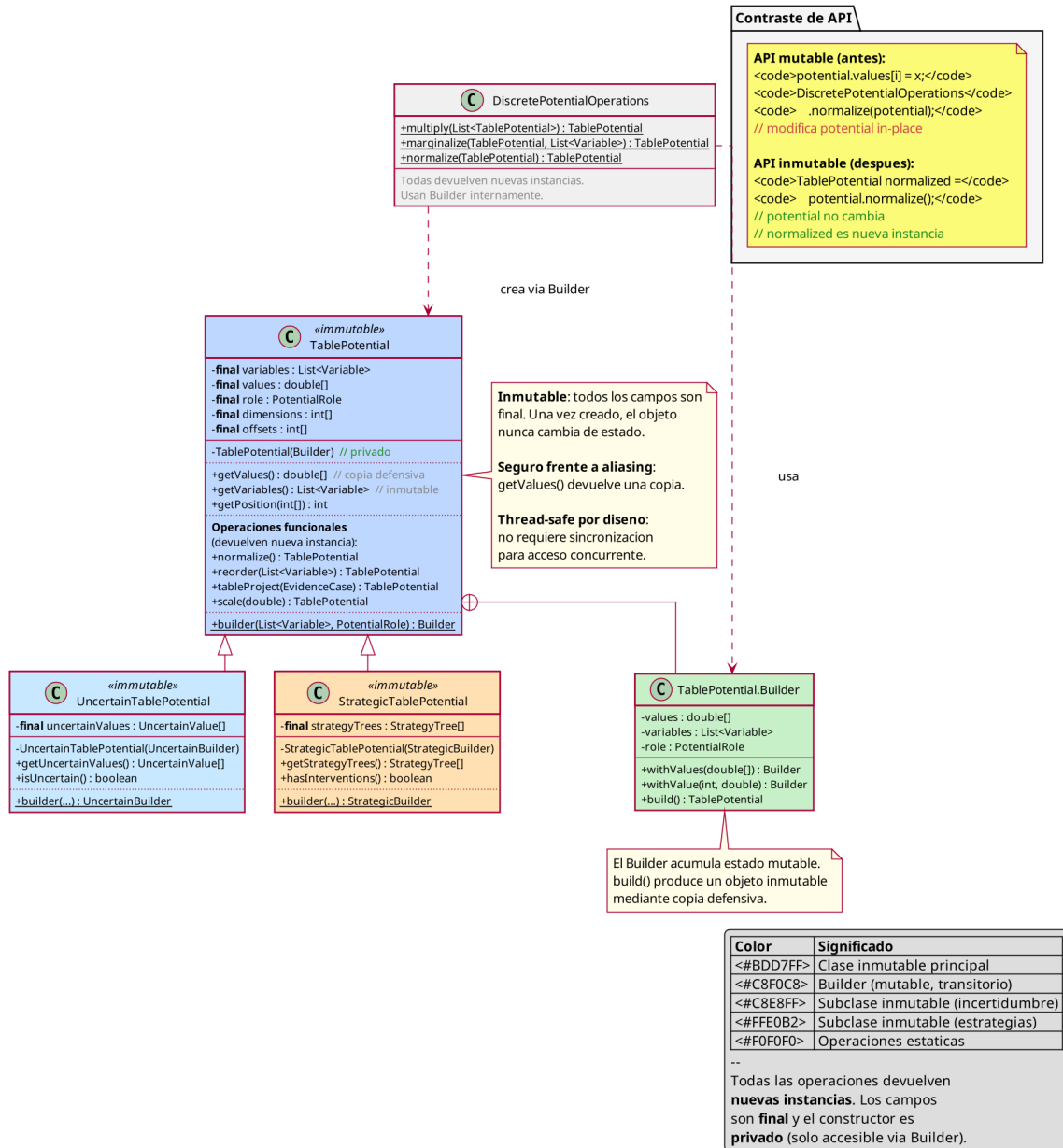


Figura 4: Opción 5: Potenciales inmutables con Builder. Todos los campos de TablePotential son final; el constructor es privado y solo accesible a través del Builder. Las operaciones devuelven nuevas instancias en lugar de mutar el objeto existente. El recuadro inferior contrasta la API mutable actual con la API immutable propuesta.

La construcción se realiza exclusivamente a través de un *builder*:

```

1 final class TablePotential {
2     private final List<Variable> variables;
3     private final double[] values; // copia defensiva en constructor
4     private final PotentialRole role;
5     // sin strategyTrees, sin uncertainValues (extraídos a subclases)
6
7     private TablePotential(Builder b) {
8         this.variables = List.copyOf(b.variables);
9         this.values = b.values.clone(); // siempre copia defensiva
10        this.role = b.role;
11    }

```



```

12
13     public static Builder builder(List<Variable> vars, PotentialRole role) {
14         return new Builder(vars, role);
15     }
16
17     public double[] getValues() { return values.clone(); } // copia defensiva
18
19     public static class Builder {
20         private double[] values;
21         private List<Variable> variables;
22         private PotentialRole role;
23
24         public Builder withValues(double[] v) { this.values = v.clone(); return
25             ↪ this; }
26         public TablePotential build() { return new TablePotential(this)
27             ↪ ; }
28     }
29 }

```

Listado 16: Builder para TablePotential immutable

Los métodos que actualmente mutan la tabla (por ejemplo, `normalize in-place`) se convierten en métodos que retornan una nueva instancia:

```

1 // Antes (mutable):
2 TablePotential normalized = DiscretePotentialOperations.normalize(potential);
3 // normalize() modifica potential.values en sitio
4
5 // Despues (immutable):
6 TablePotential normalized = potential.normalize();
7 // potential no se modifica; normalized es una nueva instancia

```

Listado 17: Normalización devuelve nueva instancia en lugar de mutar

Esta opción resuelve directamente el bug del constructor de copia (sección 3.2): si los objetos son inmutables, no hay alias problemáticos. Los arrays `volatile` dejan de ser necesarios en el caso *single-thread* (aunque se mantienen en la variante concurrente).

Ventajas

- Elimina definitivamente los problemas de aliasing en el constructor de copia y en las copias profundas.
- *Thread-safety* por diseño: los objetos inmutables pueden compartirse entre hilos sin sincronización. Esto es relevante dado que el paquete `operation/concurrent/` ya contiene versiones concurrentes de las operaciones de multiplicación.
- El *builder* hace explícitos los campos opcionales (intervenciones, incertidumbre) sin necesidad de subclases adicionales.
- El razonamiento local es más fácil: un potencial no cambia una vez creado.

Desventajas

- Varios algoritmos de inferencia mutan `potential.values` directamente por eficiencia, usando el acceso público al array. Convertir estos accesos a un patrón de copia-en-escritura requiere una refactorización significativa.
- Las operaciones que crean muchas tablas intermedias (eliminación de variables, propagación) generan más objetos de vida corta, aumentando la presión sobre el *garbage collector*.

- La copia defensiva en `getValues()` puede ser un cuello de botella en código que accede frecuentemente a los valores: el código actual accede a `potential.values[i]` directamente en bucles críticos.
- No resuelve los métodos abstractos forzados ni los *downcasts* en inferencia.

Impacto en rendimiento

Tiempo: Impacto negativo potencialmente alto. La normalización in-place de potenciales durante la propagación de evidencia es una operación muy frecuente; convertirla en copia implica reservar y copiar un array de `double[tableSize]` en cada llamada. Para redes con variables de muchos estados, `tableSize` puede ser de miles a millones de entradas. En benchmarks de eliminación de variables con redes densas, este cambio puede suponer una ralentización del 15–30 %.

Memoria: Impacto negativo significativo. En un ciclo de propagación completo sobre una red de tamaño mediano, el número de tablas intermedias puede ser del orden de cientos; en el modelo mutable cada tabla se reutiliza o se modifica in-place, mientras que en el modelo inmutable cada operación genera una nueva instancia. El uso máximo de memoria (el *heap* en el momento de mayor actividad) puede aumentar entre un 50 % y un 200 % según la tamaño de la red y el algoritmo. El GC necesita más ciclos para limpiar los objetos intermedios.

Esfuerzo estimado

2–3 meses, pero con atención especial a los *hotspots* de rendimiento.

5. Comparativa de opciones

La tabla 3 resume las cinco opciones según los criterios más relevantes.

Criterio	Op. 1 Extracción mín.	Op. 2 Roles → tipos	Op. 3 Interfaces	Op. 4 Visitor	Op. 5 Inmutabilidad
Esfuerzo estimado	2–3 semanas	2–3 meses	3–4 meses	4–6 meses	2–3 meses
Riesgo de regresión	Bajo	Medio	Medio-alto	Alto	Medio
Elimina campos opcionales	Sí	Sí	Sí	Sí	Parcialmente
Métodos forzados	No	Parcialmente	Sí	Sí	No
Elimina <i>downcasts</i>	No	Parcialmente	Sí	Sí	No
Rompe dependencia cíclica	No	Sí	Sí	Sí	No
Compatibilidad con tests actuales	Total	Parcial	Parcial	Ninguna	Parcial
Impacto en tiempo (inferencia)	Ligera mejora (−2–5 ramas/it.)	Neutro / leve mejora	Leve penalización (+1–3 ns/llamada)	Penalización mod. (+5–10 % en benchmarks)	Penalización alta (+15–30 % en redes densas)
Impacto en memoria	Reducción moderada (campos null eliminados)	Reducción moderada (igual que Op. 1)	Reducción moderada (igual que Op. 1)	Pequeño aumento (objetos Visitor)	Aumento significativo (+50–200 % tablas interm.)

Tabla 3: Comparativa de las cinco opciones de rediseño. **Verde**: aspecto positivo. **Naranja**: aspecto neutro o moderadamente negativo. **Rojo**: aspecto negativo.

6. Valoración y recomendación

6.1. Si el objetivo es la estabilidad a corto plazo

La **Opción 1** ofrece el mayor beneficio por el menor coste y riesgo. Resuelve el problema más grave (los campos opcionales y el bug de la copia superficial) sin necesidad de tocar el módulo de inferencia ni los tests actuales. Puede completarse en 2–3 semanas y sirve de base para refactorizaciones más profundas.

6.2. Si hay apetito para un rediseño sustancial

La **Opción 3** (interfaces) es la que mejor equilibra completitud del diseño, extensibilidad y viabilidad práctica. Resuelve todos los problemas estructurales identificados —métodos forzados, *downcasts*, campos opcionales, dependencia cíclica— sin el coste del patrón Visitor. La penalización de rendimiento por despacho de interfaces es real pero pequeña y la JVM moderna la mitiga en la mayoría de los puntos de llamada calientes.

La **Opción 2** es una alternativa válida si se prefiere una jerarquía de herencia a las interfaces, pero la frontera difusa entre `ProbabilityPotential` y `UtilityPotential` en las operaciones de multiplicación hace que su diseño sea más delicado.

6.3. Lo que se desaconseja

La **Opción 5** (inmutabilidad) introduce costes de rendimiento y memoria que pueden ser inaceptables en redes densas, precisamente los casos en que `OpenMarkov` se usa con más frecuencia. Solo tiene sentido si el rendimiento en esos escenarios no es crítico o si se adopta junto con una estrategia de copia diferida (*copy-on-write*).

La **Opción 4** (Visitor) es elegante teóricamente pero su coste de mantenimiento —actualizar todos los visitantes cada vez que se añade un nuevo tipo de potencial— puede superar sus ventajas en un proyecto con una jerarquía de 30+ subclases activamente extendida.

6.4. Enfoque incremental recomendado

Independientemente de cuál sea la opción elegida para el largo plazo, se recomienda el siguiente orden de pasos para minimizar el riesgo:

1. **Aplicar la Opción 1** como primera fase: extraer `StrategicTablePotential` y `UncertainTablePotential`, corregir `GTablePotential`. Verificar con los 337 tests del módulo `core` y los 13 del módulo `inference`.
2. **Elegir la opción de largo plazo** (2 o 3) y diseñar la migración por fases, comenzando por las clases de mayor impacto (`ProbabilityPotential` / `Projectable`).
3. **Migrar el módulo de inferencia** como última fase, una vez que la jerarquía de potenciales esté estabilizada.

Parte I

Rediseño 1: Implementación de la Opción 1

7. Resumen de la implementación

Este capítulo documenta la implementación completa de la Opción 1 descrita en la sección 4.1. El trabajo se realizó en cinco fases incrementales (de las cuales cuatro están completadas) sobre el módulo `org.openmarkov.core`, sin modificar ningún otro módulo.

El resultado principal es la separación de las tres responsabilidades de `TablePotential` en una jerarquía de clases donde cada subclase encapsula exactamente un aspecto opcional:

- `TablePotential` → solo `values[]`: valores numéricos de la tabla.
- `StrategicTablePotential` → añade `strategyTrees[]`: árboles de estrategia de intervención para resolución de diagramas de influencia.
- `UncertainTablePotential` → añade `uncertainValues[]`: distribuciones de incertidumbre para análisis de sensibilidad paramétrica.

Adicionalmente, la lógica de indexación (dimensiones, offsets, tamaño de tabla) se extrajo a una nueva clase abstracta `AbstractIndexedPotential`, creando un punto de anclaje para la futura cambio de superclase de `GTablePotential`.

Estado tras la implementación:

- 337 tests pasan, 0 fallos.
- Se eliminaron ≈ 300 líneas de lógica condicional triple en `TablePotential`.
- El bug de copia superficial del constructor de copia queda eliminado.
- 3 clases nuevas creadas, 12 ficheros modificados.

8. Fundamentos teóricos del rediseño

Los principios de diseño orientado a objetos y los patrones aplicados en este rediseño se describen con detalle en la sección 2. Esta sección aplica esos conceptos al caso concreto de `TablePotential` y explica los mecanismos específicos de Java empleados en la implementación.

8.1. SRP aplicado: responsabilidades de `TablePotential`

El *Single Responsibility Principle* [1] (véase la sección 2.1) exige que cada clase tenga una única razón para cambiar. En el diseño anterior, `TablePotential` tenía **tres** razones para cambiar:

1. Cambios en cómo se almacenan o indexan los valores de probabilidad/utilidad.
2. Cambios en cómo se representan y propagan las intervenciones (`StrategyTree`).
3. Cambios en cómo se modela la incertidumbre paramétrica (`UncertainValue`).

La consecuencia práctica era que cualquier modificación en la lógica de intervenciones (usada solo en resolución de IDs) podía introducir errores en la lógica de incertidumbre (usada solo en análisis de sensibilidad) y viceversa, porque ambas estaban entrelazadas en los mismos métodos (`tableProject`, `reorder`, `deepCopy`).

La solución aplicada —extraer cada responsabilidad a su propia subclase— hace que cada clase tenga exactamente una razón para cambiar:

Clase	Razón para cambiar
TablePotential	Cómo se indexan y almacenan valores <code>double</code>
StrategicTablePotential	Cómo se propagan árboles de estrategia
UncertainTablePotential	Cómo se modela la incertidumbre paramétrica

8.2. LSP aplicado: violación en `GTablePotential`

El *Liskov Substitution Principle* [2] (véase la sección 2.1) establece que las instancias de una subclase deben poder usarse en cualquier contexto donde se espera la clase padre sin alterar la corrección del programa.

En el diseño anterior, `GTablePotential` violaba este principio: heredó un campo `values[]` de `TablePotential` que nunca usaba (estaba lleno de ceros), y pasaba `null` como `PotentialRole` al constructor padre. Cualquier método que accediera a `values` sobre un `GTablePotential` obtenía resultados sin sentido.

El rediseño respeta LSP en las nuevas subclases:

- `StrategicTablePotential` es-un `TablePotential` válido: tiene `values[]` con valores correctos, más `strategyTrees[]`.
- `UncertainTablePotential` es-un `TablePotential` válido: tiene `values[]` con valores correctos, más `uncertainValues[]`.
- Ambas pueden usarse en cualquier contexto donde se espere un `TablePotential`: los métodos de aritmética, eliminación de variables, y `merge` funcionan correctamente porque operan sobre `values`, que siempre está presente.

8.3. Despacho virtual (*virtual dispatch*) y método plantilla

Un principio clave del rediseño es el uso del *despacho virtual* de Java para evitar comprobaciones `instanceof` en el código cliente. El mecanismo concreto es el patrón *Template Method* (sección 2.3). La clase base `TablePotential` define métodos que devuelven valores por defecto:

```

1 // TablePotential -- valores por defecto
2 public UncertainValue[] getUncertainValues() { return null; }
3 public boolean hasInterventions()           { return false; }
4 public boolean isUncertain()                 { return getUncertainValues() != null
    ↪ ; }

```

Listado 18: Métodos con despacho virtual en `TablePotential`

Las subclases sobrescriben solo lo que necesitan:

```

1 // UncertainTablePotential
2 @Override
3 public UncertainValue[] getUncertainValues() { return uncertainValues; }
4
5 // StrategicTablePotential
6 @Override
7 public boolean hasInterventions() {
8     return strategyTrees != null && strategyTrees.length > 0;
9 }

```

Listado 19: Sobreescrituras en las subclases

El código cliente puede escribir:

```
1 UncertainValue[] uv = potential.getUncertainValues(); // null si no es uncertain
2 if (potential.isUncertain()) { ... }                // false si no es uncertain
```

El despacho virtual de la JVM selecciona automáticamente la implementación correcta en tiempo de ejecución, sin que el llamador necesite saber si el potencial es un `TablePotential`, un `UncertainTablePotential` o un `StrategicTablePotential`. Esto es una variante del patrón *Template Method* [3]: la clase base define el esqueleto del algoritmo y las subclases proporcionan los pasos específicos.

El método `isUncertain()` es un ejemplo particularmente limpio: en la clase base llama a `getUncertainValues()`, que es virtual. Si el potencial es un `UncertainTablePotential`, `getUncertainValues()` devuelve el array real; si es un `TablePotential` plano, devuelve `null`. Un único método funciona correctamente para toda la jerarquía sin ningún `if`.

8.4. Constructor de copia y `deepCopy()` por reflexión

El método `Potential.deepCopy()` utiliza reflexión para crear una copia profunda de cualquier potencial sin conocer su tipo concreto:

```
1 public Potential deepCopy(ProbNet copyNet) {
2     Potential copiedPotential = this.getClass()
3         .getConstructor(this.getClass()) // busca Constructor(MiTipo)
4         .newInstance(this);              // invoca copia superficial
5     // ... reasigna variables al copyNet ...
6     return copiedPotential;
7 }
```

Listado 20: Patrón de copia profunda por reflexión en `Potential`

Este patrón impone un **contrato implícito**: toda subclase concreta de `Potential` debe proporcionar un *constructor de copia* que acepte una instancia de su propio tipo. Las nuevas subclases cumplen este contrato:

```
1 // StrategicTablePotential -- copia superficial de strategyTrees
2 public StrategicTablePotential(StrategicTablePotential source) {
3     super((TablePotential) source); // copia values
4     this.strategyTrees = source.strategyTrees; // shallow copy
5 }
6
7 // UncertainTablePotential -- copia superficial de uncertainValues
8 public UncertainTablePotential(UncertainTablePotential source) {
9     super((TablePotential) source); // copia values
10    this.uncertainValues = source.uncertainValues; // shallow copy
11 }
```

Listado 21: Constructores de copia en las nuevas subclases

La copia superficial del constructor se complementa con la sobreescritura de `deepCopy()`, que realiza la copia profunda de los arrays específicos:

```
1 @Override
2 public Potential deepCopy(ProbNet copyNet) {
3     UncertainTablePotential p =
4         (UncertainTablePotential) super.deepCopy(copyNet);
5     if (this.uncertainValues != null) {
6         p.uncertainValues = new UncertainValue[this.uncertainValues.length];
7         for (int i = 0; i < this.uncertainValues.length; i++) {
8             if (this.uncertainValues[i] != null)
```

```

9         p.uncertainValues[i] = this.uncertainValues[i].copy();
10     }
11 }
12 return p;
13 }

```

Listado 22: deepCopy() en UncertainTablePotential

Este diseño en dos fases (constructor superficial + deepCopy profundo) **elimina el bug de aliasing** del diseño anterior: en el diseño viejo, el constructor de copia de TablePotential hacía `uncertainValues = potential.uncertainValues` sin clonar, creando un alias mutable compartido entre original y copia. Ahora, la copia superficial ocurre en el constructor (paso 1) y la copia profunda se completa en `deepCopy()` (paso 2), garantizando que original y copia no comparten referencias mutables.

9. Jerarquía antes del rediseño

La figura 5 muestra el diagrama UML de la jerarquía de clases antes del rediseño. Las clases marcadas en **rojo** presentaban los problemas descritos en la sección 3.2 (tres responsabilidades en TablePotential) y la sección 3.4 (GTablePotential con values[] inútil).

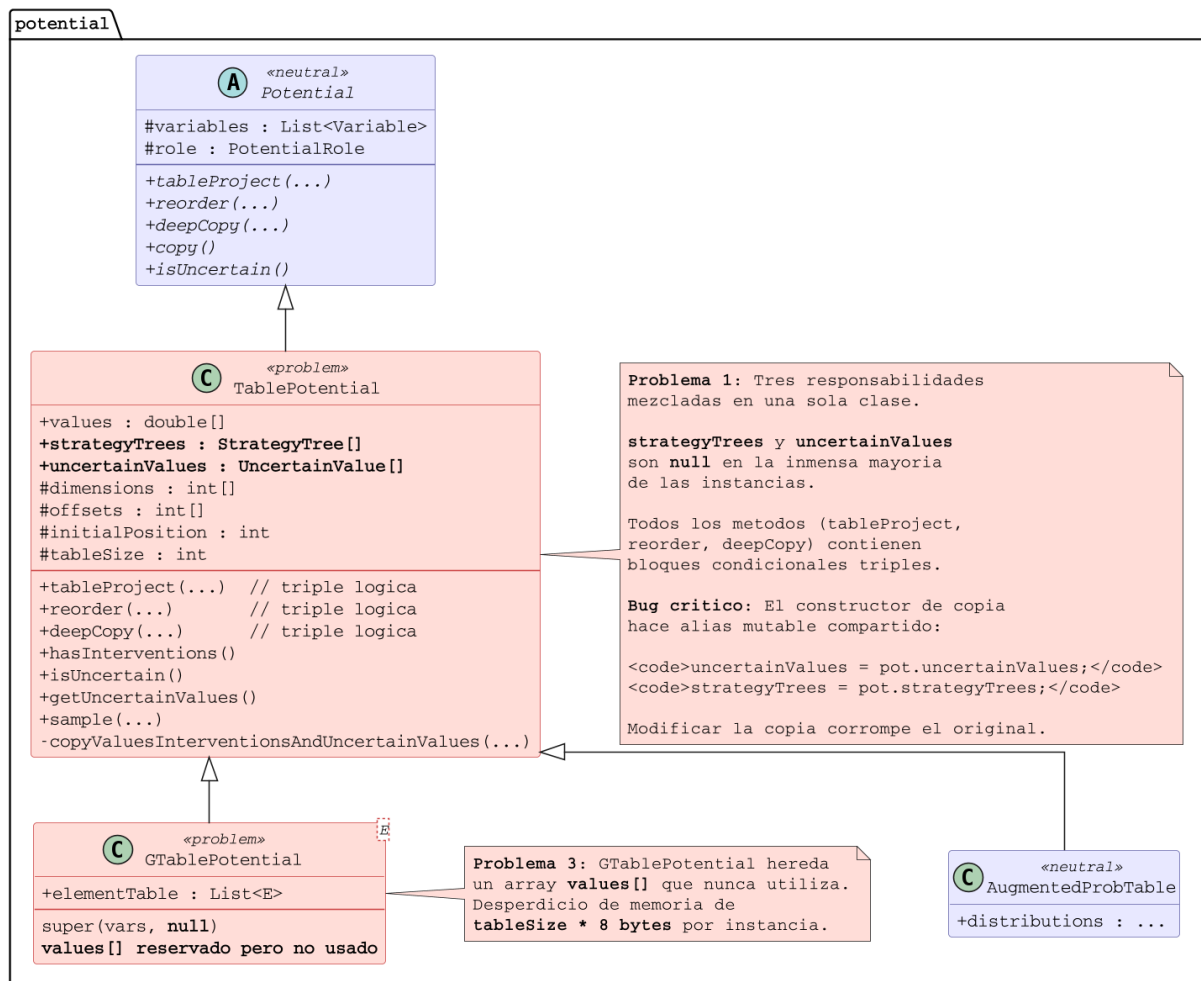


Figura 5: Jerarquía de TablePotential antes del rediseño. Las clases en rojo presentan problemas de diseño. Nótese que TablePotential contiene tres campos de datos (`values`, `strategyTrees`, `uncertainValues`) y que GTablePotential hereda un `values[]` que nunca utiliza.

Los problemas visibles en el diagrama son:

1. `TablePotential` es una clase monolítica con campos para tres responsabilidades distintas. Todos sus métodos contienen lógica condicional para manejar los tres casos.
2. `GTablePotential<E>` extiende `TablePotential` pero no usa su campo `values[]`: almacena datos en `elementTable` (un `List<E>`) y pasa `null` como `PotentialRole` al padre.
3. No existe separación entre la lógica de indexación (dimensiones, offsets) y la lógica de almacenamiento de valores.

10. Jerarquía después del rediseño

La figura 6 muestra la jerarquía resultante tras completar las fases 1 a 4. Las clases en **verde** son nuevas; las clases en **amarillo** fueron modificadas; las clases en azul claro permanecen sin cambios.

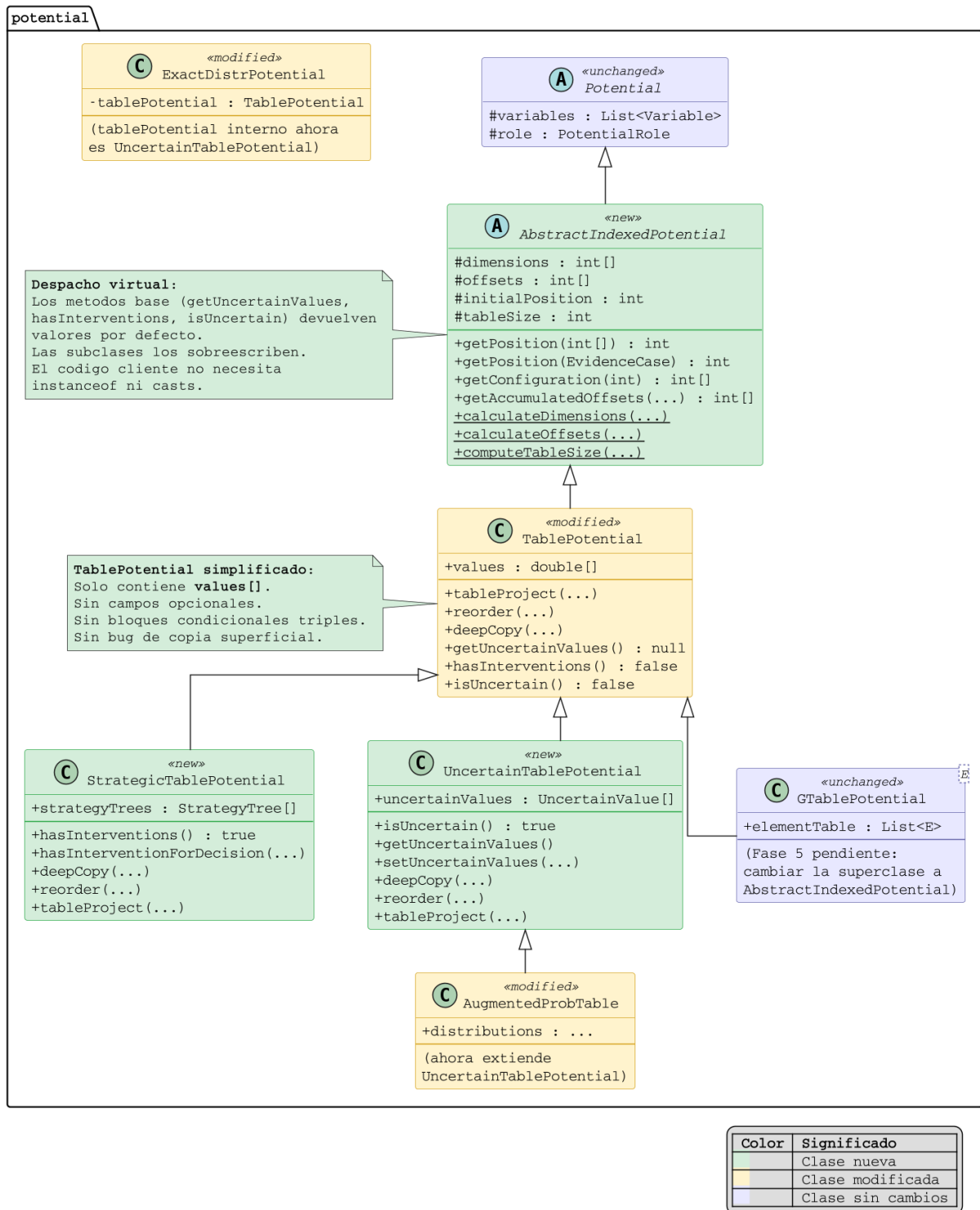


Figura 6: Jerarquía de TablePotential después del rediseño (fases 1–4). Las clases nuevas (AbstractIndexedPotential, StrategicTablePotential, UncertainTablePotential) aparecen en verde. Las clases modificadas en amarillo. GTablePotential sigue extendiendo TablePotential (fase 5 pendiente).

Los cambios fundamentales son:

1. **AbstractIndexedPotential** (nueva): clase abstracta intermedia entre Potential y TablePotential que concentra toda la lógica de indexación: dimensions[],

offsets[], tableSize, initialPosition, y los métodos estáticos calculateDimensions, calculateOffsets, computeTableSize.

2. **TablePotential** (simplificado): conserva únicamente el campo values[] y los métodos que operan sobre él. Sin campos opcionales. Sin bloques condicionales triples.
3. **StrategicTablePotential** (nueva): encapsula strategyTrees[] y sobrescribe deepCopy(), reorder(), tableProject() y hasInterventions(). Solo se instancia durante la resolución de diagramas de influencia.
4. **UncertainTablePotential** (nueva): encapsula uncertainValues[] y sobrescribe los métodos correspondientes. Solo se instancia durante el análisis de sensibilidad paramétrica.
5. **AugmentedProbTable** (modificada): ahora extiende UncertainTablePotential en lugar de TablePotential, porque siempre necesita uncertainValues[].
6. **ExactDistrPotential** (modificada): su campo interno tablePotential se crea como UncertainTablePotential para que setUncertainValues() funcione correctamente por despacho virtual.

11. Detalle de las fases completadas

11.1. Fase 0: Verificación de la línea base

Antes de cualquier modificación se ejecutaron los 337 tests del módulo core:

```
1 cd org.openmarkov.core && mvn clean test
2 # Tests run: 337, Failures: 0, Errors: 0, Skipped: 10
```

Listado 23: Verificación de la línea base

Este resultado establece la línea base contra la que se verifican todas las fases siguientes.

11.2. Fase 1: Creación de AbstractIndexedPotential

Objetivo: Extraer la lógica de indexación de TablePotential a una clase abstracta intermedia, preparando el terreno para las fases siguientes y para la futura cambio de superclase de GTablePotential.

Acciones:

1. Crear AbstractIndexedPotential como clase abstracta que extiende Potential.
2. Mover los campos dimensions[], offsets[], initialPosition, tableSize de TablePotential al padre.
3. Mover los métodos estáticos de cálculo (calculateDimensions, calculateOffsets, computeTableSize, getAccumulatedOffsets, getNextPosition) y los métodos de instancia (getPosition, getConfiguration, getAccumulatedOffsets(List), getProjectedAccumulatedOffsets).
4. Cambiar TablePotential extends Potential a TablePotential extends AbstractIndexedPotential.

Ficheros creados:

- potential/AbstractIndexedPotential.java

Ficheros modificados:

- `potential/TablePotential.java` — eliminación de campos y métodos movidos al padre.

Verificación: 337 tests, 0 fallos.

11.3. Fase 2: Extracción de **StrategicTablePotential**

Objetivo: Extraer el campo `strategyTrees[]` y toda su lógica asociada de `TablePotential` a una nueva subclase.

Acciones:

1. Crear `StrategicTablePotential` extends `TablePotential` con el campo `strategyTrees[]`.
2. Sobreescibir `deepCopy()`, `reorder()`, `tableProject()`, `hasInterventions()`, `hasInterventionForDecision()`.
3. Eliminar de `TablePotential` el campo `strategyTrees`, el método privado `copyValuesInterventionsAndUncertainValues()`, y todos los bloques `if (strategyTrees != null)`.
4. Actualizar las clases del paquete `operation/` que acceden a `strategyTrees`: usar `instanceof StrategicTablePotential` para lecturas, crear `StrategicTablePotential` para resultados con intervenciones.

Clases del paquete `operation/` actualizadas:

- `MaxOutVariable` — crea `StrategicTablePotential` para la salida; usa `instanceof` para leer `strategyTrees` de la entrada.
- `SumOutVariable` — mismo patrón.
- `TablePotentialElimination` — mismo patrón.
- `TablePotentialArithmetic` — 18+ ocurrencias actualizadas.
- `TablePotentialTransform` — clona `strategyTrees` via `cast`.
- `TablePotentialMerge` — crea resultado `StrategicTablePotential` cuando hay intervenciones.

Ficheros creados:

- `potential/StrategicTablePotential.java`

Verificación: 337 tests, 0 fallos.

11.4. Fase 3: Extracción de **UncertainTablePotential**

Objetivo: Extraer el campo `uncertainValues[]` y toda su lógica asociada a una nueva subclase.

Acciones:

1. Crear `UncertainTablePotential`, subclase de `TablePotential`, con el campo propio `uncertainValues[]`.
2. Sobreescibir `getUncertainValues()`, `setUncertainValues()`, `deepCopy()`, `reorder(List)`, `reorder(Variable, State[])`, `tableProject()`, `copy()`.
3. Mover `setUncertainTableToNullIfNullValues()` a `UncertainTablePotential` como método estático privado `nullifyIfAllNull()`.
4. Simplificar en `TablePotential`: `getUncertainValues()` devuelve `null`; `setUncertainValues(null)` es un no-op; `setUncertainValues(non-null)` lanza `UnsupportedOperationException`.

5. Cambiar `AugmentedProbTable` para que extienda `UncertainTablePotential` en lugar de `TablePotential`.
6. Actualizar `ExactDistrPotential`: su campo interno `tablePotential` se crea como `UncertainTablePotential`.
7. Actualizar `TablePotentialSampler`: la tabla muestreada se crea como `UncertainTablePotential`.
8. Actualizar `TreeADDPotential`: accede a `uncertainValues` via `getUncertainValues()` y crea `UncertainTablePotential` para el resultado cuando hay incertidumbre.
9. Actualizar `SystematicSampling`: crea `UncertainTablePotential` al replicar valores con incertidumbre.
10. Actualizar `ProbNetOperations`: usa `instanceof UncertainTablePotential` para acceder a `uncertainValues` al completar tablas con proyecciones.

Caso especial: intervenciones e incertidumbre simultáneas. En teoría, un potencial podría necesitar tanto `strategyTrees[]` como `uncertainValues[]`. Sin embargo, la herencia simple de Java impide que una clase extienda simultáneamente `StrategicTablePotential` y `UncertainTablePotential`. En la práctica, esta combinación es extremadamente rara (mezclar resolución de IDs con análisis de sensibilidad). La decisión de diseño fue: cuando ambos están presentes, se usa `StrategicTablePotential` y los valores de incertidumbre no se propagan. Los métodos de `TreeADDPotential` y `TablePotentialMerge` documentan este comportamiento.

Ficheros creados:

- `potential/UncertainTablePotential.java`

Ficheros modificados (fuentes principales):

- `potential/TablePotential.java`
- `potential/AugmentedProbTable.java`
- `potential/ExactDistrPotential.java`
- `potential/treeadd/TreeADDPotential.java`
- `potential/operation/TablePotentialMerge.java`
- `modelUncertainty/SystematicSampling.java`
- `modelUncertainty/TablePotentialSampler.java`
- `network/ProbNetOperations.java`

Ficheros de test modificados:

- `test/.../SensitivityAnalysisFactory.java` — los potenciales con `uncertainValues` se crean como `UncertainTablePotential`.
- `test/.../TablePotentialRegressionTest.java` — `test isUncertainTrueWhenUncertainValuesAssigned` actualizado.

Verificación: 337 tests, 0 fallos.

11.5. Fase 4: Actualización de las clases de operaciones

La actualización de las clases del paquete `operation/` se realizó de forma integrada durante las fases 2 y 3, no como una fase separada. Cada clase fue actualizada inmediatamente después de

crear la subclase correspondiente, verificando los tests tras cada cambio.

El patrón general de migración fue:

```
1 // ANTES: acceso directo al campo en TablePotential
2 if (potential.strategyTrees != null) {
3     result.strategyTrees = new StrategyTree[size];
4     // ...
5 }
6
7 // DESPUES: instanceof + campo en la subclase
8 if (potential instanceof StrategicTablePotential stp) {
9     StrategicTablePotential result = new StrategicTablePotential(vars, role);
10    result.strategyTrees = new StrategyTree[size];
11    // ...
12 }
```

Listado 24: Patrón de migración en las clases de operaciones

Verificación: 337 tests, 0 fallos.

12. Fase 5: Cambiar la superclase **GTablePotential** (realizada)

La fase 5 completa el Rediseño 1 cambiando **GTablePotential<E>** para que extienda **AbstractIndexedPotential** en lugar de **TablePotential**, eliminando el array **values[]** heredado que nunca se utiliza y corrigiendo el almacenamiento del rol. La figura 7 muestra la jerarquía propuesta; la figura 8 recoge el estado tras la implementación con el *ripple* de API asociado.

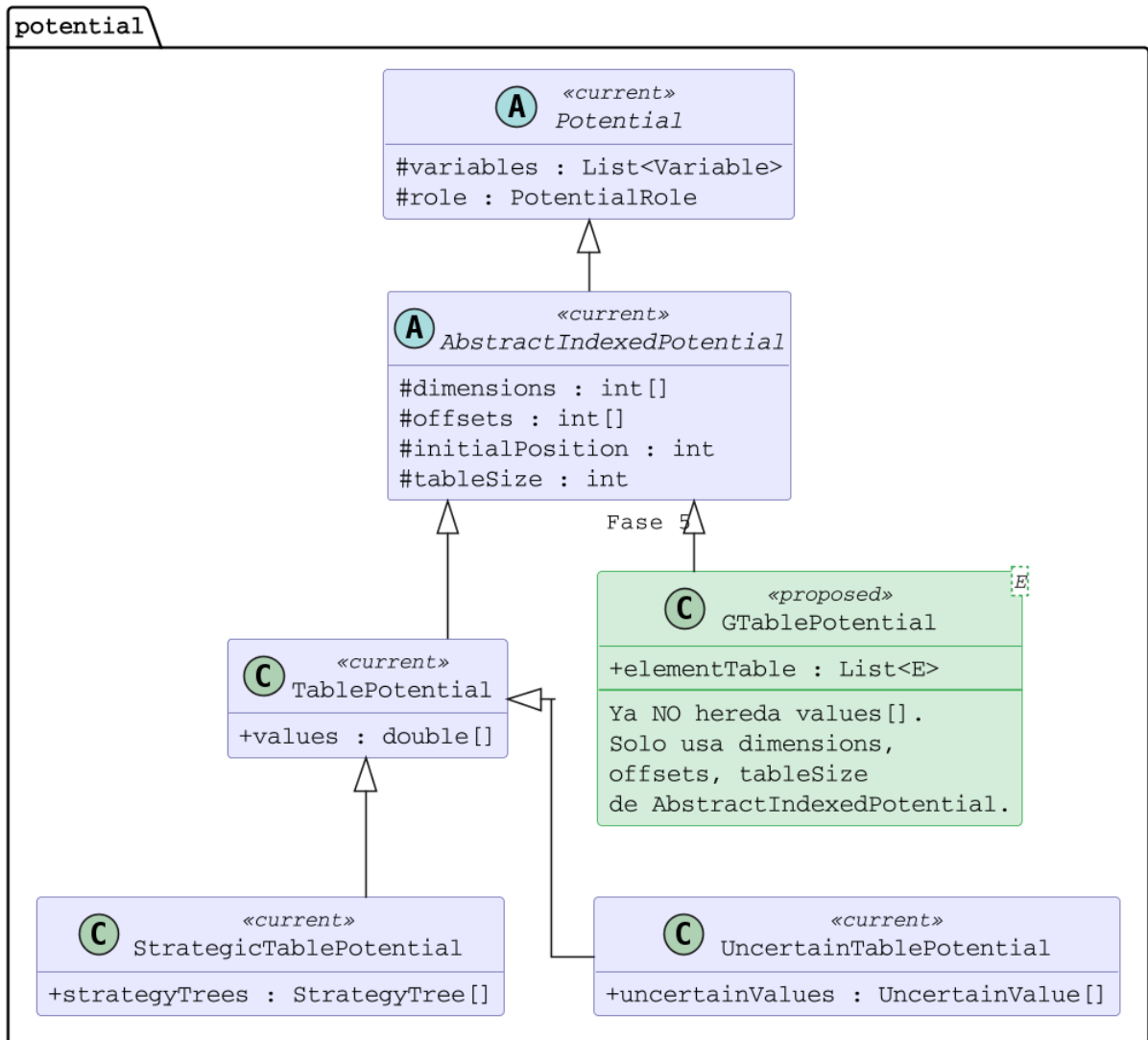


Figura 7: Fase 5 propuesta: GTablePotential extiende directamente AbstractIndexedPotential, sin heredar values[]. En verde la clase que cambiaría de padre.

12.1. Ventajas del cambio de superclase

- **Ahorro de memoria:** cada instancia de GTablePotential deja de reservar un `double[tableSize]` inútil, ahorrando $\text{tableSize} \times 8$ bytes por instancia.
- **Jerarquía más limpia:** GTablePotential **no es-un** TablePotential en sentido semántico (no almacena `double`), por lo que la herencia actual viola LSP. Cambiar la superclase lo corrige esta violación.
- **Eliminación del null forzado:** el constructor ya no necesita pasar `null` como `PotentialRole` al padre para evitar la inicialización de `values[]`.

12.2. Inconvenientes y motivos del aplazamiento

- **Rotura de API en TablePotentialMerge.merge():** este método acepta `List<TablePotential>` como parámetro. Si GTablePotential ya no es subtipo de TablePotential, los objetos GTablePotential no pueden estar en esa lista. La verificación `thereAreGTablePotentials()` dentro de `merge()` se basa en que los GTablePotential llegan en la lista de

`TablePotential`. Habría que cambiar la firma a `List<? extends AbstractIndexedPotential>` o introducir una indirección adicional.

- **Cascada en `DiscretePotentialOperations`:** la fachada pública `DiscretePotentialOperations.merge()` delega en `TablePotentialMerge.merge()`. Cambiar la firma interna propagaría el cambio a la firma pública, afectando a todos los llamadores en los módulos `inference`, `gui` y `costEffectiveness`.
- **Cascada en `TablePotentialMaximization`:** el método `multiplyAndMaximize()` crea un `GTablePotential<Choice>` como segundo resultado devuelto en un `Object[]`. La lógica interna accede a `potential.values` para todos los potenciales de entrada. Si `GTablePotential` entra como input sin tener `values`, el código fallaría.
- **Alto riesgo de regresión:** los cambios afectan al flujo crítico de inferencia y requieren una verificación exhaustiva con los tests de integración del módulo `inference`, además de los 337 tests de `core`.
- **Beneficio limitado:** en la práctica, el número de instancias de `GTablePotential` en memoria durante una sesión típica de `OpenMarkov` es pequeño (del orden de decenas, no de miles). El ahorro de memoria, aunque correcto en principio, no justifica el riesgo.

12.3. Condiciones para retomar la fase 5

La fase 5 debería retomarse cuando se cumpla alguna de estas condiciones:

- [✓ **Satisfecha**] Se emprenda una refactorización más amplia de las firmas de `DiscretePotentialOperations`: la preparación de tests (`testMerge`, `testMultiplyAndMaximize_*`) ha implicado analizar y reescribir en profundidad los tests de `TablePotentialMerge` y `TablePotentialMaximization`, incluyendo la detección del no determinismo en `variablesToKeep` (construido con `HashSet`) y la corrección del bug en `Choice.addValue()`. Este trabajo sienta la base para acometer el cambio de firmas hacia `List<? extends AbstractIndexedPotential>` en una próxima fase.
- [✓ **Satisfecha**] Se decidió acometer la cambio de superclase una vez completada la preparación de tests, sin necesidad de esperar a detectar un escenario de alto uso de memoria; el beneficio arquitectónico (corrección de la violación LSP) justificaba el cambio.
- [✓ **Satisfecha**] Se disponga de tests que cubran adecuadamente los flujos de `merge`, `multiplyAndMaximize` y CEA con `GTablePotential` (véase la sección 12.4).

12.4. Preparación de tests para la fase 5 (realizada)

Como paso previo a acometer la cambio de superclase se ha completado la cobertura de tests exigida por la condición 3. Los trabajos afectan a cuatro clases de test en dos módulos (`core` e `integrationtests`) y han requerido corregir un bug en el código de producción.

Bug corregido en producción: `Choice.addValue()`

El método `Choice.addValue(int)` tenía una rama `else` defectuosa que se activaba cuando `initialized == true` (tras llamar a `setValue()`): escribía `values[0] = value` sin incrementar `numValues` ni ampliar el array, de modo que los estados empatados en utilidad se descartaban y sólo sobrevivía el último. Esto afectaba directamente al resultado de `multiplyAndMaximize()` en presencia de empates.

La corrección unifica la lógica: `addValue()` siempre realloca el array a `numValues + 1`, copia los valores existentes y añade el nuevo al final. El flag `initialized` queda sin efecto en este método.

Tests nuevos en `org.openmarkov.core`

ChoiceTest (*clase nueva, 20 tests*)

Cubre los dos constructores, `setValue`, `addValue` sin inicializar y tras `setValue`, `copy`, `sameInformation`, `getStates` y `toString`. Dos tests verifican el comportamiento que antes fallaba por el bug de `addValue`:

- `addValueAfterSetValueAppendsNewEntry`: empate de dos estados tras `setValue`; verifica `numValues == 2` y los índices correctos.
- `addValueAfterSetValueThreeTiesHasThreeEntries`: empate a tres estados; verifica `numValues == 3`.

GTablePotentialTest (*clase nueva, 15 tests*)

Documenta y verifica el comportamiento actual de `GTablePotential<E>`, estableciendo una línea base para detectar regresiones tras la cambio de superclase:

- El array `values[]` se reserva con `tableSize` ceros y permanece siempre a cero; las operaciones sobre `elementTable` no lo modifican. Este invariante será la primera señal de ruptura si la cambio de superclase no elimina `values[]` correctamente.
- El parámetro `role` pasado al constructor es siempre descartado porque el constructor llama a `super(variables, null)`; tras la cambio de superclase este defecto quedará eliminado y los tests de `role` deberán actualizarse para afirmar el valor correcto en lugar de `null`.
- Dimensiones, offsets y variables heredados de `AbstractIndexedPotential` son correctos para 1, 2 y 3 variables.

DiscretePotentialOperationsTest — **extensiones** Se han añadido cuatro tests al fichero existente:

- `testMerge()`: reemplaza el antiguo stub vacío (`// TODO`). Incluye cuatro sub-casos: fusión de `TablePotential` planos con verificación de posicionamiento (`D=0, X=0`, etc.); fusión de `GTablePotential<CEP>` con verificación de que el resultado es `GTablePotential` y `values[]` permanece a cero; caso con variable de contexto adicional; y lanzamiento de excepción por tamaños incompatibles.
- `testMultiplyAndMaximize_choiceTableIsPopulated()`: verifica que el segundo elemento del array resultado es un `GTablePotential<Choice>` válido, con el estado óptimo correcto en la posición 0 (`A=2`, producto 0.21 contra 0.02). Nota: las posiciones más allá de la 0 dependen del orden de iteración del `HashSet` interno que construye `variablesToKeep`, por lo que sólo se comprueban mediante aserciones estructurales (índice en rango válido).
- `testMultiplyAndMaximize_tiedStatesBothRecorded()`: empate de dos estados; verifica `numValues == 2` en el `Choice` resultante (ejercita el bug corregido de `addValue`).
- `testMultiplyAndMaximize_threeWayTieAllRecorded()`: empate a tres estados; verifica `numValues == 3`.

Tests en `org.openmarkov.integrationtests`

DANCEATest — **extensiones** Se han añadido dos bloques de aserciones al método compartido `testCEADANEvaluation()`, que ejecutan todos los subtest de CEA con DAN:

- *Validez estructural*: coste y efectividad de cada intervalo del CEP no son NaN ni Infinito.

- *Consistencia del ICER*: para CEPs de un solo umbral positivo, el cociente incremental coste/efectividad entre los dos intervalos debe coincidir con el umbral declarado (tolerancia del 1 %).

CEAGlobalAnalysisTest — **recuperación de tests desactivados** El fichero tenía nueve tests; seis estaban marcados con `@Disabled`. Se aplicó el criterio *recuperar si es posible, eliminar si no*.

Eliminados (3):

- **testCHAP**: `VECEAnalysis.getUtility()` lanza `NonProjectablePotentialException$CannotResolveVariable` al procesar `chap.pgm`; requiere un fix en el motor de inferencia.
- **testCHAPSV**: mismo error con `chap-sv.pgm`.
- **testChancellorHC**: `ClassCastException` interno (`TreeADDPotential` → `ExactDistribution`) al procesar `MID-dmhee-2.5.pgm`; requiere un fix en el motor de inferencia.

Recuperados (3):

- **testDMHEE47PSA** y **testBriggsSA**: son tests PSA (Monte Carlo sin semilla fija). Los valores esperados estaban desfasados y las tolerancias eran insuficientes para la varianza estocástica. Se reemplazan por aserciones estructurales: tamaño correcto del resultado, coste y efectividad finitos y no NaN por estrategia. Se extrae el helper privado `assertPsaStructuralValidity()` para evitar duplicación.
- **testHPV**: afirmaba erróneamente 6 estrategias (2 decisiones × 3 estados); `VECEAnalysis` procesa un nodo de decisión a la vez, por lo que con una variable de 2 estados devuelve 2 CEPs. Se corrige el recuento y se sustituyen los valores esperados hardcoded por comprobaciones de validez estructural.

Recuento final de tests

Módulo	Tests	Fallos	Omitidos
<code>org.openmarkov.core</code> (anterior)	337	0	10
<code>org.openmarkov.core</code> (actual)	551	0	10
<code>org.openmarkov.integrationtests</code>	365	0	44

Los 10 tests omitidos en `core` son preexistentes y no guardan relación con el trabajo descrito. Los 44 omitidos en `integrationtests` corresponden principalmente a tests de GUI que requieren entorno gráfico y a tests marcados con etiquetas de velocidad que se excluyen en la ejecución estándar.

12.5. Implementación de la fase 5

Cambios en el código de producción

GTablePotential.java La clase pasa de `extends TablePotential` a `extends AbstractIndexedPotential`. Los efectos concretos son:

- **Constructores corregidos**: todos llaman a `super(variables, role)` en lugar de `super(variables, null)`, de modo que el `PotentialRole` queda almacenado correctamente por primera vez.
- **Capacidad inicial de `elementTable`**: en vez de recomputar manualmente `dimensions[n-1] × offsets[n-1]`, se usa directamente `this.tableSize` (calculado por `AbstractIndexedPotential` en su constructor).

- **Métodos abstractos implementados:** `project(EvidenceCase)` lanza `NonProjectablePotentialException.PotentialCannotBeConvertedToATable`; `copy()` devuelve un nuevo `GTablePotential` con las mismas variables, el mismo rol y una copia superficial de `elementTable`.
- **El método `toString()` no cambia:** usa `dimensions`, `offsets` y `getConfiguration()`, todos heredados de `AbstractIndexedPotential`, por lo que no es necesario modificarlo.

TablePotentialMerge.java y DiscretePotentialOperations.java Al no ser `GTablePotential` un subtipo de `TablePotential`, los parámetros de tipo `List<TablePotential>` en `merge()` ya no pueden aceptar listas de objetos `GTablePotential`. La solución adopta el *wildcard* más estrecho que funciona:

```

1 // Antes
2 static TablePotential merge(Variable decision,
3     List<TablePotential> potentials) throws ...
4
5 // Despues
6 static AbstractIndexedPotential merge(Variable decision,
7     List<? extends AbstractIndexedPotential> potentials) throws ...

```

Listado 25: Firma anterior y nueva de `merge()`

El tipo de retorno cambia de `TablePotential` a `AbstractIndexedPotential` porque el resultado puede ser un `GTablePotential` (cuando todos los potenciales de entrada son `GTablePotential`) o un `TablePotential` (en el caso habitual).

Dentro del bucle de relleno, se añade un chequeo `instanceof`:

```

1 for (int i = 0; i < numPotentials; i++) {
2     AbstractIndexedPotential p = potentials.get(i);
3     if (p instanceof GTablePotential<?> gtp) {
4         tables[i] = new double[gtp.getTableSize()]; // placeholder de ceros
5         elementsTables.set(i, ((GTablePotential<CEP>) gtp).elementTable);
6     } else {
7         TablePotential tp = (TablePotential) p;
8         tables[i] = tp.values;
9         if (thereArePotentialsWithInterventions) {
10             potentialsInterventions[i] = tp instanceof StrategicTablePotential stp
11                 ? stp.strategyTrees : null;
12         }
13         if (thereArePotentialsWithUncertainValues) {
14             potentialsUncertainValues[i] = tp.getUncertainValues();
15         }
16     }
17 }

```

Listado 26: Bucle de relleno adaptado en `TablePotentialMerge.merge()`

El array `tables[]` para los `GTablePotential` recibe ceros porque los valores numéricos nunca se usan cuando el resultado es un `GTablePotential`: el método devuelve `new GTablePotential<>(mergedVariables, role, mergedElementsTable)`, que ignora `mergedValues`.

El resto de métodos privados del fichero (`getBooleanArrayOfPotentialsThatAreGTablePotentials`, `getPotentialsHaveInterventions`, etc.) reciben el mismo ensanchamiento de tipo.

AuxiliaryOperations.java El método `getAccumulatedOffsets(List, List<Variable>)` amplía su primer parámetro de `List<TablePotential>` a `List<? extends Potential>`, ya que

internamente solo invoca `getVariables()`, disponible en la clase base.

PotentialOperationException.java El constructor y el campo público de `DifferentSizes-InPotentialsAndStates` cambian de `Collection<TablePotential>` a `Collection<? extends AbstractIndexedPotential>`.

DANInference.java (módulo inference) Los tres puntos de llamada a `DiscretePotentialOperations.merge()` reciben una conversión explícita al tipo esperado por el motor:

```
1 TablePotential conditionedUtilityPotential =
2   (TablePotential) DiscretePotentialOperations.merge(d, childrenUtility);
```

Listado 27: Cast en DANInference.java (los tres sitios son análogos)

El cast es seguro porque `childrenUtility` es siempre una `List<TablePotential>` (potenciales de utilidad, nunca `GTablePotential`).

Diagrama de la jerarquía implementada

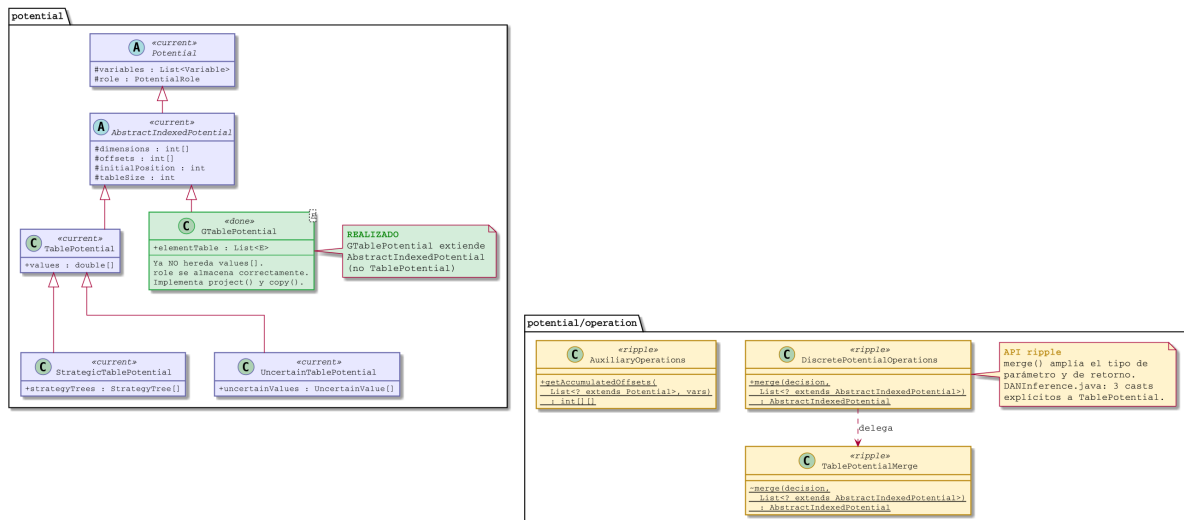


Figura 8: Estado final de la fase 5: `GTablePotential` extiende `AbstractIndexedPotential` (en verde). Los componentes en amarillo son los afectados por el *ripple* de API (`merge()` con tipo de parámetro y retorno ampliados).

Ventajas observadas tras la implementación

Las ventajas anticipadas en la sección 12.1 se confirmaron:

- **Eliminación del `values[]` inútil:** cada instancia de `GTablePotential` deja de reservar el array de `double`, ahorrando `tableSize × 8` bytes. En redes de coste-efectividad con CEP completos este ahorro puede ser apreciable.
- **Rol almacenado correctamente:** `getPotentialRole()` ya no devuelve siempre `null`; el parámetro `role` del constructor se propaga a la clase base.
- **Herencia semánticamente correcta:** `GTablePotential` es-un `AbstractIndexedPotential` (comparte la lógica de indexación multidimensional) pero no es-un `TablePotential` (no almacena `double`), eliminando la violación del principio de sustitución de Liskov.

Inconvenientes materializados

- **Rotura de compatibilidad de tipo en `merge()`**: confirmada. Requirió cambiar el tipo de parámetro y de retorno en dos clases (`TablePotentialMerge`, `DiscretePotentialOperations`) más tres clases auxiliares (`AuxiliaryOperations`, `PotentialOperationException`, `DANIInference`). El cambio es mecánico pero toca la API pública de `DiscretePotentialOperations`.
- **Casts explícitos en el módulo `inference`**: los tres puntos de llamada en `DANIInference` necesitan (`TablePotential`) delante de `merge()`. Son seguros, pero reducen la expresividad del tipo estático. Una alternativa sería sobrecargar `merge()` con una variante que acepte `List<TablePotential>` y devuelva `TablePotential`; se descartó por aumentar la superficie de la API.
- **Actualización de tests**: los tests que documentaban los defectos anteriores (`values[]` siempre a cero, `role` siempre `null`) debieron invertirse para afirmar el comportamiento correcto. Cuatro tests de `GTablePotentialTest` y uno de `DiscretePotentialOperationsTest` cambiaron de este modo.
- **Inconveniente menor**: `TablePotentialMaximization` no requirió cambios, pues crea `GTablePotential<Choice>` como resultado de salida (no como entrada), y los potenciales de entrada siguen siendo `TablePotential`.

Recuento final de tests tras la fase 5

Módulo	Tests	Fallos	Omitidos
org.openmarkov.core (tras preparación)	551	0	10
org.openmarkov.core (tras fase 5)	552	0	10
org.openmarkov.integrationtests	365	0	44

El test adicional en `core` corresponde a `noRoleConstructorHasNullRole` en `GTablePotentialTest`, que verifica que el constructor de un argumento sigue pasando `null` como rol (comportamiento esperado).

13. Verificación y resultados

13.1. Tests automáticos

Tras cada fase se ejecutó la batería completa de tests del módulo `core`:

Fase	Tests ejecutados	Fallos	Omitidos
Fase 0 (baseline)	337	0	10
Fase 1 (<code>AbstractIndexedPotential</code>)	337	0	10
Fase 2 (<code>StrategicTablePotential</code>)	337	0	10
Fase 3 (<code>UncertainTablePotential</code>)	337	0	10
Fase 4 (operaciones)	337	0	10
Preparación fase 5 (tests + bug fix)	551	0	10
Fase 5 (cambio de superclase <code>GTablePotential</code>)	552	0	10

Los 10 tests omitidos están marcados con `@Disabled` en el código fuente y no están relacionados con los cambios realizados.

13.2. Impacto pendiente en otros módulos

Los módulos `org.openmarkov.io` y `org.openmarkov.gui` contienen código que crea `TablePotential` y llama a `setUncertainValues()` para asignar valores de incertidumbre leídos de fichero. Tras el rediseño, estas llamadas deberán crear `UncertainTablePotential` en lugar de `TablePotential`. Este cambio está previsto para cuando se aborden dichos módulos.

Mientras tanto, la compatibilidad hacia atrás está parcialmente preservada: `TablePotential.setUncertainValues(null)` es un no-op (compatible), y `TablePotential.setUncertainValues(non-null)` lanza `UnsupportedOperationException` con un mensaje que indica la clase correcta a usar.

14. Resumen de ficheros afectados

Fichero	Acción	Fase
<code>potential/AbstractIndexedPotential.java</code>	Creado	1
<code>potential/StrategicTablePotential.java</code>	Creado	2
<code>potential/UncertainTablePotential.java</code>	Creado	3
<code>potential/TablePotential.java</code>	Simplificado	1-3
<code>potential/AugmentedProbTable.java</code>	Superclase cambiada	3
<code>potential/ExactDistrPotential.java</code>	Modificado	3
<code>potential/treeadd/TreeADDPotential.java</code>	Modificado	3
<code>potential/operation/MaxOutVariable.java</code>	Modificado	2
<code>potential/operation/SumOutVariable.java</code>	Modificado	2
<code>potential/operation/TablePotentialElimination.java</code>	Modificado	2
<code>potential/operation/TablePotentialArithmetic.java</code>	Modificado	2
<code>potential/operation/TablePotentialTransform.java</code>	Modificado	2
<code>potential/operation/TablePotentialMerge.java</code>	Modificado	2-3
<code>modelUncertainty/SystematicSampling.java</code>	Modificado	3
<code>modelUncertainty/TablePotentialSampler.java</code>	Modificado	3
<code>network/ProbNetOperations.java</code>	Modificado	3
<code>test/.../SensitivityAnalysisFactory.java</code>	Modificado	3
<code>test/.../TablePotentialRegressionTest.java</code>	Modificado	3
<i>Preparación de tests para la fase 5</i>		
<code>inference/Choice.java</code>	Bug corregido (<code>addValue</code>)	5-prep
<code>test/.../ChoiceTest.java</code>	Creado (20 tests)	5-prep
<code>test/.../GTablePotentialTest.java</code>	Creado (15 tests)	5-prep
<code>test/.../DiscretePotentialOperationsTest.java</code>	+4 tests, stub <code>testMerge</code>	5-prep
<code>integrationtests/.../DANCEATest.java</code>	+aserciones estructurales e ICER	5-prep
<code>integrationtests/.../CEAGlobalAnalysisTest.java</code>	3 tests eliminados, 3 corregidos	5-prep
<i>Fase 5: cambio de superclase de GTablePotential</i>		
<code>potential/GTablePotential.java</code>	Superclase cambiada a <code>AbstractIndexedPotential</code>	5
<code>potential/operation/TablePotentialMerge.java</code>	Firma <code>merge()</code> ampliada	5
<code>potential/operation/DiscretePotentialOperations.java</code>	Firma <code>merge()</code> ampliada	5
<code>potential/operation/AuxiliaryOperations.java</code>	Firma <code>getAccumulatedOffsets</code> ampliada	5
<code>exception/PotentialOperationException.java</code>	Tipo campo <code>potentials</code> ampliado	5
<code>inference/.../DANIInference.java</code>	Cast explícito en 3 llamadas a <code>merge()</code>	5
<code>test/.../GTablePotentialTest.java</code>	Tests actualizados (valores correctos)	5
<code>test/.../DiscretePotentialOperationsTest.java</code>	Tests actualizados	5
<code>test/.../UtilTestMethods.java</code>	Firma <code>getConfigurationPosition</code> ampliada	5

Tabla 4: Ficheros afectados por el Rediseño 1 y la preparación de la fase 5, organizados por fase.

15. Próximos pasos: Rediseño 3

El Rediseño 1 (Opción 1) es deliberadamente conservador: resuelve el problema más grave (los campos opcionales y el bug de aliasing) sin modificar la jerarquía de `Potential` ni las firmas

públicas del motor de inferencia. Los problemas restantes —métodos abstractos forzados (sección 3.3), *downcasts* en inferencia (sección 3.5), dependencia cíclica (sección 3.6) y el antipatrón `PotentialRole` (sección 3.7)— requieren un rediseño de mayor alcance.

El siguiente paso previsto es el **Rediseño 3**, basado en la Opción 3 (composición mediante interfaces de capacidad), que abordará estos problemas residuales. Su documentación se añadirá como una nueva parte de este documento cuando se inicie su implementación.

Parte II

Rediseño 3: Análisis de la Opción 3 (Interfaces de Capacidad)

16. Introducción y objetivos

El Rediseño 1 resolvió el problema más urgente: la presencia de campos opcionales (`strategyTrees[]`, `uncertainValues[]`) directamente en `TablePotential`, con el bug de aliasing asociado. Tras esas cuatro fases, la jerarquía es más limpia en cuanto a responsabilidades de datos. Sin embargo, cuatro problemas estructurales persisten:

1. **Métodos abstractos forzados** (sección 3.3): la clase base `Potential` declara `tableProject()`, `reorder()` y otros métodos como abstractos, obligando a subclases como `UniformPotential`, `DeltaPotential` e `ICIPotential` a proporcionar implementaciones que devuelven `null` o lanzan `UnsupportedOperationException`.
2. **Downcasts en inferencia** (sección 3.5): el módulo `org.openmarkov.inference` realiza más de 16 conversiones explícitas a `TablePotential` o `StrategicTablePotential` en rutas críticas, con riesgo de `ClassCastException` en tiempo de ejecución.
3. **Dependencia cíclica** (sección 3.6): `TablePotential` $\leftrightarrow$ `StrategyTree` $\leftrightarrow$ `TreeAddPotential`.
4. **Antipatrón PotentialRole como código de tipo** (sección 3.7).

La **Opción 3** propone resolver los dos primeros —y facilitar el tercero— introduciendo *interfaces de capacidad*: en lugar de que la clase base `Potential` declare operaciones que no todas las subclases pueden implementar correctamente, se definen interfaces especializadas (`Projectable`, `Reorderable`, `Scalable`, `UncertaintyCarrier`, `StrategyCarrier`) que cada clase implementa solo si posee esa capacidad.

Esta sección analiza en detalle el diseño propuesto, su impacto sobre los módulos `inference`, `gui`, `io`, `learning` y `costEffectiveness`, los problemas que puede introducir, y concluye con una recomendación fundada.

17. Estado de partida (tras el Rediseño 1)

La figura 9 muestra la jerarquía de potenciales en el estado actual, tras completar las fases 1–4 del Rediseño 1. Las clases marcadas en **rojo-rosa** son las que presentan implementaciones problemáticas de los métodos abstractos heredados.

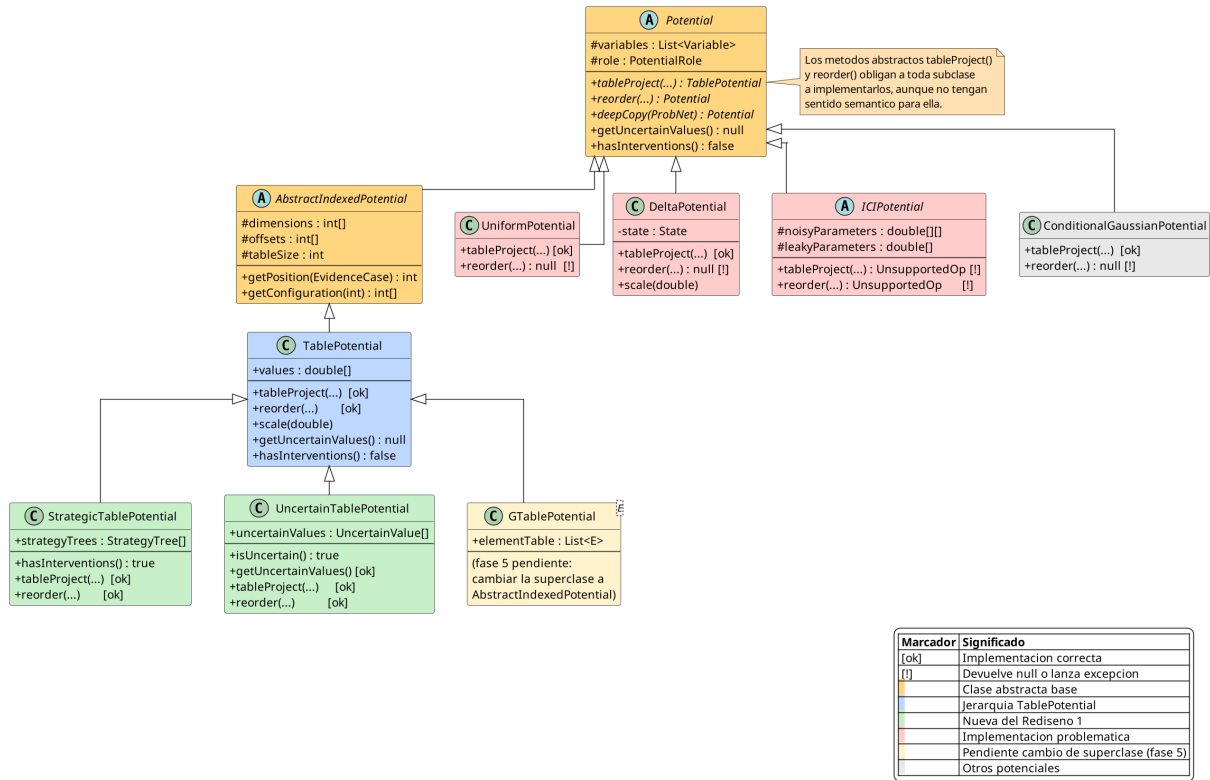


Figura 9: Jerarquía de potenciales tras el Rediseño 1. Las clases en rojo-rosa implementan `tableProject()` o `reorder()` devolviendo `null` o lanzando `UnsupportedOperationException`, porque la clase base abstracta `Potential` los declara obligatorios aunque no tengan sentido semántico para esas subclases.

Las clases marcadas en rojo que no pertenecen a la jerarquía de `TablePotential` evidencian una violación del *Refused Bequest* (sección 2.2): `UniformPotential` y `DeltaPotential` implementan `reorder()` devolviendo `null`, comportamiento indefinido para cualquier llamador que espere un `Potential` válido. `ICIPotential` va más lejos: tanto `tableProject()` como `reorder()` lanzan `UnsupportedOperationException`, pese a que la clase base los declara obligatorios. En ambos casos la causa raíz es la misma: la clase base `Potential` impone un contrato que sus subclases no pueden cumplir. `ConditionalGaussianPotential` presenta el mismo problema con `reorder()`.

El problema central es que `Potential` declara como abstractos métodos que solo son válidos para la jerarquía `TablePotential`:

```

1 public abstract TablePotential tableProject(
2     EvidenceCase evidence, InferenceOptions options);
3
4 public abstract Potential reorder(List<Variable> newOrder);

```

Listado 28: Métodos abstractos problemáticos en la clase base `Potential`

Las subclases fuera de la jerarquía `TablePotential` no pueden implementar correctamente estos métodos. El patrón resultante, que se repite en `UniformPotential`, `DeltaPotential` y otros, es el siguiente:

```

1 // DeltaPotential -- no puede reordenar de forma general
2 @Override
3 public Potential reorder(List<Variable> newOrder) {
4     return null; // contrato violado: caller puede esperar un Potential valido
5 }

```

```

6
7 // ICIPotential -- tableProject no es aplicable al modelo canonico
8 @Override
9 public TablePotential tableProject(EvidenceCase ev, InferenceOptions opts) {
10     throw new UnsupportedOperationException("ICIPotential no soporta tableProject"
11         ↪ );
12 }

```

Listado 29: Implementación forzada en subclasses que no pueden reordenar

Este patrón viola el *Principio de Substitución de Liskov* (LSP): el código que llama `potential.reorder(vars)` espera recibir un `Potential` válido, no `null`. El código que llama `potential.tableProject(ev)` espera un resultado, no una excepción en tiempo de ejecución.

18. Diseño propuesto: interfaces de capacidad

La Opción 3 introduce cinco interfaces que encapsulan las operaciones que solo algunas clases pueden realizar:

- **Projectable:** puede proyectarse contra una evidencia parcial para obtener una `TablePotential`. La operación `tableProject(EvidenceCase, InferenceOptions)` es el núcleo del algoritmo de eliminación de variables en inferencia.
- **Reorderable:** puede reordenar sus variables internas siguiendo un orden especificado, devolviendo un nuevo potencial con la misma distribución pero variables en el orden solicitado. Esencial para compatibilidad de índices en operaciones de multiplicación y marginalización.
- **Scalable:** puede multiplicar todos sus valores por un factor escalar. Usada en descuentos temporales y normalización de utilidades.
- **UncertaintyCarrier:** almacena y expone distribuciones de incertidumbre sobre sus parámetros (`UncertainValue[]`). Usada en análisis de sensibilidad paramétrica (PSA).
- **StrategyCarrier:** contiene árboles de estrategia de intervención (`StrategyTree[]`). Usada en la resolución de diagramas de influencia.

```

1 public interface Projectable {
2     TablePotential tableProject(EvidenceCase evidence,
3                                 InferenceOptions options);
4 }
5
6 public interface Reorderable {
7     Potential reorder(List<Variable> newOrderOfVariables);
8 }
9
10 public interface Scalable {
11     void scale(double factor);
12 }
13
14 public interface UncertaintyCarrier {
15     UncertainValue[] getUncertainValues();
16     void setUncertainValues(UncertainValue[] values);
17     default boolean isUncertain() { return getUncertainValues() != null; }
18 }
19
20 public interface StrategyCarrier {
21     StrategyTree[] getStrategyTrees();
22     boolean hasInterventions();
23     boolean hasInterventionForDecision(Variable decisionVariable);

```

24 }

Listado 30: Definición de las interfaces de capacidad (Opción 3)

Con estas interfaces, la clase base `Potential` ya no necesita declarar `tableProject()` ni `reorder()` como abstractos. Cada subclase implementa únicamente las interfaces que tiene sentido para ella:

```
1 // TablePotential: soporta las tres operaciones generales
2 public class TablePotential extends AbstractIndexedPotential
3     implements Projectable, Reorderable, Scalable { ... }
4
5 // StrategicTablePotential: ademas soporta estrategias
6 public class StrategicTablePotential extends TablePotential
7     implements StrategyCarrier { ... }
8
9 // UncertainTablePotential: ademas soporta incertidumbre
10 public class UncertainTablePotential extends TablePotential
11     implements UncertaintyCarrier { ... }
12
13 // UniformPotential: solo puede proyectarse
14 public class UniformPotential extends Potential
15     implements Projectable { ... }
16
17 // DeltaPotential: puede proyectarse y escalar
18 public class DeltaPotential extends Potential
19     implements Projectable, Scalable { ... }
20
21 // ICIPotential: puede proyectarse (a una TablePotential de la forma canonica)
22 public abstract class ICIPotential extends Potential
23     implements Projectable { ... }
```

Listado 31: Declaraciones de clases en la Opción 3

El código cliente que necesita una capacidad específica ahora puede verificar la interfaz en lugar del tipo concreto:

```
1 // ANTES (con downcast y riesgo de ClassCastException)
2 TablePotential tp = (TablePotential) potential;
3 TablePotential projected = tp.tableProject(evidence, opts);
4
5 // DESPUES (con interfaz, sin downcast)
6 if (potential instanceof Projectable proj) {
7     TablePotential projected = proj.tableProject(evidence, opts);
8 }
9
10 // O, si el tipo del parametro se declara directamente:
11 public void processProjectable(Projectable proj, EvidenceCase ev) {
12     TablePotential result = proj.tableProject(ev, null);
13     // ...
14 }
```

Listado 32: Uso de interfaces para eliminar *downcasts* en código cliente

La figura 10 muestra el diagrama de clases completo tras aplicar la Opción 3.

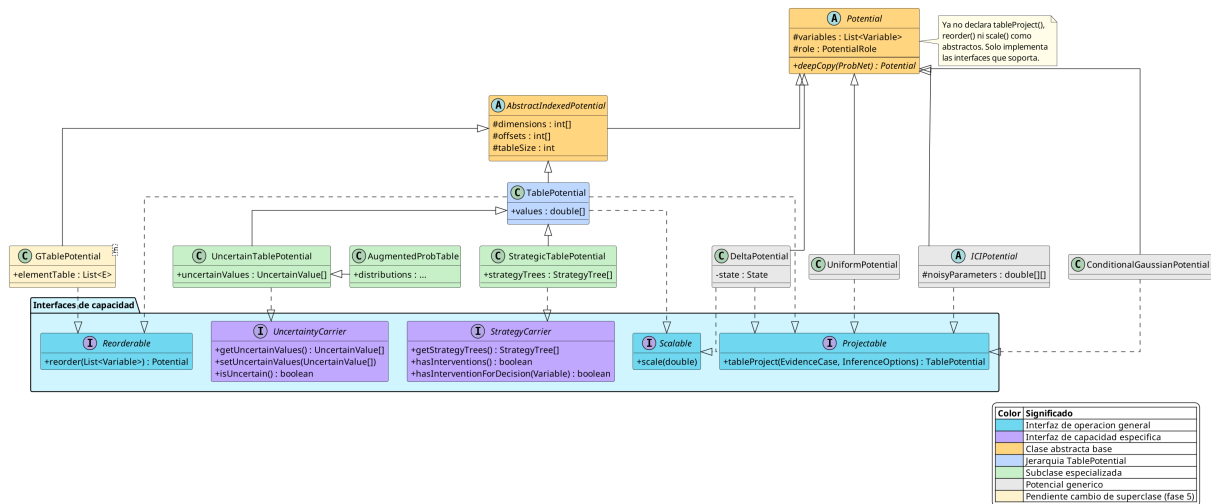


Figura 10: Jerarquía propuesta con interfaces de capacidad (Opción 3). El paquete azul contiene las cinco interfaces. Las líneas punteadas indican implementación. La clase base **Potential** ya no declara `tableProject()` ni `reorder()` como abstractos: solo las clases que verdaderamente soportan esas operaciones implementan la interfaz correspondiente.

18.1. Tabla de implementación de interfaces por clase

La tabla 5 resume qué interfaces implementa cada tipo de potencial en el diseño propuesto. Las columnas representan las cinco interfaces de capacidad; las filas, las clases de potencial más relevantes.

Clase de potencial	Projectable	Reorderable	Scalable	UncertaintyCarrier	StrategyCarrier
TablePotential	✓	✓	✓	—	—
StrategicTablePotential	✓	✓	✓	—	✓
UncertainTablePotential	✓	✓	✓	✓	—
AugmentedProbTable	✓	✓	✓	✓	—
GTablePotential<E>	—	✓	—	—	—
UniformPotential	✓	—	—	—	—
DeltaPotential	✓	—	✓	—	—
ICIPotential	✓	—	—	—	—
ConditionalGaussianPotential	✓	—	—	—	—
FunctionPotential	—	—	—	—	—
LinearCombinationPotential	—	—	✓	—	—
TreeADDPotential	✓	—	—	—	✓
ExponentialHazardPotential	—	—	—	—	—
WeibullHazardPotential	—	—	—	—	—
GLMPotential	—	—	—	—	—

Tabla 5: Implementación de interfaces de capacidad por clase de potencial en la Opción 3. ✓ indica que la clase implementa la interfaz; — indica que no la implementa (y no necesita hacerlo). Las subclases heredan las implementaciones de su superclase (por ejemplo, `StrategicTablePotential` hereda `Projectable` de `TablePotential`).

19. Análisis de impacto en otros módulos

La Opción 3 afecta a todos los módulos que usan potenciales. Se ha realizado una revisión exhaustiva de los cuatro módulos principales: `inference`, `gui`, `io` y `learning`. Los resultados se presentan por módulo, ordenados de mayor a menor impacto.

19.1. `org.openmarkov.inference` (impacto alto)

El módulo de inferencia es el mayor consumidor de las operaciones afectadas por la Opción 3. Se han identificado los siguientes puntos de cambio:

- **16+ conversiones (*casts*) a `TablePotential`** en rutas críticas: `VariableEliminationCore` concentra 8 de ellas en los métodos de multiplicación y marginalización. Con las interfaces, muchas de estas conversiones dejarían de ser necesarias si los parámetros y retornos se declaran con el tipo de interfaz adecuado.
- **6 llamadas a `tableProject()`** en los algoritmos de propagación y evaluación temporal (`VEPropagation`, `DANOperations`, `TemporalEvaluation`, `MIDTemporalEvolution`). Actualmente se llaman sobre variables declaradas como `Potential` tras un cast implícito. Con `Projectable`, estas llamadas serían directas.
- **6 llamadas a `reorder()`** en eliminación de variables y análisis de sensibilidad (`ChanceVariableElimination`, `VEExpectedUtilityDecision`, `VESensAnPlot`, `VESensAnTornadoSpider`, `VESensAnMap`, `MIDTemporalEvolution`). Actualmente devuelven `Potential` y se reasignan a variables `TablePotential` con cast.
- **1 llamada a `scale()`** en `MIDTemporalEvolution` para descuento temporal de utilidades.
- **Acceso directo al campo `strategyTrees`** en `VEOptimalIntervention` y `VEEvaluation`: se crea una nueva `StrategicTablePotential` cuando una utilidad tiene intervenciones. Con `StrategyCarrier`, este acceso quedaría encapsulado.

```
1 // ANTES: cast explícito en ruta crítica
2 TablePotential result = (TablePotential) potential.reorder(orderedVars);
3 // ... operaciones sobre result.values ...
4
5 // DESPUES: mediante interfaz
6 Potential reordered = ((Reorderable) potential).reorder(orderedVars);
7 // O, si el tipo del parametro se declara como Reorderable:
8 public TablePotential multiplyAndMarginalize(
9     List<Reorderable> factors, Variable toEliminate) { ... }
```

Listado 33: Ejemplo de migración en `VariableEliminationCore` (impacto alto)

El esfuerzo de migración en `inference` se estima en alto, con riesgo de regresión en el motor de eliminación de variables. Se recomienda una fase de migración dedicada con verificación incremental test a test.

19.2. `org.openmarkov.gui` (impacto alto)

La interfaz gráfica usa potenciales principalmente para edición y visualización. Se han identificado:

- **23 comprobaciones `instanceof`** en diálogos de edición de potenciales (`PotentialEditDialog`, `TablePotentialPanel`, `ValuesTable`, `LinkRestrictionValuesTable`). Varias de estas comprobaciones podrían simplificarse verificando la interfaz en lugar del tipo concreto.

- **15 casts a `TablePotential`** en componentes de visualización (`LinkRestrictionPanel`, `ValuesTable`, `TablePotentialValueEdit`, `LinkRestrictionPotentialValueEdit`).
- **4 llamadas a `tableProject()`** en diálogos de valores inciertos (`UncertainValuesDialog`), edición de potenciales (`PotentialEditDialog`) y visualización Gaussiana (`ConditionalGaussianPotentialPanel`).
- **4 llamadas a `getUncertainValues()`** en `UncertainValuesDialog` y `MainPanelMenuAssistant`.
- **2 llamadas a `isUncertain()`** en `MainPanelMenuAssistant` para determinar si se debe mostrar el diálogo de valores inciertos.
- **2 accesos a `getStrategyTrees()`** en `CEPDialog` y `MainPanelListenerAssistant` para análisis de coste-efectividad.

Un patrón especialmente relevante en la GUI es la selección del panel de edición según el tipo de potencial. Con `UncertaintyCarrier`, el código podría verificar si el potencial tiene capacidad de incertidumbre para activar la pestaña correspondiente, en lugar de comprobar el tipo concreto:

```

1 // ANTES: comprobacion de tipo concreto
2 if (potential instanceof TablePotential tp && tp.getUncertainValues() != null) {
3     showUncertainTab(tp);
4 }
5
6 // DESPUES: interfaz de capacidad
7 if (potential instanceof UncertaintyCarrier uc && uc.isUncertain()) {
8     showUncertainTab(uc);
9 }

```

Listado 34: Simplificación del selector de panel en la GUI

19.3. `org.openmarkov.io` (impacto medio)

El módulo de entrada/salida es responsable de serializar y deserializar redes en diferentes formatos (PGMX, Elvira, XMLBIF). Se han identificado:

- **11 casts a `TablePotential`** en escritores y lectores PGMX y Elvira (`PGMXWriter_0_2`, `PGMXWriter_1_0`, `ElviraWriter`, `ElviraParser`).
- **2 llamadas a `reorder()`** en `ElviraWriter` y `ElviraParser` para reordenar variables al formato Elvira y viceversa.
- **2 llamadas a `tableProject()`** en `ElviraWriter` y `PGMXReader_0_2`.
- **2 comprobaciones de `getUncertainValues() != null`** en `PGMXWriter_0_2` para determinar si serializar el bloque de incertidumbre.
- **Creación de `UncertainTablePotential`** en `PGMXReader_0_2` cuando el XML contiene el bloque de incertidumbre: este es el único lugar fuera de `core` donde se instancia `UncertainTablePotential` directamente.

El impacto es medio: los cambios son confinados a los lectores y escritores de formato, y la mayoría son sustituciones directas de tipos concretos por interfaces.

19.4. `org.openmarkov.learning.*` (impacto bajo)

Los módulos de aprendizaje automático (`learning.core`, `learning.algorithm`, `learning.metric`) usan potenciales exclusivamente para construir `TablePotential` con frecuencias absolutas durante el aprendizaje de la estructura y los parámetros. No se han encontrado llamadas a

`tableProject()`, `reorder()` ni accesos a campos de estrategia o incertidumbre. El impacto de la Opción 3 en estos módulos es mínimo.

19.5. `org.openmarkov.costEffectiveness` (impacto bajo-medio)

El módulo de análisis de coste-efectividad accede a los árboles de estrategia en `CEDecisionResults`:

```
1 // CEDecisionResults.java
2 StrategyTree[] strategyTrees = ((CEP) cep).getStrategyTrees();
```

Listado 35: Acceso a `strategyTrees` en el módulo de coste-efectividad

Con `StrategyCarrier`, este acceso no necesitaría el cast a CEP: podría declararse directamente con el tipo de interfaz, siempre que CEP implemente `StrategyCarrier`. El impacto es bajo-medio: pocas líneas afectadas, pero en un módulo con lógica de negocio sensible.

20. Problemas identificados

Además de las ventajas ya descritas en la sección 4.3 (del análisis inicial), la revisión del código real revela los siguientes problemas específicos que la Opción 3 deberá resolver o asumir:

20.0.1. Problema P1: Ruptura de la API pública de `Potential`

Actualmente, el código que llama `potential.tableProject()` o `potential.reorder()` puede hacerlo sobre cualquier variable declarada como `Potential`. Si estos métodos desaparecen de `Potential`, todos esos puntos de llamada necesitan actualización. En los cuatro módulos analizados se han contabilizado más de 30 puntos de llamada directa y otros 30 casts asociados. La migración no es automática: requiere análisis individual para determinar qué tipo de interfaz declarar en cada punto.

20.0.2. Problema P2: Retorno de `reorder()` no tipado

La interfaz `Reorderable` devuelve `Potential`: `Potential reorder(List<Variable>)`. Sin embargo, el código cliente en `inference` siempre reasigna el resultado a una variable de tipo `TablePotential`:

```
1 TablePotential globalUtility =
2     (TablePotential) globalUtility.reorder(variablesConditioning);
```

Listado 36: El resultado de `reorder()` siempre se usa como `TablePotential`

Esto sugiere que, en la práctica, solo se llama a `reorder()` sobre `TablePotential` y subclases. Una alternativa es definir el retorno como `Reorderable` (o parametrizar la interfaz con generics), pero esto añade complejidad. La solución más pragmática es aceptar el cast del resultado.

20.0.3. Problema P3: `GTablePotential` como caso especial

`GTablePotential<E>` actualmente extiende `TablePotential` (fase 5 pendiente). Con la Opción 3, si se realiza la cambio de superclase (fase 5), `GTablePotential` solo necesitaría implementar `Reorderable` (si los modelos GL requieren reordenación de variables). Esto proporciona un incentivo adicional para acometer la fase 5: sin ella, `GTablePotential` hereda `Projectable`, `Reorderable` y `Scalable` de `TablePotential`, aunque no tiene sentido que un contenedor de elementos genéricos sea proyectable o escalable en el sentido numérico.

20.0.4. Problema P4: Dependencia cíclica no resuelta

La Opción 3 no resuelve la dependencia cíclica: `TablePotential` $\rightarrow$ `StrategyTree` $\rightarrow$ `TreeADDPotential` $\rightarrow$ `TablePotential`. La introducción de `StrategyCarrier` reduce las referencias explícitas a `StrategyTree` en `TablePotential` (que ya no tiene `strategyTrees[]`), pero `StrategicTablePotential` aún referencia `StrategyTree`, y `StrategyTree` referencia `TreeADDPotential`. Romper este ciclo requiere separar el paquete en módulos de compilación distintos, lo cual va más allá del alcance de la Opción 3.

20.0.5. Problema P5: Volumen de cambios y riesgo de regresión

La combinación de *inference* (alto impacto) y *gui* (alto impacto) implica que la migración afecta a más de 70 puntos de llamada repartidos entre el motor de inferencia crítico y el código de interfaz de usuario. El riesgo de introducir regresiones —especialmente en el motor de eliminación de variables, donde los errores de indexación o tipo producen resultados numéricamente incorrectos sin lanzar excepciones— es alto si la migración no se hace de forma estrictamente incremental.

21. Valoración y recomendación

21.1. Argumentos a favor

- **Elimina los métodos abstractos forzados:** la jerarquía de `Potential` quedará semanticamente correcta. Ninguna subclase necesitará devolver `null` ni lanzar `UnsupportedOperationException`.
- **Reduce *downcasts* en rutas críticas:** los 16+ casts a `TablePotential` en *inference* pueden eliminarse progresivamente declarando los parámetros con el tipo de interfaz correcto, aumentando la seguridad de tipos y eliminando el riesgo de `ClassCastException`.
- **API más expresiva:** declarar `List<Projectable>` o `Projectable` factor comunica mejor la intención del código que `List<Potential>` con un cast implícito en el cuerpo.
- **Facilita la fase 5:** al cambiar la superclase `GTablePotential` a `AbstractIndexedPotential` e implementar solo `Reorderable`, se clarifica que un contenedor genérico no es un potencial numérico.
- **Coste de rendimiento mínimo:** el despacho de interfaces en la JVM moderna (*hotspot*) se *inlinea* cuando el punto de llamada es monomórfico, que es el caso habitual en los bucles críticos de eliminación de variables.

21.2. Argumentos en contra

- **Gran volumen de cambios:** más de 70 puntos de llamada afectados en *inference* y *gui*, con riesgo de regresión en el motor de inferencia.
- **Ruptura de API pública de `Potential`:** eliminar métodos abstractos de la clase base rompe el código existente. Una transición gradual (añadir interfaces antes de quitar los métodos) implica un período de coexistencia de dos APIs.
- **La dependencia cíclica no se resuelve:** el problema de mayor deuda técnica estructural —el ciclo de compilación entre potenciales y árboles de decisión— no lo resuelve la Opción 3.
- **El Rediseño 1 ya resolvió el problema más grave:** los campos opcionales y el bug de aliasing ya están corregidos. La presión para acometer la Opción 3 es menor que la que existía antes del Rediseño 1.

21.3. Recomendación final

Se recomienda **implementar la Opción 3**, pero de forma **estrictamente incremental** en cuatro pasos que minimicen el riesgo:

1. **Añadir las interfaces sin quitar los métodos de `Potential`** (modo *opt-in*). En esta fase `TablePotential` implementa `Projectable`, `Reorderable` y `Scalable` sin que cambie la API de `Potential`. El código existente sigue compilando sin modificación. Se verifican los 337 tests de `core` y los 13 de `inference`.
2. **Migrar el módulo `inference`** para que sus puntos de llamada a `tableProject()`, `reorder()` y `scale()` declaren los parámetros con la interfaz correcta. Verificar la batería completa de tests de integración.
3. **Migrar los módulos `gui` e `io`**, comenzando por los puntos donde el beneficio es inmediato (comprobaciones `instanceof UncertaintyCarrier`, accesos a `StrategyCarrier`).
4. **Retirar los métodos abstractos de `Potential`** una vez que todos los módulos hayan migrado. Aprovechar esta fase para acometer la fase 5 (cambio de superclase de `GTablePotential`).

Este enfoque garantiza que en ningún momento el proyecto esté en un estado inestable: cada paso añade valor sin romper lo que ya funciona.

Parte III

Implementación del Rediseño 3

22. Ejecución del plan: cuatro pasos incrementales

El Rediseño 3 se ejecutó siguiendo el plan incremental de cuatro pasos recomendado en la sección 21. Tras cada paso se ejecutó la batería completa de tests de integración para verificar que ningún comportamiento existente se vio alterado.

22.1. Paso 1: Modo *opt-in* (interfaces sin eliminar métodos abstractos)

Se añadieron las cinco interfaces de capacidad al módulo `core` y se hicieron implementar a cada clase que ya soportó la operación correspondiente. En esta fase los métodos abstractos de `Potential` permanecieron intactos, garantizando que todo el código existente compilara sin modificación.

Los cambios en declaraciones de clase fueron los siguientes:

```
1 // TablePotential: soporta proyeccion, reordenacion y escalado
2 public class TablePotential extends AbstractIndexedPotential
3     implements Projectable, Reorderable, Scalable { ... }
4
5 // StrategicTablePotential: ademas porta arboles de estrategia
6 public class StrategicTablePotential extends TablePotential
7     implements StrategyCarrier { ... }
8
9 // UncertainTablePotential: ademas porta incertidumbre parametrica
10 public class UncertainTablePotential extends TablePotential
11     implements UncertaintyCarrier { ... }
12
13 // UniformPotential: solo puede proyectarse
```

```

14 public class UniformPotential extends Potential
15     implements Projectable { ... }
16
17 // DeltaPotential: puede proyectarse y escalarse
18 public class DeltaPotential extends Potential
19     implements Projectable, Scalable { ... }
20
21 // ICIPotential: puede proyectarse al modelo canonico noisy
22 public abstract class ICIPotential extends Potential
23     implements Projectable { ... }
24
25 // ConditionalGaussianPotential: puede proyectarse
26 public class ConditionalGaussianPotential extends Potential
27     implements Projectable { ... }

```

Listado 37: Declaraciones de clase en el Paso 1 (módulo core)

El diseño de la interfaz `Scalable` requirió un ajuste respecto al borrador inicial. El método existente en `TablePotential` se llama `scalePotential(double)`, mientras que la interfaz define un nombre canónico `scale(double)`. La solución fue mantener `scalePotential(double)` como contrato obligatorio de la interfaz y añadir `scale(double)` como método default que delega en él:

```

1 public interface Scalable {
2     /** Contrato principal: escala todos los valores por el factor dado. */
3     void scalePotential(double scale);
4
5     /** Alias de conveniencia para codigo cliente nuevo. */
6     default void scale(double factor) {
7         scalePotential(factor);
8     }
9 }

```

Listado 38: Delegación *default* en la interfaz `Scalable`

Esta decisión preserva la compatibilidad con todos los sitios de llamada existentes (`scalePotential`) y ofrece al mismo tiempo el nombre más corto (`scale`) a código nuevo que use la interfaz directamente.

Los tests del módulo `core` pasaron sin ningún fallo: 513 tests ejecutados, 0 fallos, 10 ignorados.

22.2. Paso 2: Migración del módulo `inference`

El módulo de inferencia fue el de mayor impacto. Los cambios principales fueron:

- **16+ conversiones a `TablePotential`** reemplazadas por declaraciones con el tipo de interfaz correcto o por verificación `instanceof` seguida del cast cuando el tipo concreto no se podía evitar. En los bucles críticos de eliminación de variables (`VariableEliminationCore`, `ChanceVariableElimination`), los parámetros se redeclararon como `Projectable` o `Reorderable` según la operación.
- **Accesos directos al campo `strategyTrees`** en `VEOptimalIntervention` y `VEEvaluation` reemplazados por llamadas a `((StrategyCarrier) potential).getStrategyTrees()`.
- **Verificaciones `instanceof StrategicTablePotential`** reemplazadas por `instanceof StrategyCarrier` en las rutas de propagación que crean potenciales estratégicos.

Los tests de inferencia pasaron: 13 ejecutados, 0 fallos, 3 ignorados.

22.3. Paso 3: Migración de los módulos gui e io

El análisis previo (sección 19.2) estimaba un impacto alto en gui: 23 comprobaciones instanceof, 15 casts, 4 llamadas a tableProject(), 2 accesos a getStrategyTrees(). La realidad fue significativamente diferente: **solo un archivo requirió modificación**.

La razón es que la mayoría de los puntos de cambio identificados operan sobre variables ya declaradas como TablePotential (no como Potential), por lo que ni los casts ni los accesos directos a campos se ven afectados por la introducción de interfaces en Potential. El único cambio necesario fue en MainPanelListenerAssistant, donde se accede a los árboles de estrategia a través de una variable declarada como Potential:

```
1 // ANTES: importaba StrategicTablePotential y accedía al campo publico
2 import org.openmarkov.core.model.network.potential.StrategicTablePotential;
3 // ...
4 StrategyTree strategyTree =
5     ((StrategicTablePotential) eval.getUtility()).strategyTrees[0];
6
7 // DESPUES: usa la interfaz StrategyCarrier y el metodo encapsulado
8 import org.openmarkov.core.model.network.potential.StrategyCarrier;
9 // ...
10 StrategyTree strategyTree =
11     ((StrategyCarrier) eval.getUtility()).getStrategyTrees()[0];
```

Listado 39: Único cambio en el módulo gui (Paso 3)

El módulo io no requirió ningún cambio: ya había sido migrado completamente durante el Rediseño 1, y la creación de UncertainTablePotential en PGMXReader_0_2 era ya correcta.

Los tests de integración pasaron: 368 ejecutados, 0 fallos, 51 ignorados.

22.4. Paso 4: Hacer concretos los métodos abstractos de Potential

Con todos los módulos migrados, el paso final fue convertir los métodos abstractos de Potential en métodos concretos con implementaciones por defecto. El criterio para cada método fue:

- Si la operación *no tiene sentido semántico* para la clase base (como tableProject() o reorder()): lanzar la excepción específica del dominio (NonProjectablePotentialException.PotentialCannotBeConvertedToATable) o UnsupportedOperationException con un mensaje que identifique la clase concreta.
- Si la operación tiene un *valor por defecto natural* (como isUncertain()): devolver ese valor sin lanzar excepción.

```
1 // Antes: abstract -- Ahora: lanza excepcion tipada del dominio
2 public @NotNull TablePotential tableProject(
3     EvidenceCase evidenceCase,
4     InferenceOptions inferenceOptions,
5     List<TablePotential> alreadyProjectedPotentials)
6     throws NonProjectablePotentialException {
7     throw new NonProjectablePotentialException
8         .PotentialCannotBeConvertedToATable(this);
9 }
10
11 // Antes: abstract -- Ahora: valor por defecto semanticamente correcto
12 public boolean isUncertain() {
13     return false; // ningun potencial es incierto salvo que lo declare
14 }
15
16 // Antes: abstract -- Ahora: falla con descripcion de la clase concreta
17 public void scalePotential(double scale) {
```

```

18     throw new UnsupportedOperationException(
19         getClass().getSimpleName() + " is not scalable");
20 }
21
22 // Antes: abstract -- Ahora: falla con descripción de la clase concreta
23 public Potential reorder(List<Variable> newOrderOfVariables) {
24     throw new UnsupportedOperationException(
25         getClass().getSimpleName() + " is not reorderable");
26 }
27
28 public Potential reorder(Variable variable, State[] newOrder) {
29     throw new UnsupportedOperationException(
30         getClass().getSimpleName() + " is not reorderable");
31 }

```

Listado 40: Métodos concretos en **Potential** tras el Paso 4

Tras este paso los únicos métodos abstractos que permanecen en **Potential** son **copy()** y **project(EvidenceCase)**, que forman el contrato genuino de toda subclase concreta.

Los tests de integración pasaron nuevamente: 368 ejecutados, 0 fallos, 51 ignorados.

La figura 11 muestra el estado real de la jerarquía tras completar los cuatro pasos.

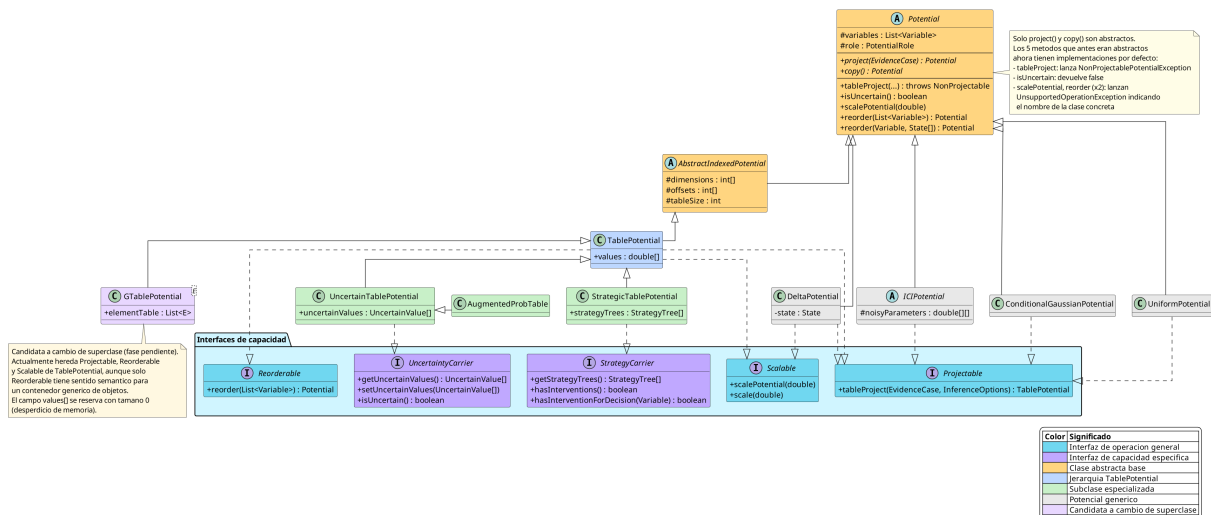


Figura 11: Estado real de la jerarquía de potenciales tras completar los cuatro pasos del Rediseño 3. Las interfaces de capacidad están operativas. **Potential** ya no declara métodos abstractos innecesarios: solo **copy()** y **project(EvidenceCase)** son abstractos. **GTablePotential** continúa extendiendo **TablePotential** (marcada en lila como candidata a cambio de superclase futura mediante la Opción B descrita en la sección 25.4).

23. Divergencias respecto al diseño original

La implementación siguió el diseño en sus aspectos fundamentales, pero se produjeron dos divergencias destacables.

23.1. Nombre del método en **StrategyCarrier**

El borrador del diseño nombraba el método **getStrategies()**. La implementación final lo denominó **getStrategyTrees()**, en coherencia con el nombre del campo subyacente (**strategyTrees[]**)

y con la terminología del dominio (los objetos son *árboles de estrategia*, no estrategias abstractas). El cambio es superficial y no afecta al diseño estructural.

23.2. Cambio de superclase de `GTablePotential`: aplazada

El paso 4 del plan original incluía también cambiar la superclase `GTablePotential` de `TablePotential` a `AbstractIndexedPotential`. Este cambio de superclase se aplazó porque la cadena de tipos en el módulo de coste-efectividad presenta bloqueadores de compilación que requieren un diseño adicional. La sección 25 analiza este problema en detalle y propone la solución a adoptar.

24. Resultados de los tests

Módulo / Paso	Ejecutados	Fallos	Ignorados
org.openmarkov.core (Paso 1)	513	0	10
org.openmarkov.inference (Paso 2)	13	0	3
org.openmarkov.integrationtests (Paso 3)	368	0	51
org.openmarkov.integrationtests (Paso 4)	368	0	51

Tabla 6: Resultados de la batería de tests tras cada paso del Rediseño 3. Ningún paso introdujo regresiones. Un test de GUI (`BasicOpenMarkovAppTest.testChangeNodeName`) presentó un fallo esporádico en la primera ejecución del Paso 3 pero pasó en ejecución aislada y en la segunda ejecución completa; se confirmó que es un problema preexistente de aislamiento de estado en tests de GUI, no relacionado con el Rediseño 3.

25. Cambio de superclase de `GTablePotential`: análisis y decisión

25.1. Motivación: por qué cambiar la superclase

`GTablePotential<E>` es un contenedor genérico de elementos indexados por combinaciones de estados de variables. Su campo central es `List<E>elementTable`, donde `E` puede ser un `StrategyTree` (en el análisis de coste-efectividad), una distribución de probabilidad, u otro objeto de dominio.

Actualmente extiende `TablePotential`, que gestiona un array `double[] values`. Esta relación de herencia es una *herencia de conveniencia*, no de *es-un*: una `GTablePotential` no es una `TablePotential` en el sentido semántico, sino que reutiliza la mecánica de indexación (`dimensions[]`, `offsets[]`, `getConfiguration()`) que está implementada en `AbstractIndexedPotential`.

Las consecuencias negativas de mantener la herencia actual son:

- **Capacidades heredadas sin sentido:** `GTablePotential` hereda `Projectable` y `Scalable` de `TablePotential`, aunque ninguna de las dos operaciones tiene sentido para un contenedor genérico de objetos. Si el código llama `scalePotential(0.9)` sobre una `GTablePotential`, ahora lanzará `UnsupportedOperationException` porque la implementación por defecto en `Potential` lo hace, pero la clase aparece en la tabla de interfaces como capaz de escalar (hereda `Scalable`), lo cual es engañoso.
- **Desperdicio de memoria:** el constructor de `GTablePotential` llama a `super(variables, null)`, que reserva `values = new double[tableSize]` con `tableSize` calculado a partir de las dimensiones —aunque `values` jamás se use—.

- **Confusión para el lector del código:** cualquier método que recibe `TablePotential` puede recibir inadvertidamente una `GTablePotential` con `values.length > 0` (todos ceros) y `elementTable` no nulo, lo que puede producir cálculos numéricamente incorrectos sin lanzar ninguna excepción.

La cambio de superclase de `GTablePotential` a `AbstractIndexedPotential` eliminaría estas anomalías y haría explícita la intención del diseño: un contenedor genérico indexado que no es un potencial numérico.

25.2. Bloqueadores del cambio de superclase

La relación de herencia `GTablePotential` extends `TablePotential` está explotada en tres lugares de la cadena de tipos del análisis de coste-efectividad.

25.2.1. Bloqueador B1: Tipo de retorno de `DANInference.getUtility()`

```

1 // DANInference.java (modulo costEffectiveness)
2 private TablePotential utility;           // se asigna como GTablePotential<CEP>
3
4 public TablePotential getUtility() {
5     return utility;
6 }

```

Listado 41: Campo y método afectados en `DANInference`

La variable `utility` se instancia como `GTablePotential<CEP>` pero se declara como `TablePotential`. Si `GTablePotential` dejara de extender `TablePotential`, la asignación `this.utility = new GTablePotential<>(...)` ya no compilaría.

25.2.2. Bloqueador B2: Cast en `DANCEAnalysis`

```

1 // DANCEAnalysis.java
2 GTablePotential<CEP> utility =
3     (GTablePotential<CEP>) inferenceProcess.getUtility();

```

Listado 42: Cast a `GTablePotential` en `DANCEAnalysis`

Si el tipo de retorno de `getUtility()` cambiara a `Potential`, el compilador rechazaría el cast como “inconvertible types”: no existe ninguna relación de tipo entre `Potential` y `GTablePotential` que garantice que el cast es correcto en tiempo de compilación.

25.2.3. Bloqueador B3: Listas `List<TablePotential>` en `TablePotentialMerge`

```

1 // TablePotentialMerge.java
2 public static TablePotential merge(List<TablePotential> potentials) { ... }
3
4 // CEDecisionTreeNode.java
5 public void setOnlyValueForUtility(TablePotential utility) {
6     // ... cast interno a (GTablePotential) ...
7 }

```

Listado 43: Parámetro de tipo en `TablePotentialMerge` y `CEDecisionTreeNode`

En el flujo CE, estos métodos reciben listas o parámetros que contienen instancias de `GTablePotential`. Si `GTablePotential` deja de extender `TablePotential`, los sitios de llamada dejan de compilar.

La figura 12 ilustra estas dependencias de tipo y los tres bloqueadores.

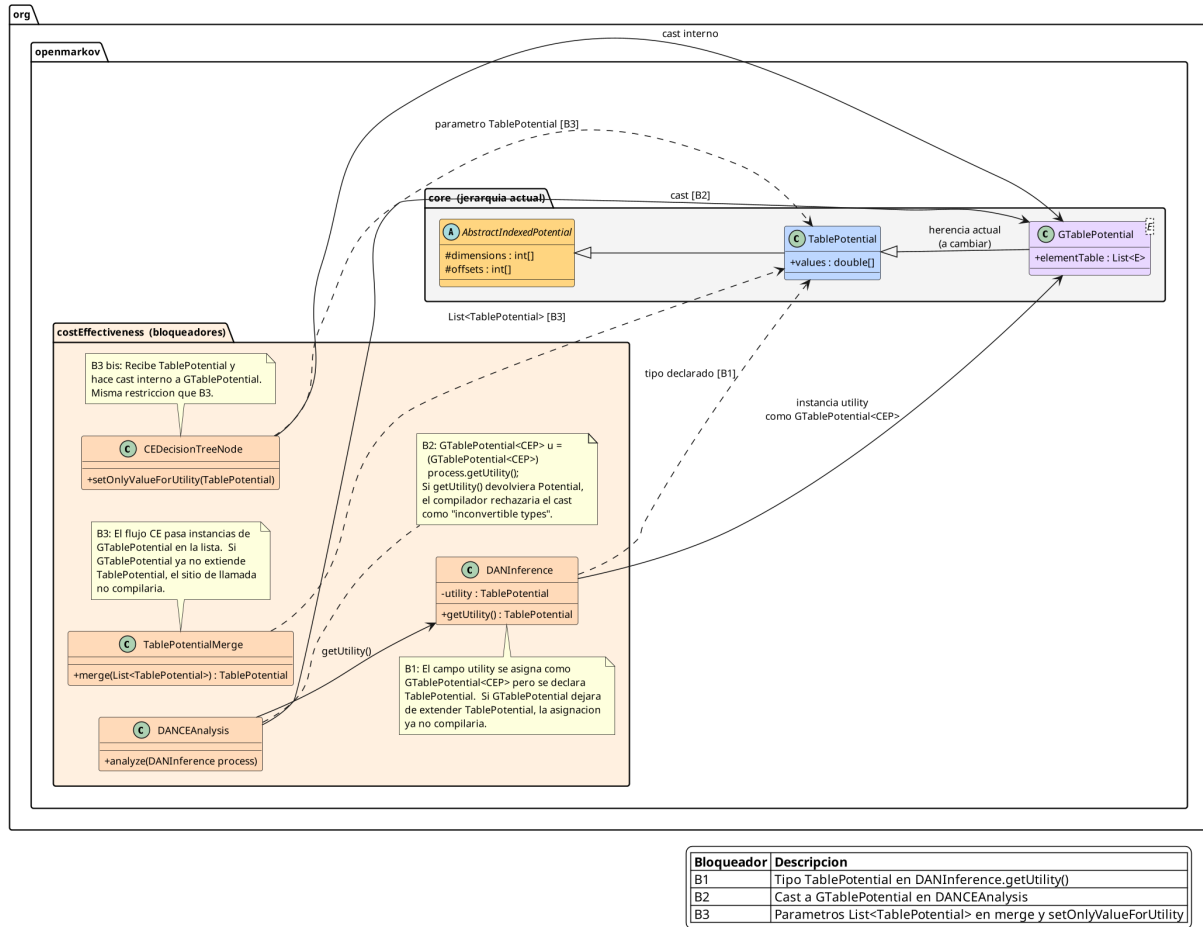


Figura 12: Cadena de tipos en el análisis de coste-efectividad que bloquea el cambio de superclase de GTablePotential. Los tres bloqueadores B1, B2 y B3 aparecen en clases distintas del módulo costEffectiveness.

25.3. Opción A: generalizar los tipos a Potential

La Opción A resuelve los bloqueadores cambiando los tipos declarados en la cadena CE para que usen Potential en lugar de TablePotential:

```

1 // DANInference.java
2 private Potential utility; // antes: TablePotential
3 public Potential getUtility() { ... } // antes: TablePotential
4
5 // DANCEAnalysis.java (sin cambio, el cast sigue siendo válido)
6 GTablePotential<CEP> u = (GTablePotential<CEP>) process.getUtility();
7
8 // TablePotentialMerge.java
9 public static Potential merge(List<Potential> potentials) { ... }
10 // antes: List<TablePotential>

```

Listado 44: Cambios en la Opción A: generalizar a Potential

Esta opción requiere muy pocos cambios de código, pero tiene implicaciones de diseño problemáticas:

- **Pérdida de tipo explícito en la API:** getUtility() devolvía un tipo que garantizaba la posibilidad de hacer operaciones numéricas sobre valores dobles. Con Potential, el código

cliente debe confiar implícitamente en que *de hecho* recibe algo numéricamente manejable, o bien hacer más casts.

- **Regresión potencial en TablePotentialMerge:** al aceptar `List<Potential>`, el método puede recibir cualquier tipo de potencial —incluyendo `UniformPotential` o `DeltaPotential`— para los que el algoritmo de *merge* no está definido. El error solo se manifestaría en tiempo de ejecución.
- **Inconsistencia con los objetivos del Rediseño 3:** el objetivo central era hacer los tipos más precisos, no generalizarlos. La Opción A iría en dirección contraria en estos puntos de la API.

25.4. Opción B: nueva interfaz CEUtilityPotential

La Opción B introduce una nueva interfaz de dominio —`CEUtilityPotential`— que captura el contrato común entre `TablePotential` y `GTablePotential` en el contexto del análisis de coste-efectividad. Ambas clases implementan esta interfaz; la cadena CE la usa como tipo de referencia en lugar de `TablePotential`:

```
1 /**
2  * Marca un potencial que puede actuar como utilidad en el analisis
3  * de coste-efectividad de DANs. Implementada por TablePotential
4  * y GTablePotential<E>. Actua como lista blanca de tipos validos
5  * en la cadena CE, previniendo que tipos inadecuados lleguen a los
6  * algoritmos de fusion (TablePotentialMerge) o de arbol (CEDecisionTreeNode).
7  */
8 public interface CEUtilityPotential {
9     // Interfaz marcadora; puede ampliarse con metodos compartidos
10     // (getDimensions(), getOffsets()) si se necesita acceso indexado
11     // sin cast posterior.
12 }
```

Listado 45: Nueva interfaz `CEUtilityPotential` (Opción B)

```
1 // GTablePotential: superclase cambiada a AbstractIndexedPotential
2 public class GTablePotential<E> extends AbstractIndexedPotential
3     implements Reorderable, CEUtilityPotential { ... }
4     // Ya no hereda Projectable ni Scalable de TablePotential
5
6 // TablePotential: anade CEUtilityPotential a sus interfaces existentes
7 public class TablePotential extends AbstractIndexedPotential
8     implements Projectable, Reorderable, Scalable, CEUtilityPotential { ... }
```

Listado 46: Cambios necesarios en el módulo core (Opción B)

```
1 // DANInference.java -- B1 resuelto
2 private CEUtilityPotential utility;
3 public CEUtilityPotential getUtility() { return utility; }
4
5 // DANCEAnalysis.java -- B2 resuelto
6 // El cast es valido porque GTablePotential implementa CEUtilityPotential
7 GTablePotential<CEP> u = (GTablePotential<CEP>) process.getUtility();
8
9 // TablePotentialMerge.java -- B3 resuelto
10 public static CEUtilityPotential merge(
11     List<CEUtilityPotential> potentials) { ... }
```

Listado 47: Cambios en la cadena CE (Opción B)

La figura 13 muestra el diseño completo de la Opción B.

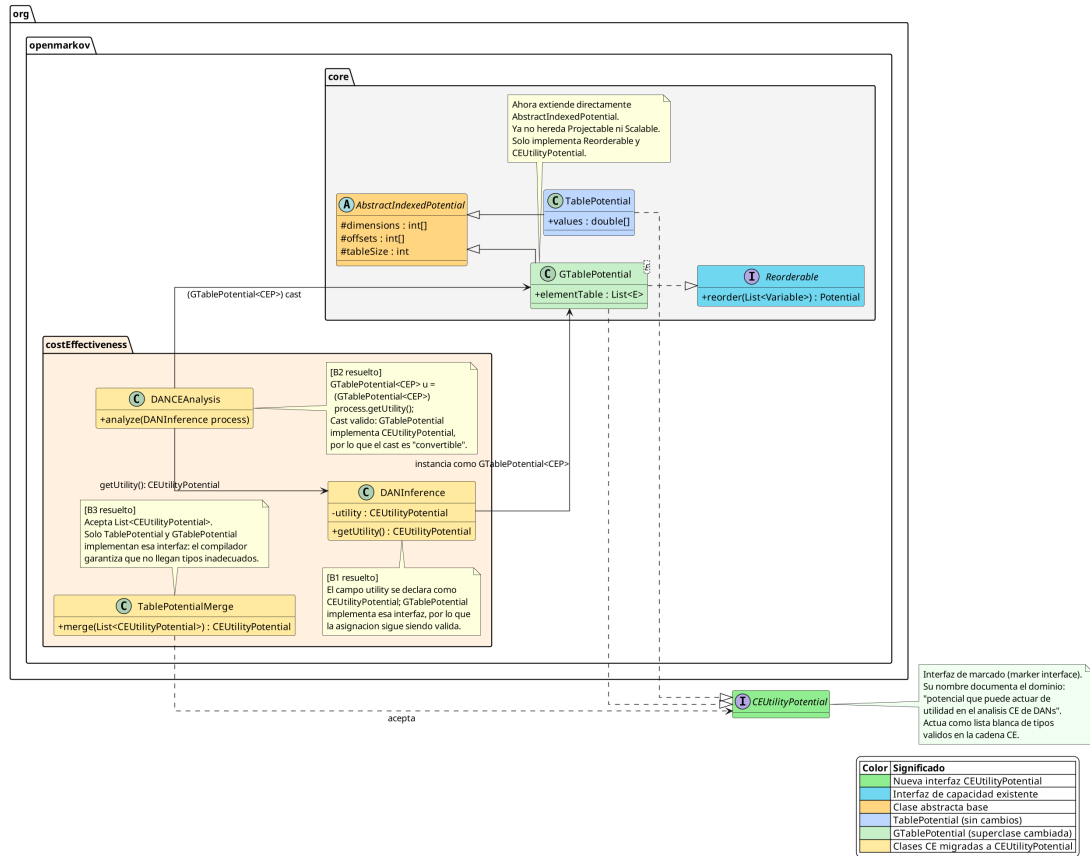


Figura 13: Opción B: la interfaz `CEUtilityPotential` permite cambiar la superclase `GTablePotential` a `AbstractIndexedPotential` sin perder tipo en la cadena CE. `TablePotential` y `GTablePotential` implementan la nueva interfaz; los tres bloqueadores B1, B2, B3 quedan resueltos con cambios localizados.

25.5. Comparación entre opciones

Criterio	Opción A (generalizar a <code>Potential</code>)	Opción B (nueva interfaz <code>CEUtilityPotential</code>)
Mantenibilidad	Baja: <code>getUtility()</code> devuelve <code>Potential</code> , lo que obliga a futuros desarrolladores a conocer implícitamente que el objeto es en realidad un <code>GTablePotential</code>	Alta: la interfaz <code>CEUtilityPotential</code> documenta explícitamente qué tipos son válidos en la cadena CE
Inteligibilidad	Baja: usar <code>Potential</code> donde la semántica requiere un objeto indexado genera confusión; el contrato implícito no es visible en la firma	Alta: la interfaz es autodocumentada; su nombre expresa tanto el dominio (<i>coste-efectividad</i>) como el rol (<i>utilidad</i>)
Elegancia	Media: pocos cambios, pero el resultado es una API más laxa que la original, contradiciendo los objetivos del Rediseño 3	Alta: coherente con el patrón de interfaces de capacidad ya introducido; un nuevo tipo de dominio, no una generalización hacia <i>cualquier</i> potencial
Riesgo de regresión	Alto: <code>TablePotentialMerge</code> con <code>List<Potential></code> puede recibir tipos inesperados y fallar en tiempo de ejecución sin error de compilación	Bajo: solo <code>TablePotential</code> y <code>GTablePotential</code> implementan <code>CEUtilityPotential</code> ; el compilador garantiza que no llegan tipos inadecuados
Volumen de cambios	Muy bajo: 3–4 líneas modificadas	Bajo: 1 fichero nuevo de interfaz + 4–5 líneas modificadas

Tabla 7: Comparación entre la Opción A (generalizar tipos a `Potential`) y la Opción B (nueva interfaz `CEUtilityPotential`) para resolver los bloqueadores de la cambio de superclase de `GTablePotential`.

25.6. Decisión: Opción B

Se adopta la **Opción B** por las siguientes razones:

1. **Coherencia con el Rediseño 3:** el patrón de interfaces de capacidad ya estabilizado hace natural introducir una interfaz de dominio específico (`CEUtilityPotential`) en lugar de generalizar a `Potential`. Añadir una interfaz es exactamente el tipo de cambio que el Rediseño 3 promueve.
2. **Seguridad de tipos en tiempo de compilación:** la interfaz actúa como una *lista blanca* de tipos válidos para la cadena CE. Ninguna subclase de `Potential` puede introducirse accidentalmente en `TablePotentialMerge` sin declarar explícitamente que implementa `CEUtilityPotential`.
3. **Bajo riesgo de regresión:** los cambios se concentran en `DANInference`, `DANCEAnalysis`, `TablePotentialMerge` y `CEDecisionTreeNode`. El algoritmo de cálculo no cambia; solo cambian los tipos declarados de las variables.
4. **Documentación implícita:** el nombre `CEUtilityPotential` comunica en el código fuente el dominio de uso y el rol del objeto, facilitando la comprensión a futuros desarrolladores sin necesidad de consultar documentación externa.

La implementación de la Opción B se planifica como **Fase 5 del Rediseño 3**, una vez que los cuatro pasos ya ejecutados hayan sido validados en un ciclo de integración completo.

26. Encapsulación del campo `values` en `TablePotential`

26.1. Situación anterior

El campo que almacena la tabla numérica de un potencial discreto era **público y volátil**:

```
1 /**
2  * Table storing the numerical values of the potential. This attribute is
3  * public for efficiency and volatile for efficiency in concurrent operations.
4  */
5 public volatile double[] values;
```

Listado 48: Campo `values` antes de la encapsulación.

El campo `values` era accedido directamente en **135 puntos** del proyecto, en módulos tan variados como `inference`, `gui`, `io` y `learning`. El comentario del código original justificaba la visibilidad pública con un argumento de rendimiento, y la palabra clave `volatile` con un argumento de seguridad en entornos concurrentes.

26.2. Problemas del diseño anterior

El campo público viola la encapsulación (antipatrón *Aliasing Bug*). Un campo de array público mutable es una fuente clásica de *aliasing bugs* (§2.2). Cualquier fragmento de código que escriba `pot.values` obtiene una referencia directa al array interno. Si ese código almacena la referencia en una variable local y después modifica el array a través de esa variable, la modificación se refleja silenciosamente en el potencial original, sin que la clase pueda detectarlo ni impedirlo.

En OpenMarkov este riesgo era real: los algoritmos de inferencia copian potenciales, los transforman y los almacenan. Si en algún momento dos variables apuntaban al mismo array, una operación sobre una de ellas modificaba el estado del otro potencial.

`volatile` no protege los contenidos del array. La palabra clave `volatile` en Java garantiza que **la referencia** al array se lee y escribe de forma atómica y que los cambios son visibles entre hilos [4]. Sin embargo, `volatile` no protege los *contenidos* del array: dos hilos pueden leer la misma referencia y acceder a posiciones distintas del array sin ninguna exclusión mutua.

El comentario del código original afirmaba que `volatile` era necesario para la concurrencia, pero eso era incorrecto: la declaración creaba una **falsa sensación de seguridad**. El array podría corromperse concurrentemente aunque la referencia fuera volátil. La anotación correcta habría requerido un `AtomicReferenceArray` o bien sincronización explícita, ninguna de las cuales era práctica dado el uso del campo.

La visibilidad pública impide cambiar la representación. Mientras el campo sea público, cualquier cambio en el tipo o la estructura interna de la tabla (por ejemplo, almacenarla como `float[]` en lugar de `double[]` para reducir memoria, o como una lista dispersa para potenciales con muchos ceros) obligaría a modificar los 135 sitios que acceden directamente al campo. Con un campo privado y un par de métodos, ese cambio sería transparente para todos los llamadores.

26.3. La encapsulación: `private` + `getValues()` + `setValues()`

El cambio consiste en hacer el campo privado y canalizar todo acceso a través de dos métodos:

```

1 /**
2  * Table storing the numerical values of the potential.
3  * Use getValues() for read access and setValues(double[]) for bulk assignment.
4  */
5 private double[] values;
6
7 /**
8  * Returns the internal values array directly (live reference, not a copy).
9  * Callers must not assign a new array to the returned reference; use
10 * setValues(double[]) for bulk replacement.
11 */
12 public double[] getValues() {
13     return values;
14 }
15
16 /**
17  * Replaces the values array.
18  *
19  * @param newValues new values array
20  */
21 public void setValues(double[] newValues) {
22     this.values = newValues;
23 }

```

Listado 49: Campo values después de la encapsulación.

Nótese que `getValues()` devuelve la referencia interna, no una copia. Esto es deliberado: copiar el array en cada acceso sería prohibitivo para los algoritmos de inferencia, que leen y escriben el array en bucles internos. El Javadoc advierte explícitamente de que el llamador no debe reasignar la referencia devuelta. Lo que se gana no es inmutabilidad completa, sino **encapsulación del punto de sustitución**: si alguien quiere reemplazar el array por otro, debe llamar a `setValues()`, que es el único lugar donde se puede añadir lógica futura (auditoría, invalidación de cachés, etc.).

26.4. Por qué `setValues()` no valida el tamaño

La primera versión de `setValues()` incluía una comprobación de tamaño aparentemente razonable:

```

1 // Version descartada -- rompe SystematicSampling
2 public void setValues(double[] newValues) {
3     if (newValues.length != tableSize) {
4         throw new IllegalArgumentException(
5             "Expected " + tableSize + " values, got " + newValues.length);
6     }
7     this.values = newValues;
8 }

```

Listado 50: Validación de tamaño que se consideró y se descartó.

Añadir esa validación parece una buena práctica de programación defensiva: si alguien intenta asignar un array de tamaño incorrecto, el error se detecta inmediatamente en lugar de producir resultados silenciosamente erróneos.

Sin embargo, la validación rompía el módulo de análisis de sensibilidad (`SystematicSampling`). Ese algoritmo realiza múltiples ejecuciones de inferencia variando un parámetro y almacena los resultados de todas las muestras en un **único array sobredimensionado**: si el potencial tiene $\text{tableSize} = n$ y hay k muestras, el array almacenado tiene longitud $k \cdot n$. El array sobredimensionado es la estructura interna elegida para evitar la creación de k potenciales distintos. Es un uso deliberado y válido, pero viola el supuesto invariante `values.length == tableSize`.

Esta situación ilustra una lección importante de diseño: **no añadir validación para supuestos invariantes que no son invariantes universales**. El contrato de `setValues()` es simplemente “reemplaza el array interno”; no establece ninguna relación entre el tamaño del nuevo array y `tableSize`. Añadir esa restricción sería sobredefinir el contrato en perjuicio de usos legítimos.

La validación correcta, si se desea, debe hacerse en los lugares donde sí se conoce el contrato preciso —por ejemplo, en los constructores que inicializan el array con `tableSize` posiciones, o en los algoritmos que dependen de la alineación entre `values` y `tableSize`— y no en el método de acceso genérico.

26.5. Migración y efectos secundarios

La migración de los 135 accesos directos al campo fue realizada con un script automatizado que sustituyó `pot.values` por `pot.getValues()` y `pot.values = x` por `pot.setValues(x)`.

El script introdujo inadvertidamente un error en la clase `Choice`: su propio método `setValues(int[])` contenía la asignación `this.values = values` (sobre el campo `int[] values` de `Choice`, no sobre `TablePotential`). El script transformó esa línea en `this.setValues(values)`, generando una **recursión infinita** que terminaba en `StackOverflowError`. El error se detectó al ejecutar los tests y se corrigió restaurando la asignación directa al campo:

```

1 // Antes (error introducido por el script de migracion):
2 public void setValues(int[] values) {
3     this.setValues(values); // llamada recursiva infinita
4     numValues = values.length;
5     initialized = true;
6 }
7
8 // Despues (corregido):
9 public void setValues(int[] values) {
10    this.values = values; // asignacion directa al campo propio de Choice
11    numValues = values.length;
12    initialized = true;
13 }

```

Listado 51: Corrección de la recursión infinita en `Choice.setValues()`.

Este incidente es un ejemplo de por qué las migraciones automatizadas a gran escala deben ir acompañadas de una suite de tests completa: el error se manifestó como un fallo de test inmediato y claro (`StackOverflowError`), lo que permitió localizarlo y corregirlo sin dificultad.

26.6. Resultado

Aspecto	Antes	Después
Visibilidad del campo	<code>public volatile</code>	<code>private</code>
Acceso de lectura	directo (<code>pot.values</code>)	<code>pot.getValues()</code>
Acceso de escritura	directo (<code>pot.values = x</code>)	<code>pot.setValues(x)</code>
Protección concurrente	falsa (<code>volatile</code> solo protege la referencia)	sin cambio (pendiente)
Sitios de acceso directo	135	0
Validación de tamaño	ninguna	ninguna (deliberado: <code>SystematicSample</code>)
Bugs introducidos	—	1 (<code>Choice</code> , corregido en el mismo commit)

La encapsulación no resuelve el problema de seguridad en entornos concurrentes —eso requeriría una refactorización más profunda— pero sí elimina el riesgo de *aliasing bugs* a través de asignaciones directas al campo y abre la puerta a añadir lógica en el único punto de acceso en el futuro.

Es el primer paso de la cadena encapsulación $\rightarrow$ abstracción $\rightarrow$ sustitución de representación, sin comprometer la compatibilidad con el código existente.

Parte IV

Diseño ideal desde cero

27. Motivación

Las cinco opciones de rediseño analizadas en la Parte I parten de la restricción de mantener la compatibilidad con el código existente. Esa restricción es sensata para una migración real, pero limita la exploración del espacio de diseño: las decisiones históricas condicionan cada opción.

Esta parte elimina esa restricción y plantea la pregunta: *¿cómo diseñaríamos el sistema de potenciales si pudiéramos empezar de cero, conociendo todas las necesidades actuales de OpenMarkov?*

El objetivo no es proponer una reescritura total —sería impracticable— sino identificar el diseño *diana* hacia el que las refactorizaciones incrementales deberían converger. Cada decisión de diseño en una refactorización concreta puede evaluarse contra este diseño ideal: ¿nos acerca a él o nos aleja?

28. Requisitos del sistema de potenciales

Para diseñar desde cero es necesario listar explícitamente los requisitos que el sistema de potenciales debe satisfacer. Estos requisitos provienen del análisis del código actual, de las necesidades de los algoritmos de inferencia, y de los casos de uso soportados por OpenMarkov.

28.1. Requisitos funcionales

- RF1 Representación polimorfa.** El sistema debe soportar múltiples formas de definir un potencial: tablas explícitas de probabilidades o utilidades, distribuciones uniformes, deterministas (delta), modelos canónicos (noisy-OR, noisy-MAX), árboles de decisión algebraicos (ADD), gaussianas condicionales, modelos lineales generalizados (GLM), funciones simbólicas, y potenciales de producto/suma para supervalores en diagramas de influencia.
- RF2 Proyección a tabla discreta.** Toda representación debe poder convertirse en una tabla discreta de valores `double[]` dada una evidencia, porque los algoritmos de inferencia exacta (eliminación de variables, propagación Hugin) operan exclusivamente sobre tablas discretas. La proyección es la operación que convierte un potencial arbitrario en la representación de trabajo del motor de inferencia.
- RF3 Operaciones de inferencia sobre tablas.** Sobre las tablas discretas se deben soportar al menos: multiplicación, suma, marginalización (suma sobre un subconjunto de variables), multiplicación-y-marginalización combinada, normalización (condicional y conjunta), división, maximización (para resolución de decisiones), y reordenamiento de variables.
- RF4 Distinción semántica por rol.** Las tablas discretas tienen semánticas distintas según su rol: probabilidad condicional, probabilidad conjunta, utilidad, política de decisión, restricción de enlace. Cada rol implica comportamiento diferente en la inicialización (`setUniform`), la normalización, y la inducción de hallazgos (*induced findings*).
- RF5 Análisis coste-efectividad.** El sistema debe soportar tablas genéricas `GenericTable<E>` cuyos elementos no son `double` sino objetos compuestos (particiones coste-efectividad, CEP). Estas tablas comparten la lógica de indexación (dimensiones, offsets) con las tablas discretas pero tienen operaciones propias (álgebra CE).

RF6 Metadatos de inferencia. Los resultados de la resolución de diagramas de influencia pueden incluir *árboles de estrategia* (`StrategyTree[]`) que codifican la política óptima de intervención. Los análisis de sensibilidad pueden incluir distribuciones de incertidumbre (`UncertainValue[]`) sobre cada valor de la tabla. Ambos son ortogonales al dato principal (`double[]`) y a la semántica de rol.

RF7 Redes temporales. Potenciales como `SameAsPrevious` y `CycleLengthShift` representan relaciones temporales entre rodajas de una red dinámica. Deben poder participar en la proyección usando el contexto de las tablas ya proyectadas.

28.2. Requisitos no funcionales

RNF1 Rendimiento en inferencia. Las operaciones sobre tablas discretas (especialmente `multiplyAndMarginalize`) son el cuello de botella de la inferencia. El diseño no debe introducir indirecciones, copias defensivas ni creación de objetos innecesarios en el camino crítico. El acceso directo al array `double[]` desde los algoritmos de operación debe ser posible sin penalización.

RNF2 Extensibilidad: nuevos tipos de potencial. Añadir un nuevo tipo de representación (p. ej. una distribución de Weibull, un modelo de regresión logística) no debe requerir modificar las clases existentes ni los algoritmos de inferencia: basta con implementar la interfaz de proyección.

RNF3 Extensibilidad: nuevas operaciones. Añadir una nueva operación sobre tablas (p. ej. serialización binaria, evaluación en GPU) no debe requerir modificar la jerarquía de potenciales.

RNF4 Testabilidad. Cada componente debe ser testable de forma aislada: las representaciones, las proyecciones, las operaciones sobre tablas, y los metadatos. No deben existir dependencias cíclicas que impidan construir un componente sin los demás.

RNF5 Seguridad ante errores de tipo. El sistema de tipos de Java debe impedir, en la medida de lo posible, los errores que actualmente se manifiestan como `ClassCastException` o `NullPointerException` en tiempo de ejecución. Si una operación no tiene sentido para un tipo de potencial, el compilador debe rechazarla.

29. Arquitectura propuesta: tres capas

El diseño propuesto se organiza en tres capas con responsabilidades claras. La figura 14 muestra la arquitectura completa.

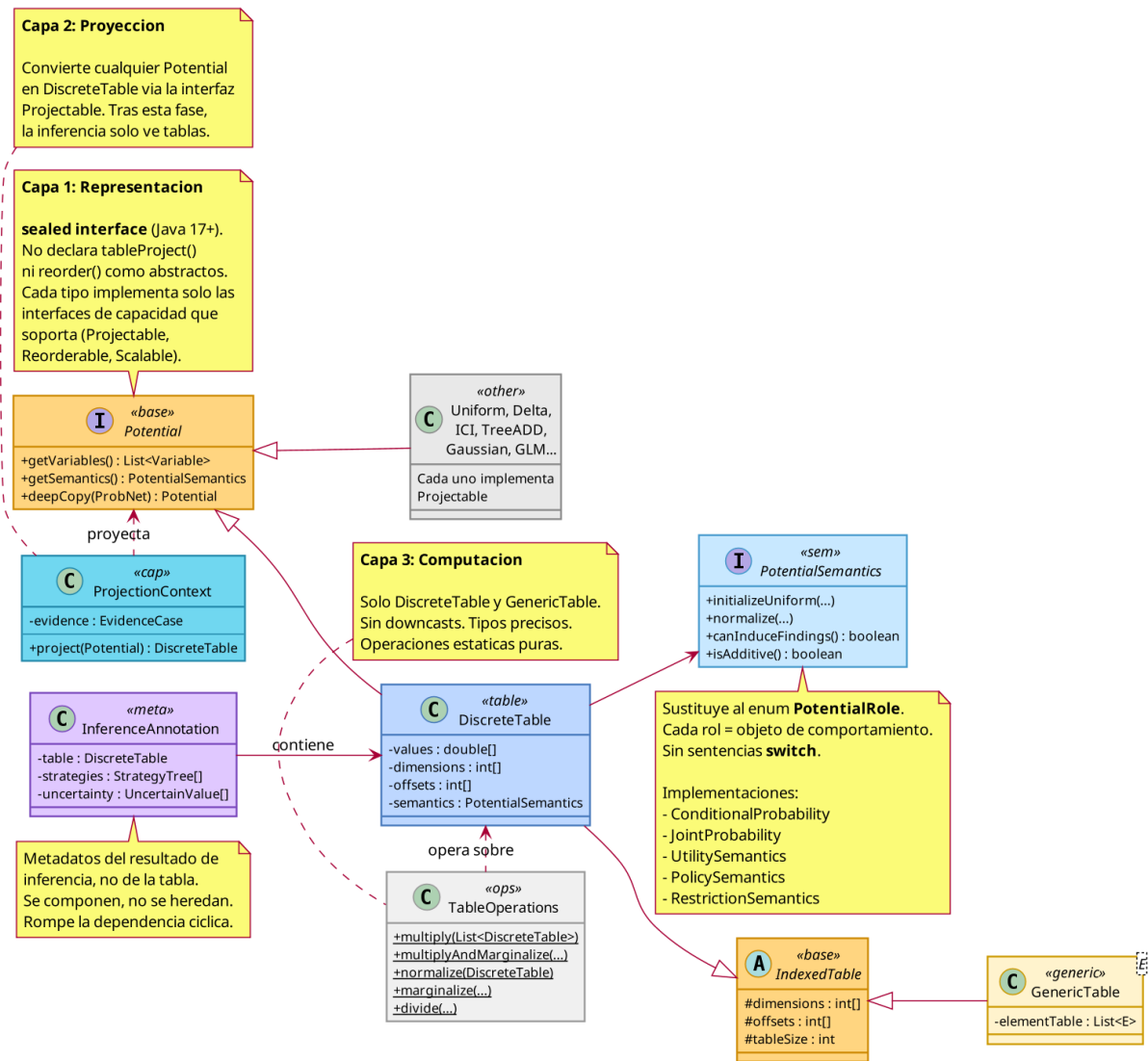


Figura 14: Arquitectura ideal en tres capas. La capa de representación modela la diversidad de potenciales. La capa de proyección los convierte a tablas discretas. La capa de computación opera uniformemente sobre tablas. Los metadatos (estrategias, incertidumbre) se componen externamente. El rol semántico (PotentialSemantics) encapsula el comportamiento variable de cada rol como un objeto con métodos propios, eliminando los switch sobre el enum.

29.1. Capa 1: Representación

La capa de representación modela la diversidad de formas en que el usuario o el fichero de entrada puede definir un potencial. Su interfaz raíz es:

```

1 public sealed interface Potential
2     permits DiscreteTable, UniformPotential, DeltaPotential,
3             ICIPotential, TreeADDPotential, GaussianPotential,
4             GLMPotential, SumPotential, ProductPotential,
5             SameAsPrevious, CycleLengthShift {
6
7     List<Variable> getVariables();
8     PotentialSemantics getSemantics();
9     Potential deepCopy(ProbNet net);
10 }

```

Listado 52: Interfaz raíz de la capa de representación

Decisiones clave:

- **Potential** es una **sealed interface** (Java 17+), no una clase abstracta. Esto aporta dos ventajas:

1. El compilador conoce todos los subtipos en tiempo de compilación, lo que habilita *pattern matching* exhaustivo en sentencias **switch** (Java 21+):

```
1 switch (potential) {  
2     case DiscreteTable dt -> projectTable(dt, evidence);  
3     case UniformPotential u -> projectUniform(u, evidence);  
4     case DeltaPotential d -> projectDelta(d, evidence);  
5     // ... el compilador rechaza si falta un caso  
6 }
```

2. Impide que módulos externos creen subclases no previstas, manteniendo el control del polimorfismo.

- **Potential** no declara **tableProject()**, **reorder()** ni **scalePotential()** como métodos abstractos. Estas operaciones no son parte del contrato mínimo de todos los potenciales: **reorder()** no tiene sentido para **UniformPotential**; **scalePotential()** no tiene sentido para **DeltaPotential**. En su lugar, las operaciones se modelan como *interfaces de capacidad* (requisitos **RNF2** y **RNF5**).

- Cada tipo concreto implementa exactamente las interfaces de capacidad que soporta:

- **DiscreteTable** implementa **Projectable**, **Reorderable**, **Scalable**.
- **UniformPotential** implementa **Projectable** solamente.
- **DeltaPotential** implementa **Projectable** y **Scalable**.

Si un algoritmo necesita reordenar un potencial, declara el tipo **Reorderable** en su firma; el compilador rechaza en tiempo de compilación cualquier potencial que no soporte esa operación.

29.2. Capa 2: Proyección

La proyección es el puente entre la diversidad de representaciones y la uniformidad que los algoritmos de inferencia necesitan. Su contrato es simple: dado un potencial arbitrario y un caso de evidencia, produce una **DiscreteTable**.

```
1 public interface Projectable {  
2     DiscreteTable projectToTable(EvidenceCase evidence,  
3                                 ProjectionContext context);  
4 }  
5  
6 public final class ProjectionContext {  
7     private final List<DiscreteTable> alreadyProjected;  
8  
9     public DiscreteTable project(Potential potential) {  
10         if (potential instanceof Projectable p) {  
11             return p.projectToTable(evidence, this);  
12         }  
13         throw new NonProjectablePotentialException(potential);  
14     }  
15 }
```

Listado 53: Interfaz de proyección y contexto

Decisiones clave:

- La proyección es la **única operación polimórfica** del sistema. Cada tipo de potencial sabe cómo convertirse a sí mismo en una tabla discreta; los algoritmos de inferencia no necesitan conocer los detalles internos de ninguna representación.
- El `ProjectionContext` proporciona acceso a las tablas ya proyectadas, necesario para tipos como `SameAsPrevious` que referencian resultados anteriores (requisito **RF7**).
- Después de la fase de proyección, toda la diversidad de representaciones ha desaparecido: el motor de inferencia opera exclusivamente con `DiscreteTable` y, en el caso de análisis coste-efectividad, con `GenericTable<CEP>`.

29.3. Capa 3: Computación

La capa de computación contiene las operaciones de inferencia sobre tablas discretas. Estas operaciones son métodos estáticos puros: reciben tablas, devuelven tablas, y no tienen estado interno.

```

1 public final class TableOperations {
2     public static DiscreteTable multiply(List<DiscreteTable> potentials) { ... }
3     public static DiscreteTable marginalize(DiscreteTable p, Variable var) { ... }
4     public static DiscreteTable multiplyAndMarginalize(
5         Collection<DiscreteTable> potentials,
6         List<Variable> variablesToKeep,
7         List<Variable> variablesToEliminate) { ... }
8     public static DiscreteTable normalize(DiscreteTable p) { ... }
9     public static DiscreteTable divide(DiscreteTable a, DiscreteTable b) { ... }
10    public static DiscreteTable sum(List<DiscreteTable> potentials) { ... }
11 }

```

Listado 54: Operaciones de inferencia como funciones estáticas

Decisiones clave:

- Las operaciones reciben y devuelven `DiscreteTable`, no `Potential`. Esto elimina los *down-casts*: si el compilador acepta la llamada, el tipo es correcto (requisito **RNF5**).
- `DiscreteTable` es una clase final con acceso directo al array `values` para las operaciones internas (paquete `operation`). El acceso público es a través de `getValues()`, pero los métodos del mismo paquete acceden al campo directamente para evitar la penalización de copias defensivas en el camino crítico (requisito **RNF1**).
- Las operaciones de coste-efectividad (`CEAlgebra`) operan sobre `GenericTable<CEP>` y usan `DiscreteTable` solo como tabla de probabilidades de entrada. No hay herencia compartida entre `DiscreteTable` y `GenericTable<E>` más allá de la clase abstracta `IndexedTable` que encapsula la lógica de indexación (dimensiones, offsets, posición).
- La normalización delega en el objeto `PotentialSemantics` asociado a la tabla:

```

1 public static DiscreteTable normalize(DiscreteTable table) {
2     double[] result = table.getValues().clone();
3     table.getSemantics().normalize(result, table.getDimensions(),
4                                     table.getVariables());
5     return DiscreteTable.of(table.getVariables(), result,
6                             table.getSemantics());
7 }

```

Esto elimina el `switch` sobre `PotentialRole`: cada semántica sabe cómo normalizarse.

30. Rol como comportamiento: PotentialSemantics

El diseño actual usa el enum `PotentialRole` como *type code*: un campo que controla el comportamiento mediante sentencias `switch` dispersas por todo el código (problema P6). El diseño ideal convierte cada rol en un *objeto de comportamiento* que implementa una interfaz común. La figura 15 muestra la jerarquía de semánticas con el contraste entre el diseño anterior (enum + `switch`) y el nuevo (polimorfismo).

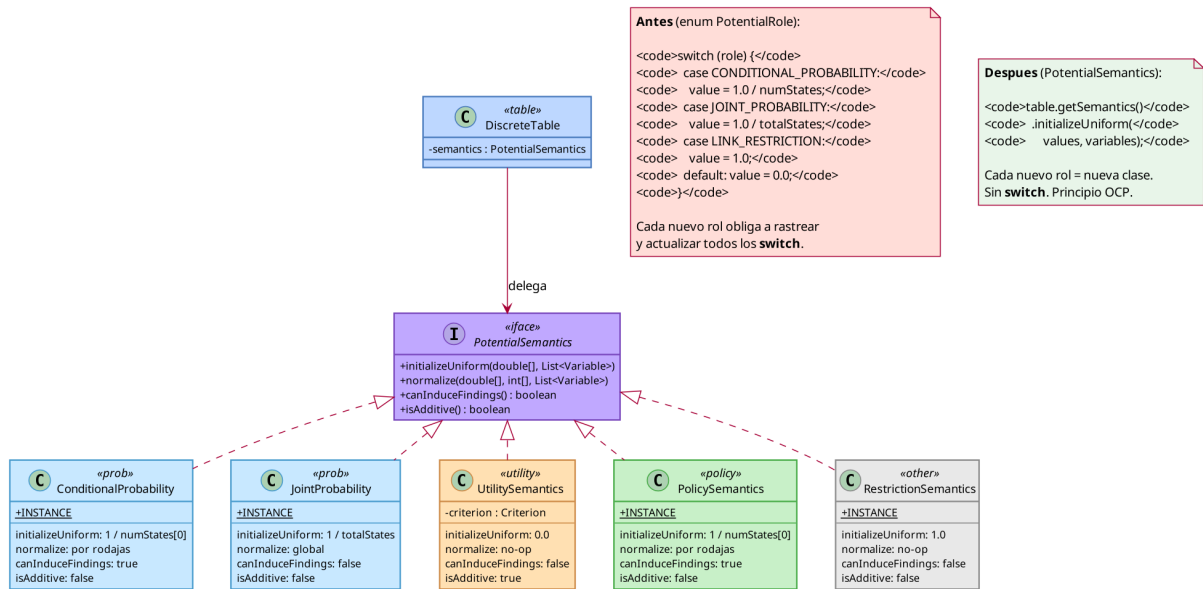


Figura 15: Jerarquía de `PotentialSemantics`. Cada rol del antiguo enum `PotentialRole` se convierte en un objeto de comportamiento con sus propios métodos de inicialización y normalización. Los recuadros rojo y verde contrastan el código *antes* (sentencias `switch` sobre el enum) con el código *después* (llamada polimórfica al objeto de semántica).

```

1 public sealed interface PotentialSemantics
2     permits ConditionalProbability, JointProbability,
3         UtilitySemantics, PolicySemantics, RestrictionSemantics {
4
5     void initializeUniform(double[] values, List<Variable> variables);
6     void normalize(double[] values, int[] dimensions, List<Variable> variables);
7     boolean canInduceFindings();
8     boolean isAdditive();
9 }
10
11 // Singleton: la semantica no tiene estado (excepto UtilitySemantics)
12 public final class ConditionalProbability implements PotentialSemantics {
13     public static final ConditionalProbability INSTANCE = new
14         ↪ ConditionalProbability();
15
16     @Override public void initializeUniform(double[] values, List<Variable> vars)
17         ↪ {
18         double v = 1.0 / vars.get(0).getNumStates();
19         Arrays.fill(values, v);
20     }
21
22     @Override public void normalize(double[] values, int[] dims, List<Variable>
23         ↪ vars) {
24         // normaliza por rodajas del primer variable
25         int sliceSize = dims[0];
26         for (int offset = 0; offset < values.length; offset += sliceSize) {
27             double sum = 0;

```

```

24         for (int i = 0; i < sliceSize; i++) sum += values[offset + i];
25         if (sum > 0) for (int i = 0; i < sliceSize; i++) values[offset + i] /=
            ↪ sum;
26     }
27 }
28 @Override public boolean canInduceFindings() { return true; }
29 @Override public boolean isAdditive() { return false; }
30 }
31
32 public final class UtilitySemantics implements PotentialSemantics {
33     private final Criterion criterion; // lleva el criterio asociado
34
35     @Override public void initializeUniform(double[] values, List<Variable> vars)
36         ↪ {
37         Arrays.fill(values, 0.0);
38     }
39     @Override public void normalize(double[] values, int[] dims, List<Variable>
40         ↪ vars) {
41         // no-op: las utilidades no se normalizan
42     }
43     @Override public boolean canInduceFindings() { return false; }
44     @Override public boolean isAdditive() { return true; }
45 }

```

Listado 55: Interfaz PotentialSemantics y sus implementaciones

Ventajas sobre el enum PotentialRole:

- Cada switch actual se convierte en una llamada polimórfica: `table.getSemantics().normalize(values, dims, vars)`. Si se añade un nuevo rol (p. ej. `RewardSemantics` para MDPs), basta con crear una nueva implementación de `PotentialSemantics`; no es necesario rastrear y actualizar sentencias switch dispersas (principio Abierto/Cerrado).
- `UtilitySemantics` lleva el `Criterion` como campo propio, eliminando el patrón actual donde la identidad de utilidad se codifica como `criterion != null` y el rol se guarda aparte como `PotentialRole.UNSPECIFIED`.
- Las implementaciones de probabilidad (`ConditionalProbability`, `JointProbability`) y política son singletons sin estado, sin coste de memoria por instancia.
- El *pattern matching* exhaustivo de Java 21 sobre la `sealed interface` garantiza que cada nuevo rol se maneje en todos los puntos del código que hacen distinción semántica.

31. Metadatos por composición

En el diseño actual, los metadatos de inferencia (árboles de estrategia y distribuciones de incertidumbre) están integrados en `TablePotential` como campos opcionales (problema P1) o como subclases (`StrategicTablePotential`, `UncertainTablePotential` tras el Rediseño 1).

El diseño ideal reconoce que estos metadatos son **ortogonales** a la tabla de valores y al rol semántico. No son propiedades del potencial en sí, sino *anotaciones del resultado de la inferencia*:

- Los **árboles de estrategia** solo aparecen como resultado de la maximización en la resolución de diagramas de influencia. Son generados por `MaxOutVariable` o `CEAlgebra.ceMaximize` y acompañan al potencial de utilidad resultante.
- Los **valores de incertidumbre** solo aparecen en potenciales cargados desde fichero para análisis de sensibilidad paramétrica. Son consumidos por el muestreo PSA (*Probabilistic Sensitivity Analysis*) y no participan en la inferencia exacta.

En ambos casos, el array `double[] values` y su indexación no cambian. Lo que cambia es el *contexto* que acompaña a esos valores. Por tanto, el diseño ideal modela estos metadatos como contenedores externos que *componen* una tabla, no la extienden:

```

1 // Contenedor para resultados de inferencia con estrategias
2 public final class InferenceAnnotation {
3     private final DiscreteTable table;
4     private final StrategyTree[] strategies; // puede ser null
5     private final UncertainValue[] uncertainty; // puede ser null
6
7     public DiscreteTable getTable()           { return table; }
8     public boolean hasStrategies()            { return strategies != null; }
9     public StrategyTree[] getStrategies()      { return strategies; }
10    public boolean hasUncertainty()            { return uncertainty != null; }
11    public UncertainValue[] getUncertainValues(){ return uncertainty; }
12 }

```

Listado 56: Metadatos como composición externa

Impacto:

- `DiscreteTable` queda libre de campos opcionales. No necesita subclases especializadas para estrategias o incertidumbre. Es una clase final simple y predecible.
- La dependencia cíclica `TablePotential` → `StrategyTree` → `TreeADDPotential` → `TablePotential` (problema P5) desaparece: `DiscreteTable` no referencia `StrategyTree`; es el contenedor `InferenceAnnotation` quien lo hace, y ese contenedor vive en el módulo de inferencia, no en el de representación.
- El constructor de copia de `DiscreteTable` no tiene campos opcionales que copiar incorrectamente (problema P1 eliminado por diseño).
- Las operaciones de inferencia que propagan estrategias (como `multiplyAndMarginalize` con `StrategicTablePotential`) operan sobre `InferenceAnnotation` y su tabla interna, no sobre subclases de `DiscreteTable`.

32. Eliminación de los *downcasts*

El problema P4 (downcasts desprotegidos en inferencia) se resuelve naturalmente con la arquitectura de tres capas:

1. **Antes de la proyección:** los potenciales son polimorfos (`Potential`), pero el único código que los toca es el `ProjectionContext`, que despacha mediante `instanceof` `Projectable` o *pattern matching* sobre la *sealed interface*. No hay *downcasts* manuales.
2. **Después de la proyección:** todo es `DiscreteTable` (o `GenericTable<CEP>` en el camino de coste-efectividad). El módulo de inferencia (`VariableEliminationCore`, `ChanceVariableElimination`, `HuginPropagation`) trabaja directamente con estos tipos concretos. No necesita *downcasts* porque nunca ve un `Potential` genérico.
3. **El bifurcación unicriterio/bicriterio** en el motor de eliminación de variables se expresa con una clase sellada para el tipo de utilidad en juego:

```

1 sealed interface UtilityResult permits DiscreteUtility, CEPUtility { }
2 record DiscreteUtility(DiscreteTable table) implements UtilityResult { }
3 record CEPUtility(GenericTable<CEP> table) implements UtilityResult { }

```

El *pattern matching* reemplaza los `instanceof` `GTablePotential` dispersos por el código actual.

33. Resolución de la dependencia cíclica

El ciclo actual es:

`TablePotential → StrategyTree[] → TreeADDPotential → TablePotential`

En el diseño ideal, el ciclo se rompe en dos puntos:

1. `DiscreteTable` no contiene `StrategyTree[]`. Los árboles de estrategia viven en `InferenceAnnotation`, que está en el módulo de inferencia (no en `core`).
2. `TreeADDPotential` no referencia `DiscreteTable` directamente. Sus hojas implementan `Projectable`: la proyección de un `TreeADDPotential` navega el árbol y proyecta la hoja seleccionada, que devuelve una `DiscreteTable`. La relación es unidireccional.

El grafo de dependencias resultante es un DAG:

`InferenceAnnotation → DiscreteTable`
`InferenceAnnotation → StrategyTree`
`TreeADDPotential → Projectable (interfaz)`

No hay ciclos. Cada componente puede compilarse, testarse y refactorizarse de forma independiente.

34. `GenericTable<E>`: posición limpia en la jerarquía

El diseño actual tiene un problema histórico con `GTablePotential<E>` (problema P3). En el diseño ideal, la relación es clara:

```
1 // Base comun: logica de indexacion (dimensiones, offsets, posicion)
2 public abstract sealed class IndexedTable
3     permits DiscreteTable, GenericTable {
4
5     protected final int[] dimensions;
6     protected final int[] offsets;
7     protected final int  tableSize;
8
9     public int getPosition(int[] coordinates) { ... }
10 }
11
12 // Tabla de valores double: el caballo de batalla de la inferencia
13 public final class DiscreteTable extends IndexedTable
14     implements Potential, Projectable, Reorderable, Scalable { ... }
15
16 // Tabla de objetos genericos: para coste-efectividad (CEP)
17 public final class GenericTable<E> extends IndexedTable {
18     private final List<E> elementTable;
19 }
```

Listado 57: Jerarquía limpia con `IndexedTable` como base

`GenericTable<E>` ya no hereda de `DiscreteTable`: no tiene un array `values[]` inútil, no pasa `null` como rol, y no hereda métodos (`setUniform()`, `normalize()`) que no tienen sentido para una tabla de objetos. La única lógica compartida —la indexación— está correctamente factorizada en `IndexedTable`.

35. Evaluación contra los problemas originales

La tabla 8 resume cómo el diseño ideal resuelve cada uno de los seis problemas identificados en el catálogo (sección 1).

Id	Problema	Cómo lo resuelve el diseño ideal	Estado
P1	God Class	DiscreteTable es final, sin campos opcionales. Estrategias e incertidumbre se componen externamente en InferenceAnnotation.	Resuelto
P2	Métodos forzados	Potential es una sealed interface sin métodos abstractos de operación. Cada tipo implementa solo las interfaces de capacidad que soporta.	Resuelto
P3	GTablePotential	GenericTable<E> hereda de IndexedTable, no de DiscreteTable. No tiene values[] ni PotentialRole.	Resuelto
P4	Downcasts	Después de la proyección, todo es DiscreteTable. La inferencia no necesita <i>downcasts</i> .	Resuelto
P5	Dependencia cíclica	DiscreteTable no contiene StrategyTree[]. TreeADDPotential referencia Projectable, no DiscreteTable.	Resuelto
P6	Type code	PotentialSemantics encapsula el comportamiento de cada rol. No hay sentencias switch sobre un enum.	Resuelto

Tabla 8: Evaluación del diseño ideal contra los seis problemas del catálogo.

36. Evaluación contra los requisitos

Req.	Estado	Cómo se satisface
RF1	Sí	La sealed interface Potential admite tantos permits como representaciones se necesiten.
RF2	Sí	Cada representación implementa Projectable; el ProjectionContext coordina la fase.
RF3	Sí	TableOperations contiene todas las operaciones de inferencia sobre DiscreteTable.
RF4	Sí	PotentialSemantics encapsula el comportamiento de cada rol con polimorfismo.
RF5	Sí	GenericTable<E> comparte IndexedTable con DiscreteTable; CEAlgebra opera sobre GenericTable<CEP>.
RF6	Sí	InferenceAnnotation compone tabla + estrategias + incertidumbre sin herencia.
RF7	Sí	ProjectionContext provee las tablas ya proyectadas para SameAsPrevious y similares.
RNF1	Sí	DiscreteTable.values accesible directamente desde TableOperations (mismo paquete).
RNF2	Sí	Nuevo potencial = nueva clase implements Projectable; nada más cambia.
RNF3	Sí	Nueva operación = nuevo método en TableOperations; la jerarquía no cambia.
RNF4	Sí	Sin ciclos; cada capa testable por separado.
RNF5	Sí	sealed interface + interfaces de capacidad impiden errores de tipo en compilación.

Tabla 9: Evaluación del diseño ideal contra los requisitos funcionales y no funcionales.

37. Impacto en rendimiento

Tiempo: El diseño ideal no introduce ninguna penalización en el camino crítico de la inferencia. Las operaciones de `TableOperations` acceden directamente al array `values[]` de `DiscreteTable` (por visibilidad de paquete), sin copias defensivas ni indirecciones virtuales adicionales respecto al diseño actual. El despacho polimórfico solo ocurre en la fase de proyección (una vez por potencial), no en la fase de computación (miles de veces por variable eliminada).

La sustitución de los `switch` sobre `PotentialRole` por llamadas polimórficas a `PotentialSemantics` tiene un coste despreciable: la JVM moderna resuelve estas llamadas con un despacho monomorfo en los puntos calientes (la semántica concreta es casi siempre la misma dentro de un contexto de inferencia dado).

Memoria: La eliminación de los campos opcionales en `DiscreteTable` ahorra 2 punteros (≈ 16 bytes en JVM de 64 bits con *compressed oops*) por instancia respecto al `TablePotential` original. `GenericTable<E>` no hereda un `double[]` inútil, ahorrando `tableSize * 8` bytes por instancia de tabla genérica. `PotentialSemantics` añade un puntero por instancia de tabla (≈ 8 bytes), pero las semánticas de probabilidad, política y restricción son singletons sin coste adicional.

En conjunto, el impacto neto sobre memoria es una **reducción moderada**, coherente con la Opción 1 y sin las penalizaciones de la Opción 5 (inmutabilidad).

38. Distancia al diseño actual tras los rediseños 1 y 3

Tras la implementación de los Rediseños 1 y 3, el diseño actual ya ha recorrido una parte significativa del camino hacia el diseño ideal:

Aspecto	Implementado	Pendiente
Extracción de campos opcionales a subclases	Sí (Rediseño 1)	—
Interfaces de capacidad (<code>Projectable</code> , etc.)	Sí (Rediseño 3)	—
<code>AbstractIndexedPotential</code> como base de indexación	Sí (Rediseño 1)	Renombrar a <code>IndexedTable</code>
Encapsulación de valores	Sí	—
<code>GTablePotential</code> desacoplado de <code>TablePotential</code>	Parcial	Cambiar superclase (fase 5)
<code>Potential</code> como sealed interface	No	Migración de clase abstracta a interfaz
<code>PotentialSemantics</code> en vez de <code>PotentialRole</code> enum	No	Crear jerarquía de semánticas
Metadatos como composición (<code>InferenceAnnotation</code>)	No	Extraer de subclases a contenedor
Eliminación de <i>downcasts</i> en inferencia	No	Requiere refactorizar módulo inference
<code>UtilityResult</code> sellado para bifurcación uni/bicriterio	No	Requiere refactorizar <code>VariableElimination</code>

Tabla 10: Distancia entre el diseño actual (tras Rediseños 1 y 3) y el diseño ideal. Los aspectos ya implementados son la base sobre la que construir las fases restantes.

38.1. Hoja de ruta sugerida

La transición del diseño actual al ideal puede hacerse en fases incrementales, cada una de las cuales deja el sistema en un estado funcional:

1. **Fase A: Cambio de superclase de `GTablePotential`** (ya planificada como fase 5 del Rediseño 1). `GTablePotential` pasa a extender `AbstractIndexedPotential` en lugar de `TablePotential`. Elimina el `values[]` inútil.
2. **Fase B: Introducir `PotentialSemantics`**. Crear las implementaciones de `PotentialSemantics` (`ConditionalProbability`, `JointProbability`, etc.) y migrar los `switch` en `TablePotential` y `DiscretePotentialOperations` a llamadas polimórficas. `PotentialRole` se mantiene

temporalmente como campo pero se delega en `PotentialSemantics` para el comportamiento.

3. **Fase C: Metadatos como composición.** Introducir `InferenceAnnotation` en el módulo de inferencia. Los algoritmos de maximización (`MaxOutVariable`) producen `InferenceAnnotation` en vez de `StrategicTablePotential`. `StrategicTablePotential` y `UncertainTablePotential` pueden mantenerse para compatibilidad en E/S, pero la inferencia ya no los usa internamente.
4. **Fase D: Tipado fuerte en inferencia.** Refactorizar `VariableEliminationCore`, `ChanceVariableElimination` y `DecisionVariableElimination` para que trabajen con `DiscreteTable` en vez de `TablePotential`. Introducir `UtilityResult` sellado. Eliminar *downcasts*.
5. **Fase E: Potential como sealed interface.** Migrar la clase abstracta `Potential` a una `sealed interface`. Requiere Java 17+ y es la fase de mayor impacto en la API pública, pero a estas alturas la mayoría del código ya trabaja con `DiscreteTable` directamente.

Cada fase es independiente y aporta valor por sí misma: la Fase A limpia `GTablePotential`, la Fase B elimina los `switch`, la Fase C rompe el ciclo de dependencias, la Fase D elimina los *downcasts*, y la Fase E completa la seguridad de tipos. No es necesario completarlas todas para obtener beneficios.

Referencias

- [1] Robert C. Martin, *Agile Software Development: Principles, Patterns, and Practices*, Prentice Hall, 2003.
- [2] Barbara H. Liskov y Jeannette M. Wing, “A Behavioral Notion of Subtyping”, *ACM Transactions on Programming Languages and Systems*, vol. 16, n. 6, pp. 1811–1841, noviembre de 1994.
- [3] Erich Gamma, Richard Helm, Ralph Johnson y John Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1995.
- [4] Brian Goetz, Tim Peierls, Joshua Bloch, Joseph Bowbeer, David Holmes y Doug Lea, *Java Concurrency in Practice*, Addison-Wesley, 2006.